

Big Data Platforms and Analytics

Exam-Ready Revision Notes

Approximately 100 pages of explanations, formulas, comparisons, traps, and answer frames

Every syllabus topic retained

118 exact topic IDs across 13 teaching weeks

Tier A = high-yield core concept; Tier B = supporting/application concept; Tier C = introduction, setup, or recap.

Build date: 8 August 2026

How to Use This Exam Edition

Read the definition first, then reproduce the core explanation without looking. For calculation or architecture questions, write the formula or flow before the prose. Use the red exam-trap line to protect marks, and finish each week with its answer frame. The exact topic badge is also the coverage key used by the audit report.

Coverage and priority. All 118 source lessons are present: Tier A 88, Tier B 16, Tier C 14. Condensation changes depth, not syllabus coverage. Definitions, mechanisms, applicable formulas, examples or contrasts, quick-revision facts, and pitfalls are selected from the edited course notes.

Mark-wise method. For 2 marks: definition + one distinction. For 5 marks: definition + mechanism + example + limitation. For 10 marks: add a diagram or formula, compare alternatives, state metrics, and justify the final decision.

Week Roadmap

Week	Focus	Coverage
Week 1	Big Data Constraints and Scaling	11 topics
Week 2	Distributed Systems Challenges	9 topics
Week 3	The MapReduce Programming Model	10 topics
Week 4	Batch Processing Architectures: Hadoop and Spark	8 topics
Week 5	Spark Core I: RDD Fundamentals	9 topics
Week 6	Spark Core II: Execution Model	9 topics
Week 7	Resilience I: Lineage Mechanisms	9 topics
Week 8	Resilience II: Checkpointing	8 topics
Week 9	Data Partitioning I: Strategies and Structures	10 topics
Week 10	Data Partitioning II: Optimization and Skew	9 topics
Week 11	Distributed ML I: Concepts and Algorithms	9 topics
Week 12	Distributed ML II: Architectures	9 topics
Week 13	Stream Processing Systems	8 topics

Contents

How to Use This Exam Edition	2
Week 1: Big Data Constraints and Scaling	3
Week 2: Distributed Systems Challenges	11
Week 3: The MapReduce Programming Model	18
Week 4: Batch Processing Architectures: Hadoop and Spark	26
Week 5: Spark Core I: RDD Fundamentals	33
Week 6: Spark Core II: Execution Model	40
Week 7: Resilience I: Lineage Mechanisms	47
Week 8: Resilience II: Checkpointing	54
Week 9: Data Partitioning I: Strategies and Structures	61
Week 10: Data Partitioning II: Optimization and Skew	69
Week 11: Distributed ML I: Concepts and Algorithms	76
Week 12: Distributed ML II: Architectures	83
Week 13: Stream Processing Systems	90
Rapid Revision Master Sheet	96

Week 1: Big Data Constraints and Scaling

Week roadmap: Big Data Platforms and Analytics: Course Roadmap → Big Data Constraints and Scaling: Module Overview → The Three Vs of Big Data: Volume, Velocity, and Variety → How the Three Vs Reshape System Architecture → Hardware Constraints: CPU, RAM, and I/O → Why Bigger Boxes Eventually Fail → 5 more topics.

Coverage: 11 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W01-T01 Big Data Platforms and Analytics: Course Roadmap

Tier B

Definition and why it matters: Data volume today exceeds what any single machine can store, process, or serve reliably. The core challenge is not merely storing more bytes - it is designing infrastructure that scales cost-effectively, survives hardware failure, and delivers results at the speed the business demands

Core explanation: Big data forces a shift from monolithic computing to distributed systems: clusters of commodity machines coordinated by specialized software Concept flow: Raw Data at Scale - Infrastructure Constraints - 3 Vs: Volume, Velocity, Variety - Hardware Limits - Distributed Storage & Processing - MapReduce - Spark - Streaming - Distributed ML Topic: Three Vs (Volume, Velocity, Variety); Core Question: What makes data "big"?; Key Outcome: Architectural pressures that break single-node designs Topic: Vertical vs horizontal scaling; Core Question: How do we add capacity?; Key Outcome: Scale-out with commodity hardware clusters Topic: CAP theorem; Core Question: What trade-offs are unavoidable in clusters?; Key Outcome: Choose consistency vs availability under partition Topic: ACID vs BASE; Core Question: How do we guarantee data integrity at scale?; Key Outcome: Strict transactions vs eventual consistency MapReduce is the foundational programming model for batch analytics at petabyte scale: Map: Transform each record independently into key-value pairs Shuffle & Sort: Route all values for the same key to one reducer Reduce: Aggregate values per key into a final result Weakness: Disk-based intermediate state makes iterative and multi-pass jobs slow Spark addresses MapReduce's disk bottleneck through: In-memory computing - up to ~100x faster for iterative workloads RDDs (Resilient Distributed Datasets) - fault-tolerant distributed collections Lazy evaluation - build a DAG of transformations; execute only on action DAG execution engine - optimizes stage boundaries and task scheduling When nodes fail mid-job, systems must recover without restarting from scratch:

Must remember:

- Big data is an infrastructure problem, not just a storage problem
- Course arc: constraints → MapReduce → Spark → resilience → partitioning → distributed ML → streaming
- Week 1-2: three Vs, scaling, CAP theorem, ACID vs BASE
- Week 3: MapReduce map/shuffle/reduce pipeline; disk-based, fault-tolerant
- Weeks 4-6: Spark in-memory RDDs, lazy evaluation, DAG execution
- Weeks 7-8: lineage, checkpointing, failure recovery
- Weeks 9-10: partitioning, skew, salting, broadcast joins
- Weeks 11-12: data/model parallelism, TensorFlow strategies, ring-allreduce
- Week 13: Kafka + Storm for real-time fraud detection and dynamic pricing
- Central theme: trade-offs at every layer - no free lunch in distributed systems

Exam trap: Treating big data as only a size problem - velocity and variety are equally architectural drivers Assuming MapReduce and Spark solve the same problems equally well - Spark wins on iterative/in-memory; MapReduce remains relevant for simple one-pass batch jobs

BD-W01-T02 Big Data Constraints and Scaling: Module Overview

Tier C

Definition and why it matters: Why can't a regular computer handle big data? The instinct to buy a faster CPU or add more RAM works for modest workloads, but big data is defined by growth rate, format diversity, and sheer volume that outpaces what a single machine can absorb - not just a large file on disk

Core explanation: Understanding this distinction is the foundation for every platform covered in this course: Hadoop, Spark, Kafka, and beyond

Must remember:

- Big data = volume + velocity + variety, not just large files
- Single machines hit CPU, RAM, and I/O walls - physics and economics, not bad code
- Vertical scaling: easy but hits price, hardware, and single-point-of-failure walls
- Horizontal scaling: cluster of commodity nodes, linear cost, fault tolerance
- Three Vs map to three architectural shifts: distributed storage, parallel processing, flexible schemas
- This module answers why clusters exist before teaching how to program them

Example or contrast: The three Vs are not marketing labels - they are constraints that force system redesign

Exam trap: Equating big data with file size alone - velocity and variety are independent architectural drivers Assuming vertical scaling is "wrong" - it is appropriate for bounded, small-scale workloads; the trap is using it as the long-term strategy for unbounded growth

BD-W01-T03 The Three Vs of Big Data: Volume, Velocity, and Variety

Tier A

Definition and why it matters: Data becomes "big" when any single dimension - size, speed, or format diversity - overwhelms conventional systems. A terabyte of CSV files is a volume problem; a gigabyte-per-second sensor stream is a velocity problem; mixing images, JSON logs, and relational records is a variety problem. Real platforms face all three simultaneously

Core explanation: Intuition: Volume measures how much data exists. The unit scale shifted from megabytes and gigabytes to terabytes, petabytes, and exabytes as digital activity exploded Unit: Gigabyte (GB); Approximate Size: 10^9 bytes; Context: A 16 GB phone (circa 2015) Unit: Terabyte (TB); Approximate Size: 10^{12} bytes; Context: Large enterprise data warehouse Unit: Petabyte (PB); Approximate Size: 10^{15} bytes; Context: Netflix video catalog, large e-commerce click logs Unit: Exabyte (EB); Approximate Size: 10^{18} bytes; Context: Global internet traffic (annual aggregates) Category: Structured; Examples: SQL tables, billing records, age/name fields; Schema: Fixed, predefined Category: Semi-structured; Examples: JSON, XML, digital prescriptions, lab reports; Schema: Flexible, self-describing Category: Unstructured; Examples: Emails, PDFs, X-rays, MRI videos, voice notes, satellite images; Schema: No inherent schema Intuition: Velocity is not about total size - it is about how fast data arrives and must be processed. Traditional data is a glass of water; big data velocity is a fire hose that never shuts off Intuition: Traditional enterprise data lived in neat rows and columns (structured). Today, roughly 20% of organizational data is structured; 80% is unstructured or semi-structured Concept flow: Volume - Velocity - Variety - Distributed Storage: HDFS, S3 - Parallel & Stream Processing: MapReduce, Spark, Kafka - Flexible Models: NoSQL, Schema-on-Read - Big Data Platform V: Volume; Forces: Storage exceeds one disk; Technology Response: Distributed file systems (HDFS, S3) V: Velocity; Forces: Sequential processing too slow; Technology Response: Parallel processing (MapReduce, Spark) V: Variety; Forces: Fixed schemas reject mixed formats; Technology Response: NoSQL, data lakes, schema-on-read

Must remember:

- Volume: terabytes to exabytes; e-commerce click logs need distributed storage
- Velocity: data in motion; fraud detection needs ~20 ms response, not hourly batch
- Variety: ~20% structured, ~80% unstructured/semi-structured; healthcare mixes all three
- Volume → distributed storage; Velocity → parallel/stream processing; Variety → flexible schemas
- Real big data systems face all three Vs simultaneously
- The three Vs are architectural pressures, not buzzwords

Formula / decision rule: 10^9 ; 10^{12}

Example or contrast: Amazon or Walmart generates click streams, cart events, transactions, and search queries from millions of customers every second. Recording every interaction globally produces data that no single server can store. The solution is a platform that spreads data across thousands of machines - distributed file systems like HDFS or object stores like S3 Processing Mode: Batch (hourly); Latency: Hours; Outcome for Fraud Detection: Thief long gone; losses realized Processing Mode: Real-time stream; Latency: Milliseconds; Outcome for Fraud Detection: Block fraudulent charge instantly

Exam trap: Defining big data as volume only - a high-velocity, low-volume stream (IoT alerts) is still a big data problem Confusing velocity with volume - a petabyte stored slowly is volume; a gigabyte arriving per second is velocity

BD-W01-T04 How the Three Vs Reshape System Architecture

Tier B

Definition and why it matters: Knowing that volume, velocity, and variety exist is insufficient - each V forces a specific architectural shift away from traditional single-machine, single-database designs. These three shifts form the foundation of every big data platform

Core explanation: A single hard drive has finite capacity. When data grows from gigabytes to petabytes, buying a bigger drive stops working. At Netflix scale, storing all video on one server creates a catastrophic single point of failure - one fire destroys the entire catalog Concept flow: Single Super Server: All Petabytes - Node 1: Shard A - Node 2: Shard B - Node 3: Shard C - Node N: Shard ... - old - new Concept flow: 1M Records - Split - Worker 1: 1K records - Worker 2: 1K records - Worker N: 1K records - Merge Results Centralized storage → Distributed storage Instead of one giant vault, data is split into shards (pieces) spread across thousands of standard disks in a network One processor analyzing one million records sequentially might take an hour. One thousand processors working simultaneously can finish in seconds - if the problem decomposes cleanly Approach: Schema-on-write (RDBMS); When Schema Defined: At ingestion; Best For: Stable, structured transactions Sequential processing → Parallel processing Approach: Schema-on-read (Data Lake); When Schema Defined: At query time; Best For: Mixed, evolving, unstructured data Work is split into independent tasks executed concurrently across cluster nodes Traditional relational databases enforce rigid

schemas - every row must fit predefined columns, like ice cube trays requiring identical shapes. A video file, a tweet, and a sensor reading cannot be forced into the same table structure without losing information

Must remember:

- Volume → distributed storage (HDFS/S3); Netflix shards petabytes across thousands of nodes
- Velocity → parallel processing; Uber/Spark splits real-time pricing across workers
- Variety → flexible data models / schema-on-read; Instagram stores mixed post formats raw
- Three shifts: centralized→distributed, sequential→parallel, fixed schema→flexible
- These shifts are the foundation for all subsequent big data platforms
- Each V maps to a distinct architectural pressure - know the pairing for exams

Example or contrast: Petabytes of video content distributed across thousands of commodity disks. No single machine holds the full catalog. Replication (typically 3x) ensures availability even when individual disks fail Impact: HDFS, S3, and object stores replace monolithic storage appliances Calculating ride price requires real-time inputs: traffic, driver locations, weather. A central supercomputer processing requests one at a time cannot respond in milliseconds. Uber uses platforms like Apache Spark to split the computation across thousands of workers solving micro-problems in parallel

Exam trap: Mapping only volume to distributed storage - velocity also demands distributed compute, not just distributed storage Confusing parallel processing with distributed storage - they solve different Vs (velocity vs volume)

BD-W01-T05 Hardware Constraints: CPU, RAM, and I/O

Tier B

Definition and why it matters: The CPU executes computation. For decades, clock speeds increased yearly (Moore's Law). Around 2005, single-core speed gains stalled due to fundamental physics RAM is fast, volatile working memory - the desk where active data sits while the CPU processes it. Adding RAM seems like the obvious fix for larger datasets

Core explanation: The instinct to solve data growth by purchasing the most powerful single computer is logical for small workloads. For big data, three hardware subsystems - CPU, RAM, and I/O - each hit physical and economic walls that no price tag can permanently overcome Limit: Heat / power density; Mechanism: Packing transistors closer generates more heat; chips would melt beyond a threshold; Consequence: Cannot indefinitely increase clock frequency Limit: Speed of light; Mechanism: Electrical signals traverse a chip in finite time; Consequence: Physical ceiling on signal propagation speed Capacity: 512 GB - 1 TB; Relative Cost Behavior: More than 2x price increase Capacity: Standard - high-performance ECC RAM; Relative Cost Behavior: Premium pricing at each tier High-frequency trading firms once paid millions for CPUs 0.1% faster. Today they deploy many cores working in parallel instead of chasing marginal single-core gains Concept flow: CPU Brain - RAM Desk: 10 TB capacity - Cannot feed CPU: fast enough Analogy: One person reads a book at a fixed maximum speed; 100 people read 100 pages simultaneously Implication for big data: Scale out (more CPUs across nodes), not up (one impossibly fast CPU) Component: Hard drive; Analogy: Massive library; Speed Characteristic: Huge capacity, slow access Component: I/O port; Analogy: Tiny checkout window; Speed Characteristic: One book per minute Component: CPU; Analogy: World's fastest reader; Speed Characteristic: Idle, waiting for books Cost does not scale linearly. Doubling RAM on one machine can cost quadruple the price at enterprise tiers 2. Memory wall (bandwidth bottleneck) Access Type: L1 CPU cache; Relative Latency: 1x (baseline)

Must remember:

- CPU speed plateaued ~2005 due to heat density and speed-of-light limits
- Industry shifted to parallel cores across many machines, not faster single cores
- RAM cost grows exponentially; memory bus bandwidth creates a "memory wall"
- I/O (disk/network) is the slowest subsystem - the fundamental bottleneck
- Big data response: clusters + data locality (move computation to data)
- These hardware walls explain why vertical scaling fails and horizontal scaling wins

Exam trap: Believing more RAM always solves big data - the memory wall (bandwidth) and cost curve limit single-node RAM scaling Confusing CPU core count with CPU speed - big data benefits from more cores across nodes, not marginally faster single cores

BD-W01-T06 Why Bigger Boxes Eventually Fail

Tier A

Definition and why it matters: Scaling up by buying increasingly powerful single machines follows a non-linear cost curve - doubling performance can cost 5x or more. Beyond economics, bigger boxes introduce catastrophic reliability risks and hard physical ceilings. This is why cloud providers run millions of cheap servers instead of a few supercomputers

Core explanation: Server Tier: Standard; Relative Power: 1x; Relative Cost: USD 5,000; Cost per Unit Power: Baseline Server Tier: 4x power; Relative Power: 4x; Relative Cost: ~USD 150,000; Cost per Unit Power: 7.5x worse The relationship between power and price is a curve, not a straight line. Each incremental performance gain costs disproportionately more Scaling Model: Vertical; Ceiling: Motherboard slot limits; Growth Path: Buy bigger machine

(expensive, disruptive) AWS, Google Cloud, and Azure operate millions of standard servers because: Scaling Model: Horizontal; Ceiling: Effectively none; Growth Path: Add another standard node (cheap, incremental) Cost of 100 normal servers || Cost of 1 monster server with equal total power This shift from specialized proprietary hardware to commodity hardware is foundational to modern cloud economics A motherboard has finite slots for CPUs, RAM modules, and storage controllers. Once every slot is occupied, vertical scaling stops. The only option is to discard the entire machine and purchase an even larger (exponentially more expensive) replacement Analogy: A bigger box is a locker with fixed compartments. Horizontal scaling is adding another locker to the network whenever storage or compute is needed Vertical scaling is not universally wrong: Small businesses with bounded data and tolerance for downtime Datasets that fit comfortably in single-node memory Short-term capacity fixes before cluster migration Specialized workloads requiring single-node low-latency (some HPC) The trap is using vertical scaling as the long-term strategy for unbounded data growth

Must remember:

- Factor: Cost curve; Bigger Box (Vertical): Exponential; Cluster (Horizontal): Linear
- Factor: Failure impact; Bigger Box (Vertical): Total outage; Cluster (Horizontal): Partial degradation
- Factor: Growth ceiling; Bigger Box (Vertical): Physical slot limits; Cluster (Horizontal): Add nodes indefinitely
- Factor: Hardware type; Bigger Box (Vertical): Specialized, proprietary; Cluster (Horizontal): Commodity, standardized
- Factor: Replacement time; Bigger Box (Vertical): Custom parts, weeks; Cluster (Horizontal): Swap identical unit, hours
- Power-price relationship is a curve: 2x power may cost 5x; 4x power may cost 30x
- Cloud providers use millions of cheap servers because 100 normal machines cost less than 1 monster machine
- Bigger box = single point of failure; one crash stops the entire business
- Motherboard slot limits create a hard ceiling for vertical scaling
- Horizontal scaling: add nodes indefinitely with linear cost
- Vertical scaling works for small/bounded workloads; fails for infinite-growth enterprises

Formula / decision rule: Cost of 100 normal servers || Cost of 1 monster server with equal total power

Example or contrast: If Delta or Emirates ran their reservation system on one super-server and it crashed during a holiday weekend, thousands of flights would be grounded. In big data, a cluster tolerates individual node failure - lose 1 of 1,000 machines and 999 continue operating Resilience property: Failure of one component degrades capacity but does not halt the system

Exam trap: One massive, expensive server running the entire database means one failed power supply, one motherboard short circuit, or one cooling failure stops the entire business Concept flow: Bigger Box: All Business Logic - Total Outage - Node 1 - Node 2 - Node 999 - 999 nodes continue

BD-W01-T07 Vertical Scaling (Scale Up): Deep Dive

Tier A

Definition and why it matters: Vertical scaling - also called scaling up - means making a single existing machine more powerful by upgrading its components: more RAM, faster CPU, larger disk. The software stack remains unchanged; the same application runs on the same box, just with better hardware underneath

Core explanation: Business Growth: Scooter can't carry packages; Vertical Scaling Response: Buy a van Business Growth: Van insufficient; Vertical Scaling Response: Buy a truck Concept flow: Same Application Code - Machine v1: 16 GB RAM - Machine v2: 256 GB RAM - Machine v3: 1 TB RAM Advantage: Simplicity; Explanation: No code changes; application sees one computer Advantage: Low maintenance; Explanation: One OS, one security policy, one machine to patch One driver, one vehicle - progressively more powerful, but still a single unit Advantage: Fast to implement; Explanation: Order bigger hardware, migrate data, done Vertical scaling involves hardware upgrades on the same machine: Advantage: Predictable behavior; Explanation: No distributed coordination, no network partitions Increase RAM capacity (e.g., 16 GB → 256 GB → 1 TB) Upgrade CPU (faster clock, more cores on same motherboard) RAM Upgrade: 16 GB - 32 GB; Approximate Cost Behavior: Modest increase (~USD 20/month cloud) Expand storage (larger or additional disks in same chassis) RAM Upgrade: 1 TB - 2 TB high-performance; Approximate Cost Behavior: ~USD 20,000+ No changes to application code, network topology, or deployment architecture Scenario: Small business, bounded data; Vertical Scaling Fit: Excellent Scenario: Dataset fits in single-node memory; Vertical Scaling Fit: Excellent For a startup with 50 GB of data and no strict uptime SLA, vertical scaling is often the best initial choice Scenario: Short-term capacity fix; Vertical Scaling Fit: Good (temporary) Scenario: Low availability requirements; Vertical Scaling Fit: Acceptable Scenario: Petabyte-scale, always-on enterprise; Vertical Scaling Fit: Poor - must scale out Cost does not scale linearly. Specialized high-performance RAM and enterprise CPUs follow exponential pricing Scenario: Unbounded data growth; Vertical Scaling Fit: Poor - hits all three walls Motherboards have finite slots for CPU sockets and RAM DIMMs. Once every slot is filled, no more capacity can be added without replacing the entire machine - and the next tier is disproportionately expensive Industry practice: Use vertical scaling as a short-term fix

or for specialized single-node tasks. Enterprises with infinite growth requirements eventually must shift to horizontal scaling

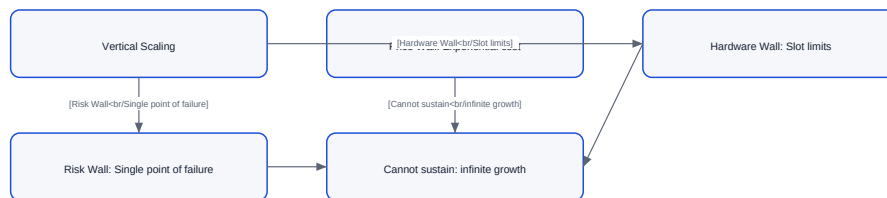
Must remember:

- Vertical scaling = upgrade same machine (RAM, CPU, disk); no code changes
- Advantages: simple, low maintenance, one OS, fast to deploy
- Three walls: price (exponential cost), hardware (slot limits), risk (single point of failure)
- Best for: small businesses, bounded data, short-term fixes
- Fails for: Netflix/Uber/Google-scale infinite growth
- Industry uses vertical scaling as short-term fix; horizontal scaling for long-term big data

Example or contrast: Criterion: Code changes; Scale Up (Vertical): None; Scale Out (Horizontal): Distributed logic required
 Criterion: Cost at scale; Scale Up (Vertical): Exponential; Scale Out (Horizontal): Linear
 Criterion: Failure mode; Scale Up (Vertical): Total outage; Scale Out (Horizontal): Graceful degradation

Exam trap: One super-server, one power supply, one business. If the truck breaks, deliveries stop. For companies like Netflix, Uber, or Google where downtime costs millions per hour, this risk is unacceptable
 Concept flow: Vertical Scaling - Price Wall: Exponential cost - Hardware Wall: Slot limits - Risk Wall: Single point of failure - Cannot sustain: infinite growth

Vertical Scaling (Scale Up): Deep Dive



Vector reconstruction of the source Mermaid visual

Concept map: Vertical Scaling (Scale Up): Deep Dive

BD-W01-T08

Horizontal Scaling (Scale Out): Deep Dive

Tier A

Definition and why it matters: Node: An individual server in the cluster
 Cluster: A group of interconnected nodes acting as one cohesive system

Core explanation: Horizontal scaling - also called scaling out - adds more standard machines (nodes) to a group (cluster) rather than upgrading a single machine. This model enabled Google, Amazon, and Netflix to handle global-scale workloads on affordable infrastructure
 Strategy: Vertical (scale up); Analogy: Buy a bigger and bigger truck;
 System Design: One powerful server Strategy: Horizontal (scale out); Analogy: Hire 10 more drivers with 10 smaller vans;
 System Design: Cluster of standard nodes The system grows by adding units, not by inflating a single unit
 Scaling Model: Vertical; Cost Behavior: Exponential; Example: 1 TB high-end RAM costs USD 20,000+
 Scaling Model: Horizontal; Cost Behavior: Linear; Example: 5 x 2,000 standard servers = 10,000
 Event: One component fails;
 Vertical System: Entire business stops; Horizontal System: 99 of 100 nodes continue
 Event: User experience; Vertical System: Outage; Horizontal System: Uninterrupted (if designed correctly)
 Concept flow: task running - A - M - B
 When data doubles, cost roughly doubles - not explodes. This is how startups grow into giants without upfront supercomputer investment
 Total cluster cost $\sim n \times \text{cost per commodity node}$
 In Hadoop and Spark, the framework detects a dead node and automatically reassigns its work to surviving nodes. The customer watching a movie never notices a glitch
 Horizontal scaling is not free. Trade-offs include: Network management - nodes must communicate reliably
 Coordination overhead - master-worker models, consensus protocols
 Distributed software - CAP theorem trade-offs (Module 2)
 Data consistency - replicas must stay synchronized
 The hardware problem becomes a software coordination problem

Must remember:

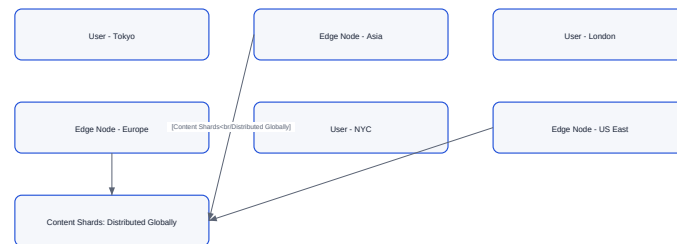
- Horizontal scaling = add more standard nodes to a cluster
- Netflix model: distribute streaming across global edge nodes, not one super-server
- Cost is linear: 5 x 2,000 nodes vs one 50,000 upgrade
- Fault tolerance: 1 of 100 fails → 99 continue; frameworks reassign work
- Complexity tax: network management, coordination, distributed consistency
- Foundation of Hadoop, Spark, and all modern big data platforms
- Enabled Google, Amazon, Netflix to scale on commodity hardware

Formula / decision rule: 2,000 standard servers =; Total cluster cost $\sim n \times \text{cost per commodity node}$

Example or contrast: On a Friday evening, millions of users stream a new release simultaneously. No single computer - regardless of RAM - can serve 200 million concurrent HD streams through one network pipe Concept flow: User - Tokyo - Edge Node - Asia - User - London - Edge Node - Europe - User - NYC - Edge Node - US East - Content Shards: Distributed Globally Solution: Distribute load across thousands of standard servers globally. When a user hits play, they connect to the nearest node (CDN edge server), not a central super-server in California

Exam trap: Assuming horizontal scaling is "simpler" because hardware is cheaper - the software complexity tax is significant Confusing horizontal scaling with load balancing alone - scaling out implies distributed storage and compute, not just multiple web servers behind a balancer

Horizontal Scaling (Scale Out): Deep Dive



Vector reconstruction of the source Mernad visual

Concept map: Horizontal Scaling (Scale Out): Deep Dive

BD-W01-T09 Introduction to Cluster Computing

Tier B

Definition and why it matters: Cluster computing is the practical realization of horizontal scaling: a collection of interconnected, independent computers (nodes) that cooperate to function as a single cohesive system. If vertical scaling strengthens the individual, cluster computing harnesses the power of teamwork

Core explanation: Commodity hardware does not mean cheap or poor quality. It means standardized infrastructure - high-quality, off-the-shelf servers available in every data center, identical and interchangeable Property: Standardized; Benefit: Any node can replace any other Property: Widely available; Benefit: No custom procurement delays Property: Linear pricing; Benefit: Predictable cost per node Property: Rapid replacement; Benefit: Swap failed unit in hours, not weeks Concept flow: Failed Node - Insert Identical: Commodity Server - Cluster Restored Role: Master node; Function: Task scheduling, metadata management, coordination; Analogy: Brain / project manager Role: Worker nodes; Function: Actual data storage and computation; Analogy: Muscle / laborers Concept flow: Master Node: Schedules tasks, tracks metadata - Worker 1: Store + Compute - Worker 2: Store + Compute - Worker N: Store + Compute Cost efficiency - best performance per dollar by using many standard units instead of one proprietary supercomputer Rapid replacement - failed node swapped for an identical unit; no waiting for custom parts Managing 100-1,000 separate computers requires a division of labor: Key insight: The master knows where everything is but does not do heavy lifting. Workers process data in parallel, achieving speeds impossible on a single machine

Must remember:

- Concept: Cluster; Definition: Group of cooperating nodes; Why It Matters: Horizontal scaling in practice
- Concept: Commodity hardware; Definition: Standardized, interchangeable servers; Why It Matters: Linear cost, rapid replacement
- Concept: Master-worker; Definition: Coordinator + executors; Why It Matters: Manages chaos of 1000 machines
- Concept: Replication (3x); Definition: Data copied across nodes; Why It Matters: Survives hardware failure
- Concept: Data locality; Definition: Compute where data lives; Why It Matters: Avoids network bottleneck
- Cluster = interconnected nodes acting as one system; foundation of horizontal scaling
- Commodity hardware = standardized, interchangeable, not low quality
- Master-worker: master schedules/coordinates; workers store and compute
- Reliability via 3x replication and automatic task reassignment
- Data locality: move computation to data, not data to computation
- Network tax is the primary performance enemy in clusters

Exam trap: Defining commodity hardware as "cheap junk" - it means standardized, high-quality, off-the-shelf Confusing master node's role - it coordinates, not computes heavy workloads

BD-W01-T10 Economics of Commodity Hardware

Tier A

Definition and why it matters: Google and Amazon run thousands of standard PCs rather than a single supercomputer because economics - not just technology - dictates big data architecture. The key metric is not raw speed but performance per dollar

Core explanation: Hardware Type: Specialized / high-end; Cost Behavior: Exponential - 2x power may cost 4x price; Growth Strategy: Buy bigger machine Hardware Type: Commodity / standard; Cost Behavior: Linear - 2x power costs ~2x price; Growth Strategy: Buy 2x more nodes Performance per dollar (commodity) Performance per dollar (specialized) A startup can: Begin with 10 commodity nodes (USD 20,000) Add nodes linearly as data grows Scenario: 1M supercomputer; Cost of 3x Replication: 3M (prohibitive for most businesses) Avoid upfront supercomputer investment (USD 1M+) Scenario: 1,000 x 1,000 commodity nodes; Cost of 3x Replication: 3,000 for 3x replication of one node's data Budget scales with data, not ahead of it Because commodity nodes are cheap, affordable redundancy becomes possible: Replicate data 3x across the cluster Lose any single node without data loss Smart software transforms unreliable cheap hardware into a highly reliable service Reliability = f(software intelligence, commodity redundancy) Not: expensive unbreakable hardware Big data economics invert traditional IT thinking: Old model: Buy the most reliable, expensive hardware to prevent failure New model: Buy cheap, standardized hardware; use software (replication, reassignment, lineage) to handle inevitable failure This is why Hadoop's HDFS replicates blocks 3x, why MapReduce reruns pure tasks on failure, and why Spark uses lineage graphs for recovery - all economic consequences of commodity hardware

Must remember:

- Key metric: performance per dollar, not raw speed
- Specialized hardware: exponential cost; commodity: linear cost
- Commodity nodes are interchangeable - rapid replacement, lower opex
- 3x replication affordable on 1,000 nodes, impossible on 1M supercomputer
- Strategy: cheap hardware + smart software = reliable service
- Economics explain why Google/Amazon use thousands of standard servers

Formula / decision rule: Performance per dollar (commodity) > Performance per dollar (specialized); 1M supercomputer |

Example or contrast: Aspect: Failed part; Proprietary Machine: Custom repair, weeks of downtime; Commodity Cluster: Pull unit, slide in identical server Aspect: Technician skill; Proprietary Machine: Specialized engineers required; Commodity Cluster: Standardized procedure Aspect: Downtime cost; Proprietary Machine: Massive (unique hardware); Commodity Cluster: Minimal (interchangeable units)

Exam trap: Believing 3x replication on commodity hardware is expensive - the point is it's affordable compared to specialized hardware Confusing "disposable hardware" with "don't care about failures" - failures are expected; software handles recovery

BD-W01-T11 Module 1 Summary: Big Data Constraints and Scaling

Tier B

Definition and why it matters: Computing at global scale requires abandoning the assumption that a faster laptop solves every problem. Module 1 established the constraints that force architectural change and the solutions that sustain infinite growth

Core explanation: Big data is defined by its constraints, not its tools Constraint: Volume; Definition: Petabyte-scale data; Architectural Response: Distributed storage (HDFS, S3) Constraint: Velocity; Definition: Fire-hose data arrival; Architectural Response: Parallel / stream processing Constraint: Variety; Definition: Mixed structured/unstructured; Architectural Response: Flexible schemas, schema-on-read Strength: No code changes; Weakness: Exponential cost (price wall) Strength: Simple operations; Weakness: Motherboard slot limits (hardware wall) The three Vs are architectural pressures, not marketing terms. Every platform choice traces to one or more of these pressures Strength: Good for small/bounded data; Weakness: Single point of failure (risk wall) Single machines hit physical and economic ceilings: CPU: Heat density and speed-of-light limits (~2005 plateau) RAM: Exponential cost; memory bandwidth bottleneck Strength: Linear cost; Weakness: Higher software complexity I/O: Disk and network are the slowest subsystems Strength: Fault tolerance (1 of 1000 fails - 999 continue); Weakness: Network coordination overhead These walls are why "just buy a bigger machine" fails for big data Strength: Effectively unlimited growth; Weakness: CAP theorem trade-offs (Module 2) Strategy: Upgrade the same machine - more RAM, faster CPU Concept flow: Commodity Hardware - Cluster - Master-Worker Model - 3x Replication - Data Locality - Resilient Big Data Platform Principle: Standardized nodes; Implementation: Rapid replacement, linear cost Principle: Master-worker; Implementation: Coordinator + parallel workers Verdict: Short-term fix and valid for small workloads. Cannot sustain infinite enterprise growth Principle: 3x replication; Implementation: Affordable redundancy Strategy: Add commodity hardware nodes to a cluster

Must remember:

- Big data constraints (3 Vs) drive architecture, not tool preferences
- Single machines hit CPU, RAM, and I/O walls
- Vertical scaling: easy but hits price, hardware, and risk walls
- Horizontal scaling: commodity clusters, linear cost, fault tolerance
- Master-worker model + 3x replication + data locality = cluster foundation
- Move code to data to avoid network tax
- Scaling choice = simplicity vs resilience trade-off
- Module 2 next: coordination, CAP theorem, consistency models

Exam trap: Summarizing Module 1 as "use clusters" without naming the three Vs and three hardware walls Forgetting that vertical scaling is valid for bounded workloads - not universally wrong

Week 1 Exam Lens

Likely questions

- Define and explain The Three Vs of Big Data: Volume, Velocity, and Variety; include the mechanism and one limitation.
- Compare The Three Vs of Big Data: Volume, Velocity, and Variety with Why Bigger Boxes Eventually Fail and state when each is preferred.
- Apply Vertical Scaling (Scale Up): Deep Dive to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: 10^9 ; 10^{12} ; Cost of 100 normal servers || Cost of 1 monster server with equal total power; 2,000 standard servers =; Total cluster cost $\approx n \times \text{cost per commodity node}$; Performance per dollar (commodity) > Performance per dollar (specialized); 1M supercomputer |

Mistakes to avoid: Treating big data as only a size problem - velocity and variety are equally architectural drivers Equating big data with file size alone - velocity and variety are independent architectural drivers Defining big data as volume only - a high-velocity, low-volume stream (IoT alerts) is still a big data problem

Week 2: Distributed Systems Challenges

Week roadmap: Distributed Systems Challenges: Module Overview → The Fallacies of Distributed Computing → Network Partitions and Partition Tolerance → The CAP Theorem: Consistency, Availability, and Partition Tolerance → CP vs AP Systems: Choosing the Right Trade-Off → ACID Properties in Distributed Contexts → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W02-T01 Distributed Systems Challenges: Module Overview

Tier C

Definition and why it matters: Module 1 established that horizontal scaling with commodity hardware clusters is the only sustainable path for big data. Module 2 confronts the cost: scaling out is not free. Moving from one box to many introduces coordination complexity that every big data professional must master

Core explanation: Banking apps and social media present a seamless experience - balance visible, photos loaded, everything in sync. This is the single system image: one perfect machine behind the UI

Must remember:

- Scaling out trades hardware limits for coordination complexity
- Single system image is an illusion maintained by distributed protocols
- Amazon inventory example: network partition → double sale
- Module covers: fallacies, partitions, CAP, ACID vs BASE, conflict resolution
- Mindset shift: design for failure, not for perfect networks
- Goal: think like a systems architect, not a single-machine developer

Example or contrast: Step: 1; Event: Customer in New York buys the last pair of sneakers

Exam trap: Believing scaling out only adds hardware complexity - the real cost is logical coordination Assuming the single system image is maintained automatically - it requires deliberate consistency protocols

BD-W02-T02 The Fallacies of Distributed Computing

Tier A

Definition and why it matters: Many distributed systems fail because developers apply single-machine intuition to clusters. The fallacies of distributed computing are a list of assumptions that feel like common sense but are dangerous lies in distributed environments. Identifying them is the first step toward designing resilient systems

Core explanation: On a laptop, saving a file to a local disk works 99.999% of the time. In a cluster, every operation crosses wires, routers, and switches - each a potential failure point. Code that assumes perfection crashes at the first glitch Design principle: Be a constructive pessimist - assume the network will fail, latency will be high, and bandwidth will be limited Messages sent between nodes always arrive intact and on time Cables accidentally unplugged Switches overheat and drop packets Operation: L1 cache access; Approximate Latency: Nanoseconds Routers reboot mid-transmission Operation: RAM access; Approximate Latency: ~100 nanoseconds Undersea cables damaged Operation: Same-datacenter network; Approximate Latency: ~0.5 milliseconds Analogy: A delivery truck during a rainstorm - roads flood, traffic lights fail, the truck cannot reach its destination Operation: Cross-continent network; Approximate Latency: ~100+ milliseconds Expect message loss; implement retries with idempotency Use heartbeat mechanisms to detect dead nodes Build timeout and circuit-breaker patterns Remote operations are as fast as local memory access Latency is the round-trip time for a request-response between nodes Analogy: Asking someone in the same room for a pen (instant) vs sending a letter across the country (days) Minimize cross-node communication Data locality: compute where data resides Batch operations to amortize latency Fiber optics mean unlimited data transfer capacity Bandwidth is the volume of data transferable per unit time. Even the largest pipe has finite capacity Analogy: A giant water pipe - no matter how wide, pushing the entire ocean through it at once will burst it In big data, moving a petabyte across the network for a simple calculation clogs the entire cluster. This is why data locality (Module 1) is non-negotiable Concept flow: Fallacies: False Assumptions - System Fragility: Crashes on first glitch - Design for Failure: Pessimistic Assumptions - Resilient System: Survives partitions Never ship full datasets across the network Assumption: Network reliable; Consequence if Wrong: Crash on packet loss; Correct Design: Retries, heartbeats, timeouts Filter and aggregate locally before transfer Assumption: Latency zero; Consequence if Wrong: Sequential remote calls; Correct Design: Data locality, batching Minimize shuffle data in MapReduce/Spark Assumption: Bandwidth infinite; Consequence if Wrong: Network congestion; Correct Design: Minimize data movement

Must remember:

- : 4; Fallacy: The network is secure; Reality: Networks are constantly attacked; encryption and auth required
- : 5; Fallacy: The topology doesn't change; Reality: Servers added/removed daily; topology is dynamic
- : 6; Fallacy: There is one administrator; Reality: Multiple teams manage different network segments
- : 7; Fallacy: Transport cost is zero; Reality: Every byte transferred has latency and bandwidth cost
- : 8; Fallacy: The network is homogeneous; Reality: Mixed hardware, protocols, and configurations

- Fallacies = dangerous assumptions that single-machine developers make
- 1 Network reliable $\rightarrow$ false; design for message loss
- 2 Latency zero $\rightarrow$ false; remote calls take milliseconds+
- 3 Bandwidth infinite $\rightarrow$ false; petabyte transfers clog networks
- Solution: data locality, retries, heartbeats, minimize cross-node traffic
- Fallacies motivate partition tolerance as a mandatory design requirement
- Successful architects assume failure, not perfection

Exam trap: Listing only 3 fallacies - exams may ask for the full set (8 fallacies by Peter Deutsch / James Gosling)
Confusing latency (time per operation) with bandwidth (volume per time) - they are distinct

BD-W02-T03

Network Partitions and Partition Tolerance

Tier A

Definition and why it matters: Partition tolerance (P): The ability of a system to continue operating even when an arbitrary number of messages are dropped or delayed between nodes In simpler terms: if the bridge is down, can both islands still do their jobs, or does the entire world stop?

Core explanation: A network partition occurs when nodes in a distributed system cannot communicate with each other - messages are dropped, delayed, or never delivered. Partitions are not rare edge cases; at cluster scale they are statistical guarantees Two groups of servers on separate "islands" normally communicate via a bridge. When the bridge collapses or fog blocks visibility, the islands are partitioned - each group continues operating but cannot coordinate with the other Concept flow: Server 1 - Server 2 - Server 3 - Server 4 - islandA - islandB Cause of Partition: Hardware failure; Frequency at Scale: Daily (1000 nodes - ~1 failure/day) Cause of Partition: Router reboot; Frequency at Scale: Weekly Cause of Partition: Undersea cable cut; Frequency at Scale: Rare but catastrophic Cause of Partition: Data center network misconfiguration; Frequency at Scale: Occasional In single-node computing, partitions don't exist. In a cluster spread across machines (and data centers), partitions will happen Big data cluster = Partitions are inevitable = P is required A system that is not partition-tolerant will experience catastrophic failure - the entire global application freezes when a single wire is unplugged in one data center

Must remember:

- Network partition = break in communication between node groups
- Partition tolerance (P) = system continues operating during partition
- ATM example: partition-tolerant systems handle disconnected nodes gracefully
- P is mandatory in big data clusters - not optional
- Partitions cause data divergence between isolated groups
- Resolving divergence = CAP theorem trade-off (consistency vs availability)
- Foundation for understanding CP vs AP system design

Formula / decision rule: Big data cluster $\Rightarrow$ Partitions are inevitable $\Rightarrow$ P is required; v_1

Example or contrast: An ATM in a neighborhood loses connection to the bank's central headquarters in another city. This is a partition System Type: Non-partition-tolerant; Behavior During Partition: ATM freezes, displays error, customer cannot transact System Type: Partition-tolerant; Behavior During Partition: ATM continues operating with local rules; reconciles when connection restores

Exam trap: When a partition occurs and Island A updates data while Island B cannot see the change: Island A has version v_1

BD-W02-T04

The CAP Theorem: Consistency, Availability, and Partition Tolerance

Tier A

Definition and why it matters: Proposed by Eric Brewer, the CAP theorem states a fundamental law: in a distributed system, you can provide at most 2 of 3 guarantees simultaneously during a network partition. Because partitions are inevitable in clusters, the real choice is always between C and A

Core explanation: Definition: Every read request receives the most recent write or an error Property: Read sees latest write; Behavior: Guaranteed Property: Stale data served; Behavior: Never - returns error instead Property: Priority; Behavior: Correctness over uptime Property: Request always answered; Behavior: Guaranteed Property: Answer may be stale; Behavior: Acceptable Analogy: A group text message. If you send "lunch at 1:00 PM," a friend checking one second later must see "1:00 PM" - not an older "12:00 PM" message Property: Operates during network break; Behavior: Guaranteed Consistency is about being correct Property: Optional in clusters; Behavior: No - mandatory Definition: Every request receives a non-error response, without guaranteeing it reflects the most recent write Property: Root cause; Behavior: Unreliable networks (fallacies) Analogy: Checking a cricket match score online. The server always returns a score - even if it's from 5 minutes ago because the live feed connection dropped Availability is about being fast and always on Definition: The system continues operating even when messages are dropped or delayed between nodes Partition tolerance is about being resilient During a partition, nodes in Group A and Group B cannot communicate:

A write occurs on Group A Group B cannot see the write A read on Group B would return stale data The system must choose: Refuse the read (Consistency) → system appears down to Group B users Serve stale data (Availability) → system stays up but incorrect During partition: $C \uparrow = A \downarrow$ or $A \uparrow = C \downarrow$ When a partition occurs: Do you choose to be consistent (perfect truth or nothing)? Or available (whatever answer you have, so the user isn't stuck)?

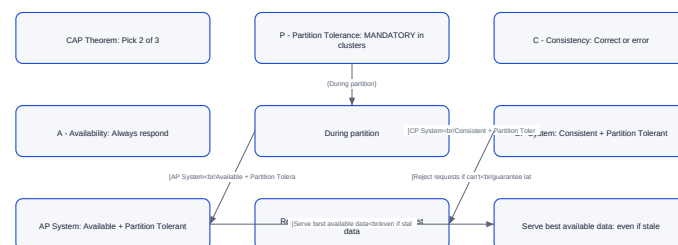
Must remember:

- Letter: C; Meaning: Consistency; Analogy: Group text shows latest message; Priority: Correctness
- Letter: A; Meaning: Availability; Analogy: Cricket score always displayed; Priority: Uptime
- Letter: P; Meaning: Partition tolerance; Analogy: System works when bridge is down; Priority: Resilience
- Combination: CP; Viable in Clusters?: Yes; Example: Banking, inventory
- Combination: AP; Viable in Clusters?: Yes; Example: Social media, analytics
- Combination: CA; Viable in Clusters?: No (partitions happen); Example: Single-node databases only
- Combination: CAP all three; Viable in Clusters?: Impossible during partition
- CAP: pick 2 of 3 - Consistency, Availability, Partition Tolerance
- C = every read gets latest write or error (correctness)
- A = every request gets a response, possibly stale (uptime)
- P = system works during network partition (resilience)
- P is mandatory in clusters → real choice is CP vs AP
- CA systems only possible without partitions (single-node)
- CAP consistency != ACID consistency - different definitions

Formula / decision rule: During partition: $C \uparrow \Rightarrow A \downarrow$ or $A \uparrow \Rightarrow C \downarrow$

Exam trap: Concept flow: CAP Theorem: Pick 2 of 3 - P - Partition Tolerance: MANDATORY in clusters - C - Consistency: Correct or error - A - Availability: Always respond - During partition - CP System: Consistent + Partition Tolerant - AP System: Available + Partition Tolerant - Reject requests if can't: guarantee latest data - Serve best available data: even if stale Stating you can have all three during a partition - CAP explicitly forbids this

The CAP Theorem: Consistency, Availability, and Partition Tolerance



Vector reconstruction of the source Mermaid visual

Concept map: The CAP Theorem: Consistency, Availability, and Partition Tolerance

BD-W02-T05 CP vs AP Systems: Choosing the Right Trade-Off

Tier A

Definition and why it matters: Because partition tolerance is mandatory in distributed clusters, every system must choose between CP (Consistency + Partition Tolerance) and AP (Availability + Partition Tolerance) during a network break. The correct choice depends entirely on the business problem, not on technology preferences

Core explanation: When nodes cannot communicate, a CP system refuses to serve requests rather than return potentially stale data Concept flow: NY - DB - LON Domain: Banking / financial transactions; Why CP: Stale balance = financial loss Domain: Inventory management; Why CP: Overselling = customer dissatisfaction + logistics cost Domain: Legal records; Why CP: Incorrect records = compliance violation Domain: Booking systems (flights, hotels); Why CP: Double-booking = operational chaos Concept flow: U1 - IN - U2 - US Domain: Social media (likes, views); Why AP: Slightly stale count acceptable Domain: Analytics / logging; Why AP: Approximate counts sufficient Domain: Sensor data / IoT; Why AP: Continuous ingestion perfect accuracy When nodes cannot communicate, an AP system continues serving requests with the best locally available data, even if stale Domain: Content delivery (CDNs); Why AP: Serve cached content during outage Domain: Search indexes; Why AP: Stale results better than no results Concept flow: Does stale data cause: financial/legal/operational harm? - CP System: Correct or offline - AP System: Available, eventually consistent - Banking, inventory, bookings - Social media, analytics, logs Criterion: Stale data acceptable?; CP: Never; AP: Temporarily yes Criterion: User sees error during partition?; CP: Yes; AP: No Criterion: Reconciliation after partition?; CP: Automatic (was offline); AP: Eventual consistency Criterion: Trust type; CP: Trust in correctness;

AP: Trust in responsiveness Criterion: Example systems; CP: HBase, ZooKeeper, traditional RDBMS; AP: Cassandra, DynamoDB, CouchDB System Type: CP (banking); User Expectation: "My balance is always exact"; Trust Model: Trust through accuracy System Type: AP (social media); User Expectation: "The app never feels broken"; Trust Model: Trust through responsiveness The architect isn't just managing servers - they're managing user trust:

Must remember:

- CP: consistent or offline during partition (banking, inventory)
- AP: available with stale data during partition (social media, analytics)
- Banking: London ATM shows error rather than stale balance
- YouTube likes: slightly wrong count OK; button must always work
- Choose CP when stale data causes financial/legal harm
- Choose AP when responsiveness matters more than perfect accuracy
- CAP framework helps decide which user trust model to prioritize

Formula / decision rule: 1000 in New York | Balance updated to

Example or contrast: Event: Withdraw 1000 in New York; CP System Response: Balance updated to 0 on NY server
Event: Spouse attempts withdrawal in London during partition; CP System Response: London ATM shows "System Unavailable" Event: Why?; CP System Response: Bank cannot guarantee both servers know the exact balance

Exam trap: Choosing AP for banking because "users hate downtime" - financial incorrectness is worse than brief unavailability Choosing CP for analytics dashboards - approximate real-time counts are acceptable; CP would cause unnecessary outages

BD-W02-T06

ACID Properties in Distributed Contexts

Tier A

Definition and why it matters: ACID is a set of four properties that traditional relational databases enforce to guarantee data is never corrupted. For decades, ACID has been the foundation of banking, airline bookings, and any system where the truth must be exact

Core explanation: Definition: A transaction is indivisible - either all operations succeed or none do Concept flow: Transaction Start - Step 1: Deduct 50 - Step 2: Add 50 - Commit: Both succeed - Rollback: Step 1 undone Outcome: Both steps succeed; Atomicity Guarantee: Committed Outcome: Any step fails; Atomicity Guarantee: All steps rolled back Outcome: Partial completion; Atomicity Guarantee: Impossible Analogy: Transferring USD 50 to a friend involves two steps: Deduct USD 50 from your account Rule Example: Balance cannot be negative; Consistency Enforcement: Reject overdraft transaction Add USD 50 to friend's account Rule Example: Age must be positive integer; Consistency Enforcement: Reject invalid insert If power fails after step 1 but before step 2, the USD 50 must not vanish. Atomicity guarantees all or nothing Rule Example: Foreign key must exist; Consistency Enforcement: Reject orphan record Without Isolation: Both read balance = 100; With Isolation: Transaction 1 locks, reads 100 Without Isolation: Both deduct 100; With Isolation: Transaction 1 deducts - balance = 0 Definition: A transaction moves the database from one valid state to another valid state, obeying all defined rules Without Isolation: Both succeed - balance = -USD 100; With Isolation: Transaction 2 waits, then fails (insufficient funds) Note: This is ACID Consistency (integrity constraints), not CAP Consistency (reads see latest write). They are different concepts Storage: RAM only; Durability: Lost on crash Storage: Committed to disk (WAL); Durability: Survives crash, power loss Definition: Concurrent transactions execute as if they were serialized - one at a time - even when running in parallel Scenario: Two people withdraw the last USD 100 from a joint account simultaneously Isolation levels (Read Uncommitted → Serializable) control how strictly this is enforced Definition: Once a transaction is committed, it persists permanently - even if the server crashes one second later ACID Property: Atomicity; Distributed Cost: Two-phase commit across nodes ACID Property: Consistency; Distributed Cost: Global constraint checking ACID Property: Isolation; Distributed Cost: Distributed locking The write-ahead log (WAL) ensures committed data survives any failure except total disk destruction ACID Property: Durability; Distributed Cost: Synchronous replication to multiple nodes

Must remember:

- Property: A Atomicity; Meaning: All or nothing; Guarantees: Complete transaction or full rollback; Failure Mode Prevented: Partial updates, lost money
- Property: C Consistency; Meaning: Valid state transitions; Guarantees: Business rules enforced; Failure Mode Prevented: Invalid data (negative balance)
- Property: I Isolation; Meaning: Serialized execution; Guarantees: No interference between concurrent transactions; Failure Mode Prevented: Race conditions, dirty reads
- Property: D Durability; Meaning: Permanent commit; Guarantees: Survives crashes; Failure Mode Prevented: Data loss after commit
- ACID: Atomicity, Consistency, Integrity constraints, Isolation, Durability
- Atomicity: all-or-nothing; rollback on any failure
- Consistency (ACID): valid state transitions; business rules enforced

- Isolation: concurrent transactions don't interfere
- Durability: committed data survives crashes (WAL + disk)
- ACID is gold standard for banking/inventory but expensive at cluster scale
- ACID Consistency != CAP Consistency - know both definitions

Formula / decision rule: 100 | Transaction 1 locks, reads; 100 | Transaction 1 deducts → balance =

Example or contrast: Term: ACID Consistency; Definition: Data satisfies integrity constraints; Context: Transaction design
Term: CAP Consistency; Definition: Reads return most recent write; Context: Distributed replication

Exam trap: Confusing ACID Consistency with CAP Consistency - most common exam trap Believing ACID scales easily to 1000-node clusters - coordination overhead makes it impractical for most big data workloads

BD-W02-T07

BASE: Basically Available, Soft State, Eventual Consistency

Tier A

Definition and why it matters: ACID enforces a strict, rule-following model that keeps financial records exact. Maintaining that level of perfection across a global cluster of thousands of machines is incredibly difficult and slow. At the scale of social networks, search engines, and analytics pipelines, a different philosophy is required - one that prioritizes flexibility and speed over instantaneous correctness

Core explanation: If ACID is a high-security bank vault, BASE is a busy neighborhood park: open, active, and occasionally messy - but it eventually settles down
Concept flow: Basically Available - Soft State - Eventual Consistency
Property: B - Basically Available; Core Idea: Serve some data rather than nothing; User Experience: Slightly stale page beats a crash
Property: S - Soft State; Core Idea: State can change without new user input; User Experience: Background sync updates replicas
Property: E - Eventual Consistency; Core Idea: All nodes agree on the last value eventually; User Experience: Temporary disagreement, then convergence
Concept flow: U - N1 - N2
In an ACID system, a network glitch may return an error page because the system cannot guarantee perfect data. A basically available system says: I'd rather give you some data than nothing at all
Serving cached or replica data keeps request latency low and avoids cascading failures when one shard is unreachable. Availability becomes a first-class guarantee, not a side effect of perfect synchronization
Domain: Social media feeds; Why BASE Fits: Stale likes/views acceptable for seconds
In the ACID world, data state is hard - it changes only when you explicitly write to it. In the BASE world, data is in soft state: the system's view can change over time even without new user input
Domain: Search indexes; Why BASE Fits: Index lag of minutes is tolerable
Different servers in a cluster constantly communicate behind the scenes to synchronize. One server may discover an update from another and change its local copy to match. In a BASE system, data is consistently flowing toward the truth rather than locked in place
Domain: Analytics / logging; Why BASE Fits: Approximate counts sufficient for dashboards
Domain: CDN edge caches; Why BASE Fits: Eventual sync from origin is standard
Soft state is not corruption - it is an intermediate phase on the path to convergence
The most famous part of BASE: if no new updates are made to a piece of data, eventually all nodes in the cluster will agree on the last value

Must remember:

- BASE trades strict ACID guarantees for scale, speed, and availability
- B (Basically Available): serve stale data rather than error pages
- S (Soft State): replicas change via background sync without new user writes
- E (Eventual Consistency): all nodes agree on the last value if writes stop
- Like counters and search indexes are classic eventual-consistency use cases
- ACID for money/health/inventory; BASE for social/search/analytics
- BASE pairs naturally with AP system design under CAP
- Soft state is flowing toward truth, not permanent incorrectness

Formula / decision rule: No new writes => All replicas converge to the same value

Example or contrast: You view a friend's profile on a social network. The server holding the latest status update is temporarily down
System Type: Strict consistency; Behavior: Error page - system unavailable
System Type: Basically available; Behavior: Shows last known update from 5 minutes ago

Exam trap: Treating eventual consistency as "never consistent" - it guarantees convergence when writes stop, not permanent disagreement
Confusing soft state with data corruption - soft state is temporary divergence during background sync

BASE: Basically Available, Soft State, Eventual Consistency



Vector reconstruction of the source Mermaid visual

Concept map: BASE: Basically Available, Soft State, Eventual Consistency

BD-W02-T08

Conflict Resolution in Eventually Consistent Systems

Tier A

Definition and why it matters: In distributed AP systems, network partitions allow nodes to remain available and accept writes independently. Once connectivity is restored, the system must reconcile divergent versions of the same data object. Without a conflict resolution strategy, concurrent updates would leave the database in an undefined state

Core explanation: This is the operational reality behind eventual consistency: convergence requires explicit rules for deciding the final state or merging concurrent updates

Concept flow: Network Partition - Node A writes v1 - Node B writes v2 - Partition Heals - Conflict Resolution Strategy - Last Write Wins - Vector Clocks - Semantic Resolution

Aspect: Mechanism; Detail: Compare wall-clock timestamps; keep newest

Aspect: Complexity; Detail: Low - single comparison per conflict

Aspect: Performance; Detail: Computationally efficient

Aspect: Risk; Detail: Clock skew can delete valid data

LWW is a timestamp-based resolution strategy. The system compares metadata of conflicting writes and retains only the update with the most recent physical timestamp

Comparison Result: $V1 < V2$; **Meaning:** Update 2 happened after update 1; **Action:** Keep update 2 (causally later)

Comparison Result: $V1 \text{ parallel } V2$; **Meaning:** Updates are concurrent; **Action:** Mark as conflict - do not auto-overwrite

Concept flow: Node 1: write A - 1,0 - Node 2: write B - Conflict detected: No auto-merge

Hardware clocks across nodes are not perfectly synchronized. A slower node's clock may assign an earlier timestamp to a write that actually happened after a faster node's write. LWW would then accidentally discard the valid, newer data

$LWW = \operatorname{argmax}(w \text{ in conflicts}) t(\text{wall-clock})(w)$

LWW is acceptable only when clock synchronization (NTP, GPS) is reliable and approximate ordering is sufficient - e.g., session metadata, non-critical counters

To avoid physical clock inaccuracies, systems use logical time through vector clocks. This method tracks the causal history of an object across nodes

Each node maintains a counter for every node that has modified the data. By comparing vectors, the system determines:

Example Domain: Shopping cart; **Semantic Rule:** Union of items from both versions

Example Domain: Collaborative document; **Semantic Rule:** Merge non-overlapping edits; flag overlapping edits for user

Example Domain: Inventory; **Semantic Rule:** Take minimum count (conservative) or maximum (optimistic)

Vector clocks preserve causal integrity: the system knows whether one event caused another or whether they happened in parallel

Concept flow: v1, v2 - DB - App - U

Amazon Dynamo and its descendants (Cassandra, Riak) use vector clocks (or variants) to detect concurrent writes without relying on synchronized wall clocks

Strategy: Last Write Wins; **Performance:** Highest; **Implementation Complexity:** Lowest; **Data Precision:** Lowest (clock skew risk); **Best For:** Low-stakes metadata

Database stores all conflicting versions

Strategy: Vector Clocks; **Performance:** Medium; **Implementation Complexity:** Medium; **Data Precision:** High (causal ordering); **Best For:** Distributed key-value stores

Must remember:

- AP systems accept concurrent writes during partitions; reconciliation is mandatory after heal
- Last Write Wins: keep newest timestamp - fast but vulnerable to clock skew
- Vector clocks: track causal history with per-node counters; detect concurrency without wall clocks
- Concurrent updates ($V1 \text{ parallel } V2$) are marked as conflicts, not silently overwritten
- Semantic resolution: database stores all versions; application merges by business rules
- Trade-off axes: performance, complexity, data precision
- LWW for low-stakes data; vector clocks for distributed KV; semantic for domain-specific merges

Formula / decision rule: $LWW = \operatorname{argmax}_{(w \text{ in conflicts})} t_{\text{wall-clock}}(w); V_1 < V_2$

Example or contrast: In complex scenarios, the database delegates resolution to the application layer - also called client-side or semantic resolution

Exam trap: Conflict resolution is a fundamental requirement of eventual consistency. The choice between LWW, vector clocks, and semantic resolution is an architectural trade-off between: System performance - how fast conflicts are resolved

BD-W02-T09

Distributed Systems Fundamentals: Module Summary

Tier C

Definition and why it matters: Scaling out is the only viable path to handle big data - but it comes with a complex price. Moving from a single powerful machine to a fleet of commodity servers means learning how to keep all nodes

coordinated when roads are blocked and radios fail. Coordination is the real network tax of big data

Core explanation: Concept flow: Scale Out with Commodity Hardware - Network Is Imperfect - Design for Failure - Partition Tolerance Required - CAP Theorem: CP vs AP - ACID vs BASE Consistency Models

Must remember:

- Scale-out is necessary for big data but introduces coordination complexity
- Fallacies of distributed computing: never assume reliable, zero-latency, infinite-bandwidth networks
- Design for failure → partition tolerance is mandatory in clusters
- CAP: during partition, choose CP (correct/offline) or AP (stale/online)
- ACID = strict truth for finance, health, inventory
- BASE = eventual truth for social, search, analytics at billion-user scale
- Conflict resolution (LWW, vector clocks, semantic) is required for AP convergence
- Coordination is the real network tax of distributed big data

Formula / decision rule: During partition: choose CP or AP - not both

Exam trap: Claiming you can have C + A + P during a partition - CAP forbids this Treating fallacies as theoretical - they cause real production failures

Week 2 Exam Lens

Likely questions

- Define and explain The Fallacies of Distributed Computing; include the mechanism and one limitation.
- Compare The Fallacies of Distributed Computing with Network Partitions and Partition Tolerance and state when each is preferred.
- Apply The CAP Theorem: Consistency, Availability, and Partition Tolerance to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Big data cluster => Partitions are inevitable => P is required; v_1 ; During partition: $C \uparrow \Rightarrow A \downarrow$ or $A \uparrow \Rightarrow C \downarrow$; 1000 in New York | Balance updated to; 100 | Transaction 1 locks, reads; 100 | Transaction 1 deducts → balance =; No new writes => All replicas converge to the same value; $LWW = \argmax_{(w \text{ in conflicts})} t_{(wall-clock)}(w)$

Mistakes to avoid: Believing scaling out only adds hardware complexity - the real cost is logical coordination Listing only 3 fallacies - exams may ask for the full set (8 fallacies by Peter Deutsch / James Gosling) When a partition occurs and Island A updates data while Island B cannot see the change:

Week 3: The MapReduce Programming Model

Week roadmap: MapReduce Programming Model: Introduction → The Map Operation: Transforming Raw Data → The Reduce Operation: Aggregating State → MapReduce Data Flow: Input Splits and Map Phase → Shuffle and Sort: The Bridge from Map to Reduce → Final Aggregation: The Reduce Phase in the Data Flow → 4 more topics.

Coverage: 10 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W03-T01 MapReduce Programming Model: Introduction

Tier A

Definition and why it matters: The first modules established the muscle of big data - commodity hardware clusters and the challenges of distributed coordination. This module addresses the brain: how do you write code that runs on 1,000 machines simultaneously without them tripping over each other?

Core explanation: The answer is MapReduce - a programming model that abstracts cluster complexity behind two simple functional operations Concept flow: Cluster Hardware: Module 1-2 - MapReduce: Programming Model - Developer writes: map + reduce logic - Framework handles: distribution + recovery Stage: Input splits; Purpose: Break petabytes into manageable chunks Stage: Map phase; Purpose: Transform raw data into key-value pairs Stage: Shuffle and sort; Purpose: Route same keys to same reducer Stage: Reduce phase; Purpose: Aggregate into final answers MapReduce traces its roots to functional programming primitives - map and fold/reduce - found in Python, Java, Lisp, and other languages. The insight was taking these simple mathematical concepts and scaling them to process the entire internet MapReduce is not just a piece of software. It is a specific way of thinking about data as an immutable stream of transformations: Input $\rightarrow$ (map) Intermediate pairs $\rightarrow$ (shuffle/sort) Grouped values $\rightarrow$ (reduce) Output Relating MapReduce to map and fold operations from functional programming - pure functions applied independently to every element, then aggregated Tracing a single piece of data through every stage of the pipeline: Legacy Problem: Single machine cannot hold petabytes; MapReduce Solution: Split data across cluster Legacy Problem: Manual parallelization is error-prone; MapReduce Solution: Framework handles distribution Understanding this sequence is the key to mastering any modern big data tool, including Apache Spark Legacy Problem: Node failures crash entire jobs; MapReduce Solution: Deterministic re-execution MapReduce implements the design for failure mindset from distributed systems theory: Legacy Problem: Complex coordination code per job; MapReduce Solution: Two functions: map and reduce Developer writes simple map/reduce logic Framework distributes work across the cluster Failed tasks are automatically re-executed on healthy nodes This is software-defined resilience - reliability through smart software, not expensive unbreakable hardware Even as Spark and other engines have evolved beyond MapReduce's disk-heavy model, the logical data flow (map $\rightarrow$ shuffle $\rightarrow$ reduce) remains the foundation of distributed data processing

Must remember:

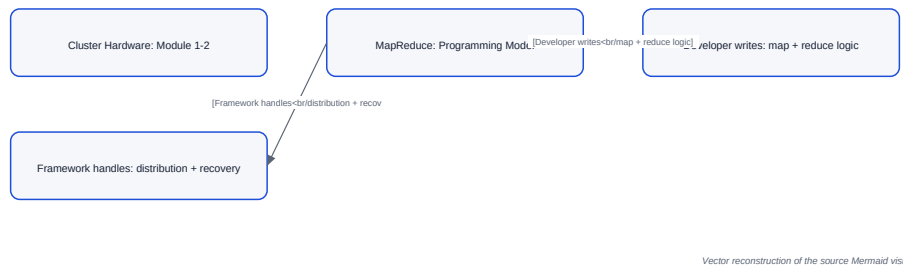
- MapReduce is the "brain" that coordinates distributed cluster computation
- Rooted in functional programming: map (transform) + reduce (aggregate)
- Data treated as immutable stream of transformations
- Pipeline: splits $\rightarrow$ map $\rightarrow$ shuffle/sort $\rightarrow$ reduce
- Framework abstracts distribution, parallelism, and failure recovery
- Design for failure: re-execute failed tasks on healthy nodes
- Web log URL popularity is the canonical case study
- Logical flow applies to Spark and all modern batch processors

Formula / decision rule: Input $\rightarrow$ (map) Intermediate pairs $\rightarrow$ (shuffle/sort) Grouped values $\rightarrow$ (reduce) Output

Example or contrast: A web analytics case study: analyzing terabytes of server logs to find the most popular URLs on a site. This grounds the abstract theory in a concrete business problem

Exam trap: Treating MapReduce as just Hadoop MapReduce software - it is a programming model applicable beyond one implementation Believing developers manage node assignment - the framework handles distribution

MapReduce Programming Model: Introduction



Concept map: MapReduce Programming Model: Introduction

BD-W03-T02

The Map Operation: Transforming Raw Data

Tier A

Definition and why it matters: Map is a transformation function applied independently to every single element in a dataset Input: One raw record (key, value); Output: Zero or more structured (key, value) pairs

Core explanation: If big data is a mountain of raw, messy material, the map phase is where processing begins - sorting through that mountain to find the specific pieces that matter. Mapping transforms data from its raw state into a structured format the system can aggregate A chef must peel 1,000 potatoes. The map instruction is simple: take one potato, peel it, place it in the bowl. The chef does this one potato at a time. Each map invocation produces one peeled potato. The results are independent Scaling Strategy: 2x data volume; Traditional: Buy faster CPU; Map Reduce: Add 2x nodes, same wall-clock time Scaling Strategy: Communication; Traditional: Shared state, locks; Map Reduce: None during map phase Scaling Strategy: Bottleneck; Traditional: Single machine; MapReduce: Network (only at shuffle) Property: Pure; Why It Matters: Same input always produces same output - enables safe re-execution on failure Property: Stateless; Why It Matters: No shared mutable state between invocations Property: Independent; Why It Matters: No communication with other mappers Property: Side-effect free; Why It Matters: Only emits key-value pairs; does not modify external state Map operations can run on hundreds, thousands, or even millions of nodes at the same time without communicating with each other. Each worker processes its own data split independently The secret to scaling: if you have twice as much data, you do not need a faster computer - you add more nodes and mapping finishes in the same amount of time. This is perfectly distributed logic These properties are what make MapReduce fault-tolerant. A failed mapper can be rerun on a different node with identical results Map transforms - it does not aggregate. A billion pairs like ("/index.html", 1) are useless until summed. That is the job of the reduce operation, which groups all values sharing the same key and combines them

Must remember:

- Map transforms each raw element into zero or more (key, value) pairs
- Signature: `map(key, value) → list of (key, value) pairs`
- Web log example: extract URL, emit (url, 1) per line
- Mappers run in parallel with no inter-node communication
- 2x data → add 2x nodes, same completion time (embarrassingly parallel)
- Map functions must be pure, stateless, and deterministic
- Map produces intermediate pairs; reduce aggregates them

Formula / decision rule: `map: (key, value) → [(key_1, value_1), (key_2, value_2), ...]`

Example or contrast: A text file contains a billion lines of server logs. Each line has an IP address, timestamp, and URL (e.g., /index.html) Raw Log Line: 192.168.1.1 2024-01-15 /index.html Chrome; Map Output: ("/index.html", 1) Raw Log Line: 10.0.0.5 2024-01-15 /home.html Firefox; Map Output: ("/home.html", 1)

Exam trap: Writing aggregation logic in the map phase - map emits pairs; reduce sums them Assuming mappers communicate - they are fully independent until shuffle

BD-W03-T03

The Reduce Operation: Aggregating State

Tier A

Definition and why it matters: Reduce is an aggregation function that combines multiple values sharing the exact same key. In functional programming, this is called a fold - folding a long list into one representative value reduce: (key, [value1, value2, ...]) - (key, result)

Core explanation: The map phase produces thousands of independent key-value pairs like ("/index.html", 1). A billion ones do not help a business make decisions - you need a final total. The reduce operation is the second half of the processing brain: it consolidates parallel streams into a single summarized answer Phase: Map; Verb: Transform; Purpose: Raw - structured pairs Phase: Reduce; Verb: Aggregate; Purpose: Pairs - summarized result Phase: Map; Input: One value; Scope: Single record Phase: Reduce; Input: List of values; Scope: All values for one key across

the cluster Stage: Raw input; Data Volume (10 TB log example): 10 terabytes The framework gathers every value associated with a key and hands the entire list to the reducer. The developer does not manually collect values - that is the framework's job after shuffle and sort Stage: After map; Data Volume (10 TB log example): ~10 terabytes (one pair per line) Stage: After reduce; Data Volume (10 TB log example): Few megabytes (ranked top URLs) Business Need: Total page views; Reduce Logic: Sum Business Need: Average response time; Reduce Logic: Sum + count, then divide Business Need: Maximum temperature; Reduce Logic: Max Business Need: Distinct users; Reduce Logic: Set union Business Need: Top-N items; Reduce Logic: Custom comparator The output is a much smaller, summarized set of key-value pairs. A real-world web log of 10 TB might yield only a few megabytes - a simple ranked list of the top 100 URLs. This compression from overwhelming volume to actionable insight is why MapReduce is powerful for big data analytics While summing is the most common operation, reduce can perform any associative aggregation: The reducer has a complete view of every instance of a key across the entire cluster, so its result is the absolute global truth for that key A critical question remains: how did the ones from 1,000 different mappers find their way to the same exact reducer? That magic happens in the shuffle and sort phase - the bridge connecting map to reduce

Must remember:

- Reduce aggregates all values sharing the same key into one result
- Signature: `reduce(key, list of values) → (key, result)`
- Functional programming equivalent: `fold`
- Framework groups values after shuffle; developer writes aggregation logic
- URL example: 10,000 ones → single count of 10,000
- 10 TB input can compress to megabytes of ranked insights
- Reduce operations: sum, average, max, set union, custom
- Shuffle/sort is the bridge that delivers grouped values to reducers

Formula / decision rule: `reduce: (key, [value_1, value_2, ...]) → (key, result); Output: (texttt(/home.html), 10000)`

Example or contrast: The map phase produced 10,000 pairs of `("/home.html", 1)`. The framework groups them. The reducer receives: Concept flow: `(/home.html, 1 - (/home.html, 1 - (/home.html, 1 - ... 10,000 ones - reduce('/home.html', [1,1,1,... - ('/home.html', 10000 Key: /home.html`

Exam trap: Treating reduce input as a single value - it is always a list of values for one key Performing filtering in reduce that should happen in map - push work left to minimize shuffle volume

BD-W03-T04

MapReduce Data Flow: Input Splits and Map Phase

Tier A

Definition and why it matters: Map and reduce functions exist in isolation as logic, but in a real cluster they are part of a highly coordinated lifecycle. The data journey begins the moment raw files sit on distributed storage and ends with structured intermediate results ready for shuffle

Core explanation: This note covers steps 1 and 2: input splits and the map phase Concept flow: Raw File on HDFS - Input Splits: 64-128 MB chunks - Map Phase: Parallel workers - Intermediate: Key-Value Pairs - Shuffle & Sort: Next stage Parameter: Split size; Typical Value: 64 MB or 128 MB; Why: Aligns with HDFS block size Parameter: Number of splits; Typical Value: $\text{lceil file size} / \text{split size rceil}$; Why: One map task per split Parameter: 10 TB file; Typical Value: ~80,000 splits at 128 MB; Why: 80,000 parallel map tasks Concept flow: Data Split A: stored locally - Map Task A: runs here - Data Split B: stored locally - Map Task B: runs here - Data on Node 3 - Map on Node 1 A single 10 terabyte log file cannot be handed to one computer - it is too large for memory and would create a single point of failure. The MapReduce framework automatically breaks the file into manageable, executable chunks Strategy: Data-local map; Network Cost: Zero (read from local disk); Performance: Optimal Strategy: Rack-local map; Network Cost: One hop within rack; Performance: Acceptable Strategy: Remote map; Network Cost: Cross-rack transfer; Performance: Expensive A massive encyclopedia is divided - one chapter per student - so everyone reads simultaneously. Splits create thousands of small tasks that individual workers can handle in parallel Because HDFS stores data in fixed-size blocks (typically 128 MB) with 3x replication, split boundaries align with block boundaries. This alignment is critical for data locality in the next step Once data is split, the framework assigns each split to a worker node. That node processes its local split in parallel with all other nodes The framework tries to run the map task on the exact same server where the 128 MB split is already stored. This avoids shipping data across the network - the dominant cost in distributed systems Property: Format; Map Output: (key, value) pairs Property: Location; Map Output: Local disk on each worker Property: Size; Map Output: Often similar to input size Each worker applies the custom map logic independently: Property: Final?; Map Output: No - requires reduce aggregation Read its assigned split line by line Step: 1. Input splits; Action: Divide 10 TB into 128 MB chunks; Parallelism: N/A (metadata operation) Apply transformation (e.g., extract URL from log line) Step: 2. Map phase; Action: Transform each chunk independently; Parallelism: Thousands of parallel tasks Emit key-value pairs to local intermediate buffer Workers do not communicate with each other during the map phase. They focus on their small piece of the puzzle

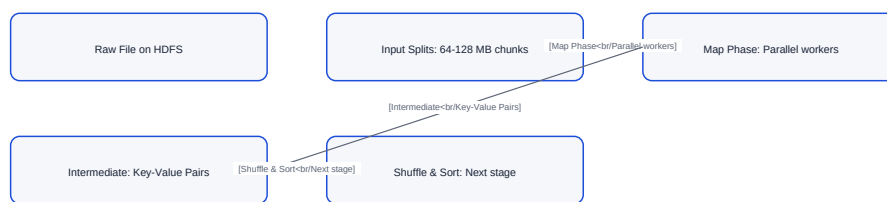
Must remember:

- Input splits break massive files into 64-128 MB executable chunks
- Split size aligns with HDFS block size for efficient I/O
- Map phase assigns one task per split to worker nodes
- Data locality: run map where data lives to avoid network transfer
- Mappers work independently with no inter-node communication
- Map output = intermediate key-value pairs on local disk
- Next stage (shuffle/sort) groups same keys for reduce

Formula / decision rule: $\lceil \text{file size} / \text{split size} \rceil$

Exam trap: Confusing HDFS block size with MapReduce split size - they are aligned but configurable independently
Assuming one map task per file - large files produce many map tasks

MapReduce Data Flow: Input Splits and Map Phase



Vector reconstruction of the source Mermaid visual

Concept map: MapReduce Data Flow: Input Splits and Map Phase

BD-W03-T05

Shuffle and Sort: The Bridge from Map to Reduce

Tier A

Definition and why it matters: After the map phase, thousands of independent workers each hold partial results. Mapper A has some data for /index.html and Mapper B has other data for /index.html. How do both get those results to the exact same reducer?

Core explanation: The answer is step 3 of the logical data flow: the shuffle and sort phase - often called the magic of MapReduce because it bridges transformation logic (map) to aggregation logic (reduce) Concept flow: Mapper 1: /index.html - 1 - Shuffle & Sort - Mapper 2: /index.html - 1 - Mapper 3: /home.html - 1 - Reducer A: /index.html - 1,1,... - Reducer B: /home.html - 1,1, Partitioning decides which reducer node receives which key. The framework uses a simple but powerful formula: $\text{reducerid} = \text{hash}(\text{key}) \bmod R$ Before Sort: (/home.html, 1), (/index.html, 1), (/index.html, 1), (/home.html, 1); After Sort: (/home.html, [1, 1]), (/index.html, [1, 1]) where R is the number of reducer nodes What Happens: Thousands of mappers send data simultaneously; Cost: Massive cross-cluster traffic What Happens: Data crosses racks, switches, and NICs; Cost: Bandwidth saturation What Happens: Intermediate data often equals input size; Cost: Terabytes over the wire If hashing is skewed, some reducers receive far more keys than others - creating stragglers that slow the entire job. Uniform hash distribution is critical for balanced load Concept flow: 1000 Mappers - Network Fabric - 100 Reducers - Shuffle = slowest phase Once data arrives at the reducer node, the framework does not dump it in a random pile. It groups all values by their key - hence "shuffle and sort." Sorting ensures that when the reducer starts, all the ones for /index.html are lined up together in a single list. The reducer reads a perfectly organized stream of values, one key at a time - no searching through a messy database This phase is the most network-intensive part of any MapReduce job In distributed systems terms, this is where you pay the network tax discussed in hardware constraints. Because so much data moves across wires and switches, shuffle is often the bottleneck that slows the entire job Performance tuning in big data almost always focuses on minimizing shuffle data: Filter early in map (emit fewer pairs) Use combiners to pre-aggregate locally Choose better partition keys to avoid skew Reduce number of shuffle stages in multi-pass jobs

Must remember:

- Shuffle routes all values for the same key to the same reducer
- Partitioning formula: $\text{hash}(\text{key}) \bmod \text{numReducers}$
- Sorting groups values by key into organized lists per reducer
- Shuffle + sort = the bridge between map and reduce phases
- Shuffle is the most network-intensive phase - the network tax
- Performance tuning focuses on minimizing shuffle data volume

- Data skew in partitioning creates reducer stragglers
- After shuffle/sort, reducers receive complete key-grouped lists

Formula / decision rule: $\text{reducer_id} = \text{hash}(\text{key}) \bmod R$

Example or contrast: Key: `/index.html`; $\text{hash}(\text{key}) \bmod 3 = 0$; Destination: Reducer A Every single instance of `/index.html` from across the entire cluster is sent to the exact same destination. Without partitioning, each reducer would have only a partial view of the truth Operation: Shuffle; What It Does: Physically move data from mappers to correct reducers; Who Benefits: Correct grouping

Exam trap: Describing shuffle as "optional" - it is mandatory whenever reduce groups by key Forgetting the hash formula - exam questions often test $\text{hash}(\text{key}) \bmod \text{numReducers}$

BD-W03-T06 Final Aggregation: The Reduce Phase in the Data Flow

Tier B

Definition and why it matters: The data journey began with raw files, split them into chunks, transformed them with mappers, and organized billions of results through shuffle and sort. Step 4 - the reduce phase - is where all parallel streams finally meet to produce the definitive answers the business needs

Core explanation: Concept flow: 1. Input Splits - 2. Map Phase - 3. Shuffle & Sort - 4. Reduce Phase - Final Results on HDFS Concept: Fold / reduce (FP); MapReduce Equivalent: Reduce phase Concept: Accumulator; Map Reduce Equivalent: Running sum, max, average Concept: Base case; MapReduce Equivalent: Empty list - identity value Concept: Combine; MapReduce Equivalent: Merge partial results The reduce phase is rooted in the mathematical concept of fold (or reduce). Imagine data as a long unfolded piece of paper representing every single click on a website. The reduce phase folds that paper down until only the total sum remains Operation: Sum; Logic: Add all values; Example Output: (`/index.html`, 45000) Operation: Average; Logic: $\text{Sum} / \text{count}$; Example Output: (`/index.html`, 2.3s avg) Operation: Maximum; Logic: Pick largest; Example Output: (`server-7`, 99.9% uptime) Operation: Filter; Logic: Keep matching; Example Output: Top-N URLs only Each reducer node acts like a funnel. It receives: Concept flow: Key: `/index.html`: Values: [1, 1, 1, ..., 1 - Reducer Funnel - Sum all values - (`/index.html`, 45000 - Write to HDFS A key (e.g., `/index.html`) Phase: Raw input; Volume (10 TB log example): 10 terabytes A list of all values associated with that key (e.g., [1, 1, 1, \ldots]) Phase: Map phase output; Volume (10 TB log example): ~10 terabytes (one pair per line) The developer writes the logic that aggregates the list. Common operations: Phase: Reduce phase output; Volume (10 TB log example): Few megabytes (ranked insights)

Must remember:

- Reduce phase is step 4: final aggregation after shuffle and sort
- Each reducer receives one key and a complete list of its values
- Developer writes aggregation logic; framework handles grouping and I/O
- Operations: sum, average, max, filter - any associative fold
- 10 TB input compresses to megabytes of ranked business insights
- Results written to distributed file system - job complete
- Full pipeline: splits $\rightarrow$ map $\rightarrow$ shuffle/sort $\rightarrow$ reduce
- Reducer has global view of its keys $\rightarrow$ produces definitive truth

Example or contrast: Term: Reduce function; Meaning: The pure logic you write: (key, values) - result

Exam trap: Stating reduce runs before shuffle - order is always map $\rightarrow$ shuffle $\rightarrow$ reduce Believing each reducer sees all keys - each reducer sees only its partition of keys

BD-W03-T07 Case Study: Analyzing Web Logs with MapReduce

Tier A

Definition and why it matters: Goal: Find the most popular URLs from 10 terabytes of raw server logs

Core explanation: Opening a 10 TB file in a spreadsheet is impossible. With MapReduce, it becomes a straightforward three-step process - map logic, shuffle (automatic), and reduce logic Concept flow: 10 TB Web Logs: on HDFS - Map: extract URL, emit 1 - Shuffle: group by URL - Reduce: sum counts - Ranked URL List: few MB Field: IP address; Example: 192.168.1.1 Field: Timestamp; Example: 2024-06-05T14:30:00Z Field: URL; Example: `/products/laptop` Field: Browser info; Example: Chrome/120.0 Log Line (simplified): `GET /google.com`; Map Output: (`"google.com"`, 1) The 10 TB of web logs sit in HDFS, split into thousands of 128 MB chunks. Each log line contains: Log Line (simplified): `GET /bits-pilani.ac.in`; Map Output: (`"bits-pilani.ac.in"`, 1) The framework assigns one mapper per chunk Key: All `google.com` pairs; Routed To: Reducer A For every single line in the log file, the mapper performs one task: extract the URL and emit a key-value pair Key: All `bits-pilani.ac.in` pairs; Routed To: Reducer B map(log line) - (`URL`, 1) Key: All `/products/laptop` pairs; Routed To: Reducer C The mapper ignores IP, timestamp, and browser info. It does this millions of times in parallel across the cluster. By the end of the map phase, there is a sea of billions of ones associated with different URLs This is not code the developer writes. The framework automatically ensures every pair with the same key is sent to the exact same reducer This is the moment of coordination - distributed noise from

mappers becomes organized piles ready for the final count Each reducer receives a manageable list. Reducer A gets: Key: google.com Values: [1, 1, 1, \ldots] (millions of ones) reduce(URL, [1, 1, ..., 1]) - (URL, total count) Problem: Word count; Map Emits: (word, 1); Reduce Computes: Sum - frequency The reducer sums the list and outputs one final result: ("google.com", 50000000) Problem: Error rate by endpoint; Map Emits: (endpoint, 1) if error; Reduce Computes: Sum - error count Problem: Revenue by product; Map Emits: (productid, price); Reduce Computes: Sum - total revenue Result: 10 TB of raw text reduced to a tiny ranked list of URL popularity Problem: Max response time; Map Emits: (endpoint, latency); Reduce Computes: Max - worst case Code pattern: Pseudocode - map function; def map(logline):: url = extracturl(logline); emit(url, 1)

Must remember:

- Case study: find most popular URLs from 10 TB web logs on HDFS
- Map: extract URL from each line, emit (url, 1)
- Shuffle: framework groups all pairs with same URL to one reducer
- Reduce: sum list of ones $\rightarrow$ (url, totalcount)
- 10 TB raw text $\rightarrow$ few MB ranked URL list
- Three wins: scalability (add nodes), simplicity (two functions), resilience (auto-restart)
- Same pattern applies to word count, error rates, revenue aggregation
- Developer writes map + reduce only; framework handles everything else

Formula / decision rule: map(log line) $\rightarrow$ (URL, 1); reduce(URL, [1, 1, ..., 1]) $\rightarrow$ (URL, total count)

Example or contrast: Achievement: Scalability; How: Handled 10 TB by adding more nodes - not faster hardware Achievement: Simplicity; How: Developer wrote only extract-and-sum logic (~10 lines) Achievement: Resilience; How: If one mapper failed, framework restarted it without failing the whole job

Exam trap: Writing shuffle logic manually - the framework handles partitioning and routing Emitting the full log line as the key - keys should be the aggregation dimension (URL)

BD-W03-T08

Fault Tolerance in MapReduce: Handling Node Failures

Tier A

Definition and why it matters: Think of a calculator: typing $5 + 5$ always yields 10, regardless of which calculator you use or whether the previous one crashed Property: Deterministic; Definition: Same input always produces the exact same output

Core explanation: In a cluster of 1,000 commodity servers, hardware failure is not a possibility - it is a statistical certainty. At any moment, a hard drive may crash, a power supply may fail, or a network cable may be unplugged. Map Reduce implements the design for failure mindset to keep a 10 TB job running even when nodes go dark Cluster of 1000 nodes = Multiple failures per day are expected Concept flow: M - W1 - FAILED - W2 Step: 1; Action: Master detects heartbeat failure Step: 2; Action: Master does not cancel the whole job Step: 3; Action: Master checks metadata to find which tasks the failed node was running Step: 4; Action: Master reassigns those exact tasks to available nodes Step: 5; Action: New worker reads data from HDFS replica (3x replication) The master node (also called JobTracker or driver) acts as a vigilant project manager. It maintains constant communication with every worker via heartbeats Without Purity: Re-running a task may produce different results; With Purity: Re-running produces identical results Without Purity: Duplicate execution corrupts counts; With Purity: Duplicate execution is harmless (idempotent) Without Purity: Failure = data loss or inconsistency; With Purity: Failure = temporary delay only Because HDFS replicates data 3 times, the master knows exactly where to find a backup copy of the data split. The new worker picks up where the old one left off If a task restarts halfway, won't that mess up the final count? No - because map and reduce functions are designed to be pure and deterministic Concept flow: Node Failure Detected - Master Reassigns Task - HDFS Provides Replica - Pure Function Re-executes - Identical Output Produced - Job Continues Component: Master node; Role in Fault Tolerance: Detect failure, reassign tasks Component: HDFS 3x replication; Role in Fault Tolerance: Data survives node loss Component: Pure map/reduce functions; Role in Fault Tolerance: Re-execution produces consistent results Component: Heartbeat monitoring; Role in Fault Tolerance: Early failure detection This determinism ensures total reliability across a messy, unpredictable cluster MapReduce achieves fault tolerance not by using expensive, unbreakable hardware, but by using smart software: Cheap commodity servers that fail frequently Intelligent scheduling that reroutes work Deterministic functions that make retries safe Replicated storage that survives disk crashes This is the same philosophy behind cloud-native systems: assume failure, design around it Fault tolerance comes with a price. Because intermediate results must survive node failure, MapReduce writes all map output to disk (with 3x replication). This disk I/O penalty is the subject of the next note - and it is precisely what motivated Apache Spark's in-memory approach

Must remember:

- Hardware failure in 1000-node clusters is a statistical certainty, not an edge case
- Master node detects heartbeat failures and reassigns tasks to healthy workers
- HDFS 3x replication ensures backup copies exist for reassigned tasks

- Map and reduce functions must be pure and deterministic
- Same input → same output → safe to re-execute without corrupting results
- Fault tolerance via smart software on commodity hardware, not expensive machines
- Reassignment + purity + replication = job survives multiple node failures
- Disk writes for intermediate state are the performance cost of this safety

Formula / decision rule: Cluster of 1000 nodes => Multiple failures per day are expected; 5 + 5

Exam trap: Believing node failure kills the entire job - only failed tasks are reassigned Assuming re-execution produces different results - pure functions guarantee identical output

BD-W03-T09 The Cost of Disk-Based Intermediate State

Tier A

Definition and why it matters: MapReduce's resilience - functional purity, task reassignment, HDFS replication - is powerful. But achieving that level of fault tolerance requires paying a significant performance tax. The biggest bottleneck in the MapReduce model is the cost of disk-based intermediate state: the 3x write penalty

Core explanation: In MapReduce, data does not flow directly from mapper memory to reducer memory. To ensure progress is not lost if a node fails, the framework writes all intermediate data (map phase output) to physical disk
 Concept flow: Mapper completes - Write to local disk - HDFS replicates 3x - Shuffle reads from disk - Reducer processes
 Stage: Post-map; What Gets Written: Intermediate key-value pairs; Where: Local disk + HDFS replicas
 Stage: Post-reduce; What Gets Written: Final results; Where: HDFS (3x replicated)
 Intermediate Data Produced: 1 TB; Actual Disk Writes: 3 TB
 Component: CPU register; Typical Latency: Nanoseconds
 Component: RAM; Typical Latency: ~100 nanoseconds
 Component: SSD; Typical Latency: ~100 microseconds
 HDFS automatically replicates every block 3 times across the cluster for safety. This means: Component: HDD; Typical Latency: ~10 milliseconds
 Physical disk writes = 3 x logical intermediate data size
 Component: Network (cross-rack); Typical Latency: ~1 millisecond
 Concept flow: CPU: map computation: fast - Wait for disk write: SLOW - Shuffle can begin - 3x write penalty
 This write penalty consumes massive storage space and, more importantly, takes a lot of time
 Job Type: Single map-reduce; Disk Penalty Impact: One round of disk writes - tolerable
 Disk I/O is the slowest component of a computer relative to CPU and memory
 Job Type: Multi-stage pipeline (2+ map-reduce passes); Disk Penalty Impact: Disk writes at every stage - severe
 Job Type: Iterative algorithms (ML training loops); Disk Penalty Impact: Read/write cycle per iteration - prohibitive
 Technique: Filter early in map; Effect: Fewer pairs emitted - less shuffle - less disk
 Technique: Use combiners; Effect: Pre-aggregate locally before shuffle
 Every time a map task finishes, the CPU must stop and wait for the disk to finish writing 3 copies before the shuffle phase can begin. This stop-and-copy rhythm creates a massive bottleneck
 Technique: Narrow map output; Effect: Emit only necessary fields
 Technique: Reduce number of job stages; Effect: Fewer disk round-trips
 MapReduce is incredibly reliable but significantly slower for jobs that pass data through multiple stages. This is a fundamental architectural limit, not a tuning issue
 The engineer's job is to reduce the volume of data hitting disk
 Aspect: Intermediate storage; MapReduce: Disk (always); Spark: Memory (preferred)
 Aspect: Multi-stage pipelines; MapReduce: Disk write per stage; Spark: In-memory pipelining
 Aspect: Iterative algorithms; MapReduce: Re-read from disk each iteration; Spark: Cache in memory
 Goal: turn 10 TB of logs into a tiny stream of results as quickly as possible. The less intermediate state, the less time spent waiting on disk

Must remember:

- MapReduce writes all intermediate map output to disk for fault tolerance
- HDFS 3x replication means physical writes = 3x logical data size
- Disk I/O is the slowest hardware component - creates stop-and-copy bottleneck
- Multi-stage and iterative jobs suffer most from repeated disk round-trips
- Tuning goal: minimize intermediate data volume (filter early, use combiners)
- Core trade-off: resilient because of disk writes; slow because of disk writes
- This disk overhead directly motivated Apache Spark's in-memory architecture
- Spark pipelines stages in RAM; MapReduce materializes every stage to disk

Formula / decision rule: Physical disk writes = 3 x logical intermediate data size; Resilience uparrow Leftrightarrow Speed downarrow (for disk-bound workloads)

Exam trap: Strength: Resilient (saves everything to disk); Cost: Slow (saves everything to disk) Strength: Survives node failures; Cost: Pays 3x write penalty per stage

BD-W03-T10 MapReduce Module Summary

Tier C

Definition and why it matters: The first two modules built the muscle of the big data platform - clusters and distributed hardware. This module mastered the brain - how to coordinate those machines to solve massive problems through the MapReduce programming model

Core explanation: Concept flow: Module 1: Hardware & Scale-Out - Module 2: Distributed Theory - Module 3: MapReduce Brain - Abstraction - Data Flow - Fault Tolerance - Disk Trade-Off

Must remember:

- MapReduce abstracts cluster complexity behind map + reduce functions
- Developer writes business logic; framework handles splits, shuffle, recovery
- Pipeline: input splits $\rightarrow$ map $\rightarrow$ shuffle/sort $\rightarrow$ reduce
- Shuffle is the network bottleneck - $\text{hash}(\text{key}) \bmod \text{numReducers}$ routes keys
- Fault tolerance: heartbeats + reassignment + HDFS replicas + pure functions
- Disk-based intermediate state = 3x write penalty = resilience vs speed trade-off
- Web log case study: 10 TB $\rightarrow$ megabytes of ranked URLs
- This disk overhead motivated Apache Spark's in-memory architecture
- Map $\rightarrow$ shuffle $\rightarrow$ reduce is the universal pattern for distributed batch processing

Example or contrast: MapReduce writes every intermediate step to disk for safety - the 3x write penalty (HDFS replication). This makes it:

Exam trap: Describing MapReduce without mentioning shuffle - it is the critical bridge and bottleneck Claiming developers manage distribution - the framework abstracts it

Week 3 Exam Lens

Likely questions

- Define and explain MapReduce Programming Model: Introduction; include the mechanism and one limitation.
- Compare MapReduce Programming Model: Introduction with The Map Operation: Transforming Raw Data and state when each is preferred.
- Apply The Reduce Operation: Aggregating State to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Input $\xrightarrow{\text{map}}$ Intermediate pairs $\xrightarrow{\text{shuffle/sort}}$ Grouped values $\xrightarrow{\text{reduce}}$ Output; map: $(\text{key}, \text{value}) \rightarrow [(\text{key_1}, \text{value_1}), (\text{key_2}, \text{value_2}), \dots]$; reduce: $(\text{key}, [\text{value_1}, \text{value_2}, \dots]) \rightarrow (\text{key}, \text{result})$; Output: $(\text{texttt{/home.html}}, 10000)$; $\lceil \text{file size} / \text{split size} \rceil$; $\text{reducer_id} = \text{hash}(\text{key}) \bmod R$; $\text{map}(\text{log line}) \rightarrow (\text{URL}, 1)$; $\text{reduce}(\text{URL}, [1, 1, \dots, 1]) \rightarrow (\text{URL}, \text{total count})$

Mistakes to avoid: Treating MapReduce as just Hadoop MapReduce software - it is a programming model applicable beyond one implementation Writing aggregation logic in the map phase - map emits pairs; reduce sums them Treating reduce input as a single value - it is always a list of values for one key

Week 4: Batch Processing Architectures: Hadoop and Spark

Week roadmap: Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing → HDFS Deep Dive: Blocks, Replication, and Rack-Aware Fault Tolerance → MapReduce Data Flow on HDFS: From Blocks to Final Output → High I/O Latency and Materialisation Cost: Hadoop's Performance Bottleneck → Introduction to In-Memory Computing: RAM as Spark's Primary Workspace → How Spark Minimises Disk Read-Write Cycles: DAG, Lazy Evaluation, and Persistence → 2 more topics.

Coverage: 8 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W04-T01 Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing

Tier A

Definition and why it matters: Big data systems must balance three competing forces: durability (data must survive hardware failure), throughput (process terabytes in acceptable time), and cost (commodity hardware vs premium RAM). For over a decade, Hadoop dominated batch analytics by optimising for durability and cost on spinning disks. As workloads evolved - especially iterative machine learning - the disk-bound model became the bottleneck. Apache Spark emerged by treating RAM as the primary workspace, not secondary cache

Core explanation: Understanding both architectures is essential for designing production pipelines: Hadoop remains relevant for archival storage and one-pass ETL; Spark wins for repeated access to the same dataset

Concept flow: HDFS Internals - MapReduce Data Flow - IO Latency & Materialisation - In-Memory Computing - Spark DAG & Optimisation - Hadoop vs Spark Benchmark - Cost-Benefit Analysis

Topic: HDFS blocks & replication; Core Question: How is data physically stored and protected? Topic: MapReduce on HDFS; Core Question: How does computation interact with storage? Topic: Materialisation cost; Core Question: Why is disk I/O the bottleneck? Topic: In-memory computing; Core Question: How does Spark eliminate repeated disk access? Topic: K-Means benchmark; Core Question: When does Spark justify higher RAM investment?

Storage Tier: L1 CPU cache; Typical Latency: ~1 ns; Relative Speed: Baseline (fastest)

Storage Tier: RAM; Typical Latency: ~100 ns; Relative Speed: ~100x slower than cache

Hadoop's design philosophy treats disk as the system of record. The Hadoop Distributed File System (HDFS) breaks files into large blocks (typically 128 MB or 256 MB) and replicates each block three times across rack-aware node placement. This architecture assumes: Storage Tier: HDD (seek + read); Typical Latency: ~10 ms; Relative Speed: ~10,000,000x slower than RAM

Hardware failure is inevitable at cluster scale

Sequential disk reads deliver high throughput for batch scans

Intermediate results can be spilled to local disk without expensive replication

Real-world analogy: HDFS behaves like a warehouse with three copies of every pallet stored in different buildings - slow to retrieve, but virtually impossible to lose

Every MapReduce stage that writes intermediate results to disk incurs materialisation cost - the latency of making data physically real on storage before the next stage can begin. Disk I/O is orders of magnitude slower than RAM: For multi-stage or iterative workloads (K-Means, PageRank, gradient descent), this "disk tax" compounds with every iteration

Criterion: Workload type; Favour Hadoop: One-pass batch ETL, archival; Favour Spark: Iterative ML, interactive analytics

Spark inverts Hadoop's assumption. Instead of writing after every transformation, Spark: Criterion: Data reuse; Favour Hadoop: Single scan per job; Favour Spark: Same data accessed many times

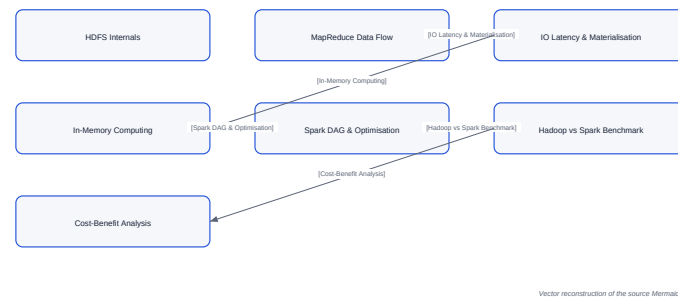
Must remember:

- Hadoop prioritises durability and cost via HDFS block storage with 3x rack-aware replication
- MapReduce's disk-bound intermediate writes create high I/O latency - the primary Hadoop bottleneck
- Materialisation cost forces read-process-write cycles that devastate iterative algorithms
- Spark treats RAM as the primary workspace, keeping data in memory across transformations and iterations
- Spark uses DAG planning, lazy evaluation, and caching to minimise disk read-write cycles
- Iterative workloads like K-Means can run up to 100x faster on Spark vs Hadoop MapReduce
- The trade-off: Spark requires RAM-heavy clusters at higher hardware cost
- Choose Spark when algorithms repeatedly access the same data; choose Hadoop for cost-effective archival and one-pass batch jobs

Exam trap: Trap: "Spark replaces HDFS entirely." Reality: Spark commonly reads from HDFS/S3; it replaces Map Reduce's execution model, not distributed storage

Trap: "In-memory means no disk I/O ever." Reality: Spark still reads initial data from disk and may spill to disk when memory is exhausted

Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing



Vector reconstruction of the source Mermaid visual

Concept map: Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing

BD-W04-T02

HDFS Deep Dive: Blocks, Replication, and Rack-Aware Fault Tolerance

Tier A

Definition and why it matters: Disk access has two cost components: Property: Optimised for; Desktop FS (~4 KB blocks): Random access, many small files; HDFS (~128 MB blocks): Sequential scans of massive files

Core explanation: Before optimising computation, you must understand how data is physically laid out. HDFS (Hadoop Distributed File System) is the storage layer underpinning the Hadoop ecosystem. Its design choices - enormous block sizes, triple replication, rack-aware placement - directly explain why Hadoop excels at sequential batch reads but struggles with low-latency, iterative access patterns. Unlike desktop file systems that use 4 KB or 8 KB blocks, HDFS uses large blocks - typically 128 MB or 256 MB. Concept flow: Node 1 (Copy 1 - local - Node 2 - Node 3 (Copy 2 - Node 4 (Copy 3 - Client Upload Failure Scenario: Single disk/node fails; Copies Surviving: 2 of 3; Data Available?: Yes Failure Scenario: Entire rack loses power; Copies Surviving: 1 of 3 (on different rack); Data Available?: Yes Failure Scenario: Two simultaneous node failures (different racks); Copies Surviving: 1 of 3; Data Available?: Yes In a cluster of thousands of commodity servers, component failure is not a risk - it is a mathematical certainty. A single server might have a 1% annual failure rate; at 1,000 nodes, multiple failures per week are expected. HDFS addresses this through replication, not RAID or specialised hardware Failure Scenario: Loss of 2 copies; Copies Surviving: 0 of 3; Data Available?: No - requires re-replication from remaining copy By default, HDFS stores three copies of every block. When a file is uploaded: Block 1 copy → local node (the node receiving the upload) Block 1 copy → different node on a different rack Concept flow: Large File (e.g. 1 GB - Block 1 (128 MB - Block 2 (128 MB - ... - Block 8 (128 MB - Copy on Node A - Copy on Node C (Rack 2 - Copy on Node D (Rack 2 Block 1 copy → another node on that same remote rack Why not put all 3 copies on one rack? A top-of-rack switch failure or power outage would destroy all copies simultaneously. Spreading copies across racks provides fault domain isolation Trade-off: 3x replication triples storage cost. For a petabyte dataset, you need ~3 PB of raw disk capacity. This is the price of durability without expensive enterprise storage arrays Blocks are the unit of replication - not entire files Blocks are immutable once written (append-only for HDFS files) NameNode maintains metadata mapping: file → list of block locations DataNodes store actual block replicas and report health to NameNode The block-replicate model creates a foundation for reliable batch processing: Data locality: Mappers can run on nodes that already hold block replicas Fault tolerance: Lost blocks are re-replicated from surviving copies automatically Parallelism: Different blocks are processed independently by different mappers However, this same model imposes constraints explored in subsequent topics: large blocks favour sequential access, and replication overhead applies to final outputs but not intermediate MapReduce spills

Must remember:

- HDFS uses large blocks (128/256 MB) to minimise seek overhead and maximise sequential read throughput
- The design optimises for batch processing, not low-latency random access
- Default 3x replication ensures durability when individual disks or entire racks fail
- Rack-aware placement puts copy 1 locally, copy 2 on a different rack, copy 3 on the same rack as copy 2
- Hardware failure at scale is expected, not exceptional - replication is the primary defence
- Block-replicate model is the bedrock of Hadoop reliability but triples storage requirements
- Blocks are the unit of replication and parallel processing, managed by NameNode metadata and DataNode storage

Formula / decision rule: Efficiency proportional to (Transfer Time)/(Seek Time + Transfer Time)

Exam trap: Trap: "HDFS block size = OS block size." HDFS blocks (128 MB) are logical units managed by the Name Node, unrelated to the underlying ext4/XFS 4 KB sectors Trap: "3 copies means 3x processing speed." Replication provides durability, not parallelism for reads (though HDFS can use closest replica for locality)

BD-W04-T03

MapReduce Data Flow on HDFS: From Blocks to Final Output

Tier A

Definition and why it matters: Knowing how HDFS stores blocks is only half the picture. MapReduce defines how computation flows across those blocks - where data moves, what gets written to HDFS vs local disk, and where network bottlenecks emerge. This end-to-end trace is the foundation for understanding why Hadoop incurs high I/O costs and why Spark's in-memory model is transformative

Core explanation: Moving terabytes across a cluster network is prohibitively slow. A 10 Gbps link transfers ~1.25 GB/s theoretically; reading 10 TB would take hours of pure network transfer before any computation begins

Concept flow: 10 TB Data - Central Compute Node - Map Code (KB - DataNode with Block - DataNode with Block - DataNode with Block Approach: Traditional; What Moves: Data - Code; Volume: Terabytes; Speed: Slow (network-bound) Approach: Hadoop/Spark; What Moves: Code - Data; Volume: Kilobytes; Speed: Fast (compute starts immediately)

Concept flow: HDFS Input Blocks (3x replicated - Map Phase: Read blocks locally, emit key-value pairs - Local Disk: Intermediate map output (NO HDFS replication - Shuffle & Sort: Network transfer to reducers - Reduce Phase: Aggregate by key - HDFS Final Output (3x replicated

Hadoop's solution: Send the map function (kilobytes of code) to the DataNode that already holds the HDFS block (gigabytes of data) Factor: Replication; HDFS Write: 3x copies across network; Local Disk Write: Single copy, no replication This principle - "don't move data, move code" - is shared by both Hadoop MapReduce and Apache Spark Factor: Durability needed?; HDFS Write: No - intermediate data is temporary; Local Disk Write: Correct - discard after reduce Factor: Speed; HDFS Write: Slower (replication pipeline); Local Disk Write: Faster (single local write) Each mapper is assigned one or more input splits (typically aligned with HDFS blocks) Factor: Fault recovery; HDFS Write: N/A for temp data; Local Disk Write: Re-run failed map task from HDFS input Mapper runs on (or near) the node holding the block replica - data-local execution Reads records sequentially from HDFS block Emits intermediate key-value pairs (e.g., (word, 1) for word count)

Must remember:

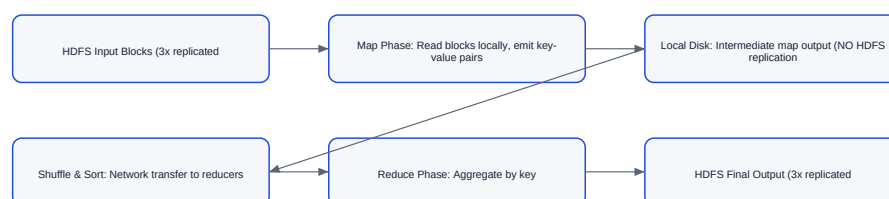
- Stage: Map input; Read From: HDFS blocks; Replicated?: N/A (read)
- Stage: Map output; Write To: Local disk; Replicated?: No
- Stage: Shuffle; Read From: Local disks (remote); Write To: Reducer memory; Replicated?: No
- Stage: Reduce output; Write To: HDFS; Replicated?: Yes (3x)
- The core Hadoop engine cycle:
- HDFS read - local disk write - network shuffle - HDFS write
- Every MapReduce job with multiple reduce groups pays this full cycle. Multi-stage jobs (chained MapReduce jobs) repeat it entirely - reading the previous job's HDFS output as the next job's input
- Hadoop follows data locality: send map code to nodes holding HDFS blocks, not data to a central processor
- Map phase reads HDFS blocks locally and emits intermediate key-value pairs
- Intermediate results go to local disk (not HDFS) - no 3x replication for temporary data
- Shuffle & sort is the most network-intensive phase, redistributing data to reducers by key
- Reduce phase aggregates shuffled data and writes final output to HDFS with full replication
- The cycle - HDFS read → local write → shuffle → HDFS write - is Hadoop's core engine
- Every disk write in this cycle contributes to the I/O latency bottleneck explored next

Formula / decision rule: HDFS read → local disk write → network shuffle → HDFS write

Example or contrast: Input: 500 GB of web server logs in HDFS (~= 4,000 blocks at 128 MB each)

Exam trap: Trap: "Intermediate map output goes to HDFS." Wrong - it goes to local disk without replication to avoid 3x overhead on temporary data Trap: "Shuffle is optional." Shuffle is mandatory whenever reducers need all values for a key from all mappers

MapReduce Data Flow on HDFS: From Blocks to Final Output



Vector reconstruction of the source Mermaid visual

Concept map: MapReduce Data Flow on HDFS: From Blocks to Final Output

BD-W04-T04 High I/O Latency and Materialisation Cost: Hadoop's Performance Bottleneck

Tier A

Definition and why it matters: Materialisation means persisting data to physical storage - making intermediate results "real" on disk before the next processing step can begin. In Hadoop MapReduce: Concept flow: Compute Stage 1 - Write to Disk - Read from Disk - Compute Stage 2 - Write to Disk

Core explanation: MapReduce is architecturally sound for fault-tolerant batch processing, yet complex jobs can run for hours. The root cause is not CPU inadequacy - modern clusters have enormous compute capacity sitting idle, waiting for disk I/O. Understanding materialisation cost explains why and identifies the workloads most affected. Algorithms like K-Means clustering or PageRank repeat the same logic on the same dataset until convergence: Iteration k: Read data - Compute - Write results - Repeat. In Hadoop, each iteration is a separate MapReduce job: Phase: HDFS read; Time per Iteration: 2 min; Total (50 iter): 100 min. Iteration 1 ends → write cluster assignments to HDFS Phase: Computation; Time per Iteration: 1 min; Total (50 iter): 50 min. Iteration 2 begins → read those assignments back from HDFS Phase: HDFS write + local spill; Time per Iteration: 3 min; Total (50 iter): 150 min. Repeat for 50, 100, or more iterations. Cost formula for iterative jobs: Total I/O Cost $\sim N(\text{iterations}) \times (\text{Read Cost} + \text{Write Cost})$. Concept flow: Iterative Hadoop Job - read + write. For 50 iterations on a 100 GB dataset, the cluster may spend more time on disk I/O than actual computation. Imagine writing a book where you must print, bind, and ship every single page to a warehouse before writing the next page. The writing itself takes minutes; the print-bind-ship cycle takes hours. That is materialisation cost in iterative MapReduce. Consider a K-Means job on 200 GB of data with 50 iterations: In this scenario, 83% of time is I/O - not computation. Adding more CPU cores does not help because cores wait on disk. The solution is not faster disks - it is avoiding disk round-trips for data that will be immediately reused: Keep intermediate results in memory across stages. Pipeline multiple transformations in a single pass. Cache datasets that multiple operations or iterations will access. This is precisely the architectural shift Apache Spark introduces.

Must remember:

- High I/O latency means the system spends most time waiting for disk reads and writes, not computing
- Materialisation = saving intermediate results to physical disk before the next step can proceed
- Disk I/O is orders of magnitude slower than RAM access
- Iterative algorithms (K-Means, PageRank) pay the disk tax on every iteration - often 50-100 times
- The read → process → write → repeat cycle creates massive overhead for ML workloads
- In iterative jobs, disk I/O time frequently exceeds computation time
- Hadoop accepts this trade-off for durability and cost on one-pass batch jobs
- The fix: keep reused data in memory - the foundation of Spark's in-memory computing model

Formula / decision rule: Iteration k: Read data → Compute → Write results → Repeat; Total I/O Cost $\sim N(\text{iterations}) \times (\text{Read Cost} + \text{Write Cost})$

Example or contrast: Operation: L1 cache reference; Approximate Latency: ~1 ns. Operation: Main memory (RAM) reference; Approximate Latency: ~100 ns. Operation: SSD random read; Approximate Latency: ~100 s

Exam trap: Hadoop's design prioritises: Fault tolerance - materialised data survives node failures

BD-W04-T05 Introduction to In-Memory Computing: RAM as Spark's Primary Workspace

Tier A

Definition and why it matters: The "disk tax" identified in Hadoop - materialising intermediate results to storage between every stage - is not a tuning problem. It is an architectural constraint. In-memory computing attacks the root cause by changing where the system does its work: from disk to RAM.

Core explanation: Aspect: Primary workspace; Hadoop MapReduce: Disk (HDFS + local spill); Apache Spark: RAM (executor memory). Aspect: Data lifecycle; Hadoop MapReduce: Read - process - write - repeat; Apache Spark: Read once - keep in memory - process repeatedly. Aspect: Optimised for; Hadoop MapReduce: One-pass batch, durability; Apache Spark: Iterative, interactive, multi-pass analytics. Aspect: Fault tolerance; Hadoop MapReduce: Replication (storage copies); Apache Spark: Lineage (recomputation). Aspect: Hardware profile; Hadoop MapReduce: Disk-heavy, cost-efficient; Apache Spark: RAM-heavy, performance-oriented. Concept flow: Read from HDFS - Process - Write to Disk - Read from Disk - Read from HDFS (once - Cache in RAM - Process). Iteration 1 - Process. Iteration 2 - Process. Iteration N. Accessing data in RAM is thousands of times faster than reading from a traditional hard drive. The gap is not marginal - it is architectural. RAM bandwidth: 50-100 GB/s per socket. HDD sequential read: 100-200 MB/s. Network (10 Gbps): ~1.25 GB/s. Phase: Iteration 1; Hadoop: Read HDFS - compute - write HDFS; Spark: Read HDFS - cache in RAM - compute. Once data resides in executor memory, subsequent operations access it at memory speed - no disk seek, no network re-fetch, no serialisation overhead. Phase: Iterations 2-100; Hadoop: Read HDFS - compute - write HDFS (each); Spark: Compute in RAM only. Phase: Disk reads (middle iterations); Hadoop: 99; Spark: 0. Spark still reads initial data from HDFS, S3, or other persistent storage. The in-memory advantage begins after the first load: Data is read from HDFS into executor RAM. Transformations operate on in-memory partitions. Optionally,

data is cached/persisted for reuse Subsequent operations or iterations access RAM directly For an algorithm requiring 100 iterations on the same dataset: The CPU's working set stays in the high-speed memory hierarchy - L3 cache and RAM - rather than round-tripping through storage In-memory computing does not make processing "a little faster." For complex, iterative workloads: Typical speedup: 10x to 100x over disk-bound MapReduce Greatest gains: algorithms that repeatedly touch the same data Modest gains: single-pass ETL where data is read once and discarded Real-world examples where in-memory wins: K-Means clustering (repeated centroid updates) Logistic regression / gradient descent (epoch-based training) Graph algorithms like PageRank (repeated edge traversals) Interactive data exploration (multiple queries on same dataset)

Must remember:

- In-memory computing treats RAM as the primary workspace, not disk
- Hadoop uses disk as workspace; Spark uses RAM - a fundamental architectural shift
- RAM access is thousands of times faster than disk I/O
- After the first HDFS read, Spark keeps data in executor memory for subsequent operations
- Iterative algorithms benefit most: iterations 2-N run entirely in RAM with zero disk reads/writes
- Performance gains of 10x-100x are common for iterative ML workloads
- Trade-off: RAM is more expensive than disk; Spark clusters need adequate memory capacity
- Golden rule: Keep reused data in memory to keep processing moving

Exam trap: Speed comes at a cost: Resource: HDD; Relative Cost (per GB): Lowest; Role in Spark: Archival, cold storage

BD-W04-T06 How Spark Minimises Disk Read-Write Cycles: DAG, Lazy Evaluation, and Persistence Tier A

Definition and why it matters: Knowing that RAM is faster than disk is necessary but not sufficient. Spark achieves its speed through three coordinated mechanisms that minimise the number of times data touches disk: the DAG execution model, lazy evaluation, and explicit persistence. Together, they replace Hadoop's rigid read-write-read cycle with intelligent in-memory pipelining

Core explanation: MapReduce sees every job as a fixed pipeline: Concept flow: `textFile()` - `flatMap()` - `map()` - `filter()` - `reduceByKey()` - `save()` Map - Shuffle - Reduce Each stage materialises output before the next begins. Chained jobs repeat the full cycle Spark builds a Directed Acyclic Graph (DAG) - a logical plan of all transformations before executing anything Properties of the DAG: Directed: Data flows in one direction (parent → child RDDs) Acyclic: No loops - each RDD is computed once per action Step: `rdd.map(f)`; What Happens: Record transformation in DAG; Data Touched?: No Global view: Spark sees the entire pipeline, not isolated stages Because Spark knows the full series of transformations upfront, it can: Pipeline narrow operations in a single memory pass Step: `rdd.count()` (action); What Happens: Optimise DAG, execute entire plan; Data Touched?: Yes Prune unnecessary steps from the graph Reorder operations for efficiency (e.g., push filters earlier) Avoid materialising intermediate results to disk When you order appetiser, main course, and dessert at a restaurant, the kitchen does not start cooking the appetiser the moment you mention it. It waits for the complete order to coordinate cooking times and kitchen resources efficiently Spark operates identically: Storage Level: MEMORYONLY; Behaviour: Keep in RAM; recompute if evicted Transformations are lazy - they build the recipe (lineage) without touching data Storage Level: MEMORYANDDISK; Behaviour: Spill to disk if RAM full Actions are eager - they trigger execution of the entire optimised plan Storage Level: DISKONLY; Behaviour: Persist on disk (faster re-read than HDFS) Combine steps: `map` + `filter` execute in one pass over data, not two Predicate pushdown: A filter at the end of the script can be moved to the beginning Skip unused branches: If only one column is needed, Spark avoids loading others Global optimisation: Catalyst optimizer evaluates multiple physical strategies In Hadoop, using the same dataset for two different tasks requires two full disk reads. In Spark, you can explicitly persist a dataset in memory: Concept flow: User Code: `map` - `filter` - `reduceByKey` - `count()` - DAG: Full logical plan built lazily - Catalyst Optimizer: Reorder, prune, pipeline - Persistence: Reuse cached partitions - Executors: Execute optimised stages in memory Dataset used in multiple actions or iterations Iterative algorithms (K-Means loops) Interactive exploration (repeated queries on same base data)

Must remember:

- Spark builds a DAG (Directed Acyclic Graph) representing the entire data pipeline before execution
- Unlike MapReduce's rigid map-shuffle-reduce, the DAG enables global optimisation across all transformations
- Lazy evaluation: transformations are recorded but not executed until an action is called
- Laziness allows pipelining multiple narrow operations in a single memory pass
- Persistence/caching keeps datasets in executor RAM for reuse across actions or iterations
- Together, DAG + laziness + persistence virtually eliminate unnecessary disk read-write cycles
- Persist only datasets with proven reuse to avoid wasting cluster memory

Formula / decision rule: Map → Shuffle → Reduce

Exam trap: Trap: "Lazy means slow." Lazy means deferred - execution is faster because of optimisation, not slower Trap: "Every RDD is automatically cached." Caching requires explicit `.persist()` or `.cache()` - otherwise Spark recomputes from lineage

BD-W04-T07 Hadoop vs Spark for Iterative Algorithms: A K-Means Performance Benchmark

Tier A

Definition and why it matters: Theoretical arguments about RAM speed are convincing but abstract. K-Means clustering provides a controlled, reproducible benchmark that exposes the architectural difference between Hadoop and Spark. K-Means is iterative (10-100+ passes over the same data), computationally moderate, and widely used - making it the canonical test case for comparing disk-bound vs in-memory architectures

Core explanation: K-Means partitions data into k clusters by repeatedly: Concept flow: Read HDFS - Compute - Write HDFS - Read HDFS - Compute - Write HDFS Assign each point to the nearest centroid Recompute centroids as the mean of assigned points Repeat until centroids converge $\text{centroid}_j^{(t+1)} = (1/|C_j|) \sum_{x \in C_j} x$ Concept flow: Read HDFS (once - Cache in RAM - Iteration 1 (RAM - Iteration 2 (RAM - Iteration N (RAM Each iteration reads the full dataset and current centroids. Convergence typically requires 10-100 iterations - the same data accessed repeatedly Each iteration is a separate MapReduce job: Metric: HDFS reads (middle iterations); Hadoop (100 iterations): 99; Spark (100 iterations): 0 Hadoop's cycle per iteration: Read HDFS - Process - Write HDFS - Repeat Metric: Data in RAM after iter 1; Hadoop (100 iterations): No; Spark (100 iterations): Yes For N iterations on dataset D: Metric: Typical speedup; Hadoop (100 iterations): Baseline; Spark (100 iterations): 10x-100x Total I/O $\approx N \times (|D|(\text{read}) + |D|(\text{write}))$ Observed result: Disk I/O time frequently exceeds computation time. The cluster's CPUs idle while waiting for storage Platform: Hadoop MapReduce; Execution Time: Hours; Dominant Cost: Disk I/O (60% of time) Platform: Apache Spark; Execution Time: Minutes; Dominant Cost: Computation Iteration 1: Read from HDFS $\rightarrow$ cache in executor RAM $\rightarrow$ compute Iterations 2-N: Compute entirely in RAM - zero disk reads, zero disk writes Factor: Hardware cost per TB; Hadoop (Disk-Heavy): Lower (commodity HDD); Spark (RAM-Heavy): Higher (RAM investment) Factor: Performance (iterative); Hadoop (Disk-Heavy): Slow; Spark (RAM-Heavy): Fast (10x-100x) Factor: Performance (one-pass ETL); Hadoop (Disk-Heavy): Adequate; Spark (RAM-Heavy): Good (modest gains) Factor: Scalability model; Hadoop (Disk-Heavy): Add disks cheaply; Spark (RAM-Heavy): Add RAM expensively Factor: Best for; Hadoop (Disk-Heavy): Archival, cold storage, simple batch; Spark (RAM-Heavy): ML training, interactive analytics Typical K-Means benchmark on a multi-GB dataset with 20-100 iterations: The speedup is not from faster algorithms - both run identical K-Means logic. The speedup is purely architectural: eliminating repeated materialisation Algorithm: PageRank; Iterative Pattern: Repeated edge propagation; Spark Advantage: High Algorithm: Gradient descent; Iterative Pattern: Repeated epoch over training set; Spark Advantage: High Algorithm: EM algorithm; Iterative Pattern: Repeated expectation-maximisation; Spark Advantage: High

Must remember:

- K-Means is the canonical benchmark: iterative, repeated data access, moderate computation
- Hadoop runs each iteration as a separate MapReduce job with full HDFS read-write cycles
- Disk I/O time often exceeds computation time in Hadoop iterative jobs
- Spark reads HDFS once, caches in RAM, and runs iterations 2-N entirely in memory
- Middle iterations have zero disk reads and zero disk writes in Spark
- Spark achieves 10x-100x speedup for iterative workloads like K-Means
- Trade-off: Spark requires RAM-heavy clusters at higher cost than disk-heavy Hadoop
- Choose Spark when algorithms repeatedly access the same data; choose Hadoop for cost-effective one-pass batch and archival storage

Formula / decision rule: $\text{centroid}_j^{(t+1)} = (1/|C_j|) \sum_{x \in C_j} x$; Read HDFS $\rightarrow$ Process $\rightarrow$ Write HDFS $\rightarrow$ Repeat

Exam trap: Trap: "Spark is always 100x faster." The 100x figure applies to iterative workloads, not all jobs Trap: "Hadoop cannot run K-Means." It can - just slowly, because each iteration is a separate MapReduce job with full disk I/O

BD-W04-T08 Module Summary: Hadoop vs Spark - Batch Processing Architecture Evolution

Tier C

Definition and why it matters: This module traced the evolution of batch processing from Hadoop's disk-bound reliability model to Spark's in-memory performance model. The central tension - durability vs speed - resolves differently depending on workload characteristics and budget constraints

Core explanation: HDFS prioritises data durability through its block-and-replicate strategy: Concept flow: User Data - Large Blocks (128 MB - Replica 1 (local node - Replica 2 (remote rack - Replica 3 (remote rack

Must remember:

- Hadoop ensures durability via HDFS large blocks + 3x rack-aware replication
- MapReduce's disk materialisation creates high I/O latency - the primary bottleneck
- Materialisation cost devastates iterative workloads (K-Means, PageRank, gradient descent)
- Spark uses RAM as primary workspace with DAG planning, lazy evaluation, and caching
- K-Means benchmarks show up to 100x speedup for Spark on iterative workloads
- Hadoop is cost-effective for archival storage and one-pass batch; Spark is fast for repeated data access
- The architectural choice depends on performance needs, data access patterns, and budget - not raw speed alone

Exam trap: Dimension: Primary storage; Hadoop: Disk (HDFS); Spark: RAM (with HDFS/S3 as source) Dimension: Fault tolerance; Hadoop: Replication (3x copies); Spark: Lineage (recomputation)

Week 4 Exam Lens

Likely questions

- Define and explain Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing; include the mechanism and one limitation.
- Compare Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing with HDFS Deep Dive: Blocks, Replication, and Rack-Aware Fault Tolerance and state when each is preferred.
- Apply MapReduce Data Flow on HDFS: From Blocks to Final Output to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Efficiency proportional to $(\text{Transfer Time})/(\text{Seek Time} + \text{Transfer Time})$; HDFS read $\rightarrow$ local disk write $\rightarrow$ network shuffle $\rightarrow$ HDFS write; Iteration k : Read data $\rightarrow$ Compute $\rightarrow$ Write results $\rightarrow$ Repeat; Total I/O Cost $\sim N_{\text{iterations}} \times (\text{Read Cost} + \text{Write Cost})$; Map $\rightarrow$ Shuffle $\rightarrow$ Reduce; $\text{centroid}_j^{(t+1)} = (1/|C_j|) \sum_{x \in C_j} x$; Read HDFS $\rightarrow$ Process $\rightarrow$ Write HDFS $\rightarrow$ Repeat

Mistakes to avoid: Trap: "Spark replaces HDFS entirely." Reality: Spark commonly reads from HDFS/S3; it replaces MapReduce's execution model, not distributed storage Trap: "HDFS block size = OS block size." HDFS blocks (128 MB) are logical units managed by the NameNode, unrelated to the underlying ext4/XFS 4 KB sectors Trap: "Intermediate map output goes to HDFS." Wrong - it goes to local disk without replication to avoid 3x overhead on temporary data

Week 5: Spark Core I: RDD Fundamentals

Week roadmap: Spark Core and RDD Fundamentals: Module Overview → Resilient Distributed Datasets (RDDs): Definition and Anatomy → RDD Resilience and Distribution: Lineage vs Replication → RDD Immutability: Why Spark Data Never Changes → RDD Partitioning: The Atomic Unit of Parallelism in Spark → Data Locality and the Spark Cluster: Move Code, Not Data → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W05-T01 Spark Core and RDD Fundamentals: Module Overview

Tier C

Definition and why it matters: Spark's higher-level APIs - DataFrames, Datasets, Spark SQL, MLlib, Structured Streaming - all compile down to the same execution engine. At the heart of that engine lies the Resilient Distributed Dataset (RDD), Spark's original and most fundamental abstraction. Understanding RDDs means understanding how Spark represents data, achieves fault tolerance, enables parallelism, and interfaces with storage systems

Core explanation: Without RDD fundamentals, performance tuning, debugging failed jobs, and reasoning about shuffle boundaries remain opaque

Must remember:

- RDD (Resilient Distributed Dataset) is the core building block of Spark's execution engine
- Module covers: definition, properties (resilience, immutability, partitioning), locality, and creation
- Immutability ensures safe parallel access and simplifies fault recovery
- Partitioning divides data into chunks processed in parallel - one partition per core
- Two creation methods: `parallelize()` for local collections, `textFile()` for external files
- Hands-on PySpark skills bridge theory to production distributed applications
- Higher-level APIs (DataFrames, Spark SQL) build on RDD foundations

Exam trap: Trap: "RDDs are obsolete - only use DataFrames." DataFrames compile to RDDs internally; understanding RDDs is essential for debugging and performance tuning Trap: Treating this module as purely theoretical. The creation methods (`parallelize`, `textFile`) are production-relevant skills

BD-W05-T02 Resilient Distributed Datasets (RDDs): Definition and Anatomy

Tier A

Definition and why it matters: Before Spark, distributed processing meant MapReduce jobs with rigid map-shuffle-reduce stages, disk-bound intermediates, and no way to express complex multi-step pipelines efficiently. Spark's creators needed an abstraction that was:

Core explanation: Fault-tolerant without expensive replication at every step Distributed across thousands of nodes yet programmable as a single collection Composable - chain transformations without materialising to disk The RDD satisfies all three requirements. It is the data structure that made in-memory distributed computing practical In a cluster of hundreds or thousands of machines, something will fail - a disk crashes, a network cable disconnects, an executor runs out of memory. "Resilient" means Spark can automatically recover lost data without permanent job failure Recovery mechanism: lineage (covered in depth in the next topic) - Spark remembers how each partition was computed and can recompute it on a surviving node Analogy: A resilient system is like a baker who keeps the recipe, not just the cake. If the cake is dropped, bake another from the recipe Concept flow: Partition 0 - Partition 1 - Partition 2 - Partition 3 - Partition 4 - Driver: Logical RDD API - Lineage Graph: Recovery Recipe A dataset too large for one machine - perhaps a petabyte of web logs - is split into partitions and spread across the RAM and disk of many cluster nodes. Despite physical distribution, Spark presents the RDD as one logical collection that you manipulate with single operations like `.map()` or `.filter()` Analogy: A distributed dataset is like a library where books are shelved in different buildings, but the catalog system lets you search the entire collection as one At its core, an RDD is simply a collection of elements: A list of integers: `[1, 2, 3, 4, 5]` Rows of text from a log file Complex customer records with multiple fields The "dataset" part grounds the abstraction in familiar programming - it is data you transform, filter, aggregate, and analyse RDD = $\text{underbrace(Collection of objects)(Dataset)} \times \text{underbrace(Spread across nodes)(Distributed)} \times \text{underbrace(Self-healing via lineage)(Resilient)}$ Before RDDs, expressing a 5-step data pipeline in MapReduce required 5 separate jobs, each with full disk I/O. With RDDs: Five transformations, one logical pipeline, executed in memory with automatic parallelism and fault tolerance. This composability - chaining operations on a distributed collection - is what made Spark transformative

Must remember:

- RDD = Resilient Distributed Dataset - Spark's fundamental data abstraction
- Resilient: fault-tolerant via lineage-based automatic recomputation on failure
- Distributed: data spread across cluster nodes as partitions, operated on as one logical unit
- Dataset: a collection of objects (numbers, strings, records) - the actual data being processed

- RDDs enable composable, in-memory, parallel processing impossible with chained MapReduce jobs
- Modern DataFrames/Datasets compile to RDDs - understanding RDDs is essential for debugging and tuning
- RDDs are immutable - transformations always create new RDDs

Formula / decision rule: $\text{RDD} = \text{underbrace}(\text{Collection of objects})_{\text{(Dataset)}} \times \text{underbrace}(\text{Spread across nodes})_{\text{(Distributed)}} \times \text{underbrace}(\text{Self-healing via lineage})_{\text{(Resilient)}}$

Example or contrast: Property: Location; Python List: One machine's RAM; Pandas DataFrame (single machine): One machine's RAM; Spark RDD: Distributed across cluster Property: Max size; Python List: RAM of one machine; Pandas DataFrame (single machine): RAM of one machine; Spark RDD: Cluster aggregate RAM + disk Property: Fault tolerance; Python List: None; Pandas DataFrame (single machine): None; Spark RDD: Lineage-based recomputation

Exam trap: Trap: "Resilient means replicated." Resilience comes from lineage (recomputation), not storing multiple copies Trap: "Distributed means replicated 3x." Distribution means partitioned across nodes; replication is an HDFS concept, not an RDD property

BD-W05-T03 RDD Resilience and Distribution: Lineage vs Replication

Tier A

Definition and why it matters: In a cluster of thousands of commodity servers, hardware failure is routine - not exceptional. Traditional fault tolerance copies data to multiple locations (replication), which is expensive in both storage and network bandwidth. Spark's RDD model achieves equivalent reliability through a fundamentally different mechanism: lineage-based recomputation. This design is what makes in-memory processing economically viable at scale

Core explanation: Traditional systems (HDFS, databases) ensure durability by storing multiple physical copies: Concept flow: `textFile('logs' - flatMap(split - map(toLower - filter(len3 - reduceByKey(+ Fault tolerance cost = $N(\text{copies}) \times \text{Storage size} \times \text{Network bandwidth for sync}$ For in-memory data replicated 3x across a cluster, RAM requirements triple - prohibitively expensive Instead of copying data, Spark records how data was created - a directed graph of transformations from source to current RDD: This graph is the lineage (also called the RDD lineage graph or dependency graph) On failure: If a worker node crashes and partition data is lost, Spark: Identifies which partition was lost Traces lineage back to the nearest reliable source (HDFS block or cached parent) Recomputes only the lost partition on a different node Continues the job without user intervention Recovery cost = Recompute lost partition only, not entire dataset Concept flow: Partition 0 \n~128 MB - Partition 1 \n~128 MB - Partition 2 \n~128 MB - Partition N \n~128 MB - Node 1 RAM - Node 2 RAM - Node 3 RAM - Node K RAM Setup: Single machine; Processing Time (example): Days Setup: 100-node cluster, 100 partitions; Processing Time (example): Minutes Property: Resilience (lineage); Provides: Safety; Enables: Run on thousands of unreliable machines without fear of permanent data loss A petabyte of web logs cannot fit on any single machine. Even if it could, processing it sequentially would take days or weeks Property: Distribution (partitioning); Provides: Power; Enables: Process datasets of any scale by adding nodes Spark divides the dataset into partitions and spreads them across cluster nodes: Property: Together; Provides: Scale + reliability; Enables: Petabyte pipelines on commodity hardware Each node holds a small portion in its RAM (or spills to local disk). Operations like .map() or .filter() execute simultaneously on all partitions across all nodes This is the divide and conquer strategy: harness collective cluster power to shrink wall-clock time from days to minutes Without resilience, a single node failure kills the entire job. Without distribution, the dataset cannot fit or be processed in reasonable time. RDDs provide both simultaneously`

Must remember:

- Resilience = fault tolerance through lineage (computation history), not replication
- Lineage records every transformation as a DAG from source to current RDD
- On failure, Spark recomputes only lost partitions from lineage - self-healing through computation
- Lineage is more storage-efficient than replication for intermediate data
- Distribution spreads data across cluster nodes as partitions for parallel processing
- Divide and conquer: tasks that take days on one machine complete in minutes on thousands
- Resilience provides safety; distribution provides power - together they enable petabyte-scale processing
- HDFS replication protects source data; RDD lineage protects computed data

Formula / decision rule: $\text{Fault tolerance cost} = N_{\text{(copies)}} \times \text{Storage size} \times \text{Network bandwidth for sync}$; $\text{Recovery cost} = \text{Recompute lost partition only, not entire dataset}$

Example or contrast: Aspect: Fault tolerance mechanism; Replication: Store N copies; Lineage (Spark): Record computation history Aspect: Storage overhead; Replication: $N \times \text{data size}$; Lineage (Spark): Minimal (graph metadata only) Aspect: Recovery action; Replication: Read from backup copy; Lineage (Spark): Recompute from parent

Exam trap: Trap: "Resilient = data is replicated in memory." Spark uses lineage, not in-memory replication, for RDD fault tolerance Trap: "Lineage means no data loss ever." If the source data (HDFS) is lost and no cache exists, lineage cannot help

BD-W05-T04

RDD Immutability: Why Spark Data Never Changes

Tier A

Definition and why it matters: In a distributed system with dozens of machines reading and writing the same data simultaneously, mutable state is a recipe for corruption, race conditions, and unrecoverable failures. Spark's insistence that RDDs never change after creation is not a stylistic preference - it is a foundational design decision that enables fault tolerance, eliminates locking overhead, and simplifies reasoning about distributed computation

Core explanation: Once an RDD is created, its contents are frozen. Any operation that appears to "modify" data actually creates a new RDD: Concept flow: `baseprices` \n [50,100,150,200] - `p-10` - `discounted` \n [40,90,140,190] - `100` - `premium` \n [140,190] Mutable RDD: Need locks/mutexes to prevent corruption; Immutable RDD: No locks needed - data never changes Mutable RDD: Race conditions possible; Immutable RDD: Concurrent reads are always safe Mutable RDD: Complex coordination protocol; Immutable RDD: Trivial: read-only access Mutable RDD: Deadlock risk; Immutable RDD: No deadlock possible Three separate RDDs coexist. The original is never touched If a worker node crashes and loses partition data from the premium RDD, Spark needs to recompute it. Because `baseprices` and `discounted` were never modified, their lineage is intact and deterministic: `premium partition = filter(100)(map(-10)(baseprices partition))` Recomputation produces identical results because inputs never changed. If RDDs were mutable, a concurrent write during recovery could produce different results - corrupting the pipeline Imagine 50 executors simultaneously reading and updating the same RDD: Operation Type: Transformation; Creates New RDD?: Yes (new RDD); Executes Immediately?: No (lazy); Example: `map()`, `.filter()`, `.flatMap()` With immutability, every node reads data knowing it will never suddenly change underneath them. This eliminates an entire class of distributed systems bugs Operation Type: Action; Creates New RDD?: No (returns value or writes); Executes Immediately?: Yes (triggers execution); Example: `count()`, `.collect()`, `.save()` A common misconception: "Creating new RDDs for every transformation must waste memory." Reality: Spark does not duplicate data unless necessary: Narrow transformations (`map`, `filter`) can pipeline without materialising intermediate RDDs Code pattern: `baseprices = sc.parallelize([50, 100, 150, 200])`; Transformation: creates NEW RDD, `baseprices` unchanged; `discounted = baseprices.map(lambda p: p - 10)`; `discounted = [40, 90, 140, 190]`; Another transformation: creates ANOTHER new RDD; `premium = discounted.filter(lambda p: p > 100)`; `premium = [140, 190]`; `baseprices` is STILL [50, 100, 150, 200] Lineage graph stores transformation functions (kilobytes), not data copies (gigabytes) Data is materialised only when an action triggers execution or `.persist()` is called Unreferenced RDDs are garbage collected by the JVM Memory cost $\sim$ Cached/persisted RDDs only, not every intermediate Transformations on immutable RDDs build the lineage graph lazily. Actions trigger computation of the entire graph

Must remember:

- RDDs are immutable - once created, they never change
- Transformations create new RDDs; the original remains untouched
- Immutability enables deterministic lineage-based recovery - recomputation produces identical results
- No locks needed - concurrent reads are always safe on immutable data
- Spark stores transformation functions in lineage, not redundant data copies
- Only persisted/cached RDDs and action-triggered results materialise data in memory
- Always assign transformation results to a new variable - discarding them loses the computation
- Immutability is foundational to Spark's fault tolerance and concurrency safety

Formula / decision rule: `premium partition = filter_(>100)(map_(-10)(base_prices partition))`; Memory cost $\sim$ Cached/persisted RDDs only, not every intermediate

Exam trap: Trap: "Immutability means Spark copies all data on every transformation." Spark pipelines narrow transformations and stores functions in lineage, not data copies Trap: "I can update an RDD with `.map()`." `.map()` returns a new RDD; the original is unchanged

BD-W05-T05

RDD Partitioning: The Atomic Unit of Parallelism in Spark

Tier A

Definition and why it matters: A partition is a logical slice of an RDD's data - the smallest unit of work in Spark. Each partition is a contiguous subset of elements that can be processed independently Concept flow: RDD: 1 Million Records - Partition 0 \n 250K records - Partition 1 \n 250K records - Partition 2 \n 250K records - Partition 3 \n 250K records - Core 0 - Core 1 - Core 2 - Core 3

Core explanation: An RDD spread across a 100-node cluster is useless if only one node does the work. Partitioning is the mechanism that converts a logically unified dataset into physically independent chunks, each processable by a separate CPU core. Without correct partitioning, cluster resources sit idle and jobs run no faster than on a single machine Imagine scanning a massive library of 100,000 books: Approach: One person scans all books; Workers: 1; Time: 10 hours Approach: Split into 10 piles, 10 people scan in parallel; Workers: 10; Time: 1 hour Cluster Cores: 100 cores; Partitions: 1 partition; Result: 99 cores idle; $\sim$ same speed as 1 core Cluster Cores: 100 cores; Partitions: 10 partitions; Result: 90 cores idle; 10x speedup Cluster Cores: 100 cores; Partitions: 100 partitions; Result: All cores busy; $\sim$ 100x speedup Cluster Cores: 100 cores; Partitions: 500 partitions; Result: All cores busy;

slight scheduling overhead Configuration: 1 core, 1 partition; Relative Time (example): 10 hours (baseline) Partitions are the "piles." More piles with more workers → proportionally faster completion Configuration: 10 cores, 10 partitions; Relative Time (example): 1 hour (10x faster) Configuration: 100 cores, 100 partitions; Relative Time (example): 6 minutes (~100x faster) Creation Method: `sc.parallelize(data, numSlices=N)`; Default Partitioning: N partitions (user-specified) Creation Method: `sc.textFile(path, minPartitions=N)`; Default Partitioning: One partition per HDFS block (or `minPartitions`, whichever is larger) Creation Method: `rdd.repartition(N)`; Default Partitioning: Exactly N partitions (triggers shuffle) Target: Partitions $\approx$ number of available cores (typically 2-4x for I/O-bound workloads) Creation Method: `rdd.coalesce(N)`; Default Partitioning: Reduce to N partitions (narrow, no shuffle if reducing) Spark's core value proposition is linear scalability - doubling partitions and cores roughly halves processing time (until I/O or shuffle bottlenecks dominate): Creation Method: Transformations (map, filter); Default Partitioning: Same number as parent (narrow) Time proportional to (Data size)/(Partitions x Processing rate per core) Creation Method: Wide transformations (groupByKey); Default Partitioning: Configurable via `spark.default.parallelism` Concept flow: Massive Dataset - Split into Partitions - Worker 1: P0, P1 - Worker 2: P2, P3 - Worker 3: P4, P5 - Driver collects results Each worker processes its assigned partitions independently. Results are combined only when necessary (actions, wide transformations)

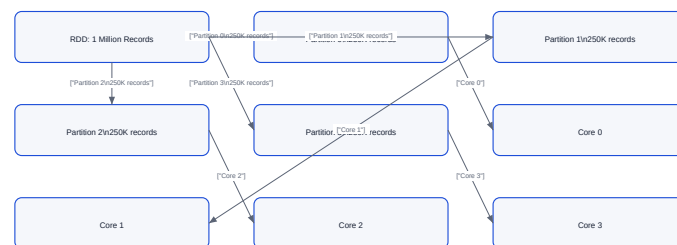
Must remember:

- Partitions are logical chunks of an RDD - the atomic units of parallelism in Spark
- One partition $\approx$ one CPU core in one task during execution
- More partitions with more cores enable linear scalability (hours → minutes)
- Default partition count depends on creation method (HDFS blocks for `textFile`, user-specified for `parallelize`)
- Too few partitions leave cores idle; too many create scheduling overhead
- Partitioning enables Spark's divide-and-conquer strategy across the cluster
- Partition count should be tuned to match available cluster cores (typically 2-4x core count)

Formula / decision rule: $\approx$; Time proportional to (Data size)/(Partitions x Processing rate per core)

Exam trap: Problem: Too few partitions; Symptom: Cores idle, slow job; Fix: `repartition()` or increase `minPartitions`
 Problem: Too many partitions; Symptom: Task scheduling overhead, small tasks; Fix: `coalesce()` to reduce

RDD Partitioning: The Atomic Unit of Parallelism in Spark



Vector reconstruction of the source Mermaid visual

Concept map: RDD Partitioning: The Atomic Unit of Parallelism in Spark

BD-W05-T06

Data Locality and the Spark Cluster: Move Code, Not Data

Tier A

Definition and why it matters: Partitioning divides work across cores, but where that work executes determines whether the cluster achieves peak throughput or wastes time waiting on network transfers. Data locality - scheduling computation on the node that already holds the data - is the principle that prevents network bandwidth from becoming the bottleneck in petabyte-scale processing

Core explanation: In conventional programming, data is small and code is complex: What: Data (partition); Size: Megabytes to gigabytes; Movement Cost: Expensive to move What: Code (map/filter function); Size: Kilobytes; Movement Cost: Cheap to move Concept flow: 10 GB Data - Central CPU - 2 KB Function - Node with 10 GB Data The data moves to where the programme runs. This works when data fits in one machine's memory Moving 10 TB across a 10 Gbps network: Transfer time = (10 TB)/(1.25 GB/s) $\approx$ 2.3 hours Before a single line of computation executes, hours are spent on transfer. CPUs sit idle. This is the network bottleneck Spark flips the model: Concept flow: HDFS: Block locations - S3: Object locations - Spark DAG Scheduler - Executor on Server A - Executor on Server B - Executor on Server C Motto: "Don't move your data - move code." Level: PROCESSLOCAL; Description: Data in same JVM as executor; Performance: Best Level: NODELOCAL; Description: Data on same node, different JVM; Performance: Good The computation task (serialized function + partition metadata) ships to the executor that already holds - or is nearest to - the data partition Level: RACKLOCAL; Description: Data on same rack;

Performance: Acceptable Spark's DAG scheduler communicates with the cluster manager and storage system (HDFS, S3) to determine where each partition lives: Level: NOPREF; Description: No preference; Performance: Neutral Query HDFS NameNode: "Which DataNode holds block replica for partition 3?" Level: ANY; Description: Data anywhere in cluster; Performance: Worst (network transfer) Answer: "Server A, Rack 1" Mechanism: Partitioning; Solves: Divides work for parallelism; Analogy: Splitting books into piles Schedule partition 3's task on an executor running on Server A Mechanism: Data locality; Solves: Places work near data; Analogy: Assigning each pile to the librarian nearest its shelf Code pattern: `data = readfromdatabase()` Megabytes; result = `process(data)` Code runs locally Spark prefers scheduling in this order: Together they ensure: Every core has work (partitioning) Every task runs where data already lives (locality) Network pipes carry kilobytes of code, not gigabytes of data Data locality applies to narrow transformations where each output partition depends on exactly one input partition. Wide transformations (`groupBy Key`, `join`) require a shuffle - redistributing data across the network by key. During shuffle, data locality is intentionally violated to colocate related keys

Must remember:

- Moving terabytes across a network creates a network bottleneck - CPUs wait for data
- Spark's principle: "Move code, not data" - ship kilobytes of functions to gigabytes of data
- Spark is data-aware - queries HDFS/S3 for block locations before scheduling tasks
- Tasks are scheduled on executors nearest to the data partition (PROCESSLOCAL NODELOCAL RACKLOCAL)
- Partitioning provides parallelism; data locality ensures efficient execution
- External file loading (`textFile`) respects locality; `parallelize()` does not (data starts on driver)
- Wide transformations (`shuffle`) intentionally break locality to colocate keys

Formula / decision rule: Transfer time = (10 TB)/(1.25 GB/s) ≈ 2.3 hours

Example or contrast: Scenario: 1 TB log file in HDFS across a 50-node cluster `sc.textFile("hdfs://logs/")` creates ~8,000 partitions (one per HDFS block) Spark queries NameNode for block locations

Exam trap: Trap: "Data locality means zero network traffic." Shuffles deliberately move data across the network for wide transformations Trap: "Spark always achieves PROCESSLOCAL." If the preferred executor is busy, Spark may schedule on a less local node

BD-W05-T07

Creating RDDs from Local Collections: `parallelize()`

Tier A

Definition and why it matters: Not all data starts in HDFS or S3. During development, testing, and prototyping, you need to convert small in-memory Python collections into distributed RDDs. The `parallelize()` method is Spark's bridge from local code to the cluster - fast, simple, and deliberately limited in scale

Core explanation: `sc.parallelize()` takes a collection already in your Python programme (list, tuple, range) and distributes it across the cluster as an RDD: Concept flow: Driver Memory → [List of names - Partition 0 - Worker 1 - Partition 1 - Worker 2 - Partition 2 - Worker 3] Parameter: `numSlices` omitted; Effect: Defaults to `spark.default.parallelism` (usually total cores) Parameter: `numSlices=1`; Effect: Entire collection on one partition (no parallelism) Parameter: `numSlices=N`; Effect: N partitions distributed across workers Spark takes the list from the driver's memory Splits it into partitions (default: total cores in cluster) Serialises and ships each partition to a worker node Returns an RDD reference to the driver Code pattern: `from pyspark.sql import SparkSession; spark = SparkSession.builder.appName("demo").getOrCreate(); sc = spark.sparkContext; names = ["Alice", "Bob", "Charlie", "Diana", "Eve"]; names_rdd = sc.parallelize(names); print(names_rdd.collect()) ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']; print(type(names_rdd))` Code pattern: Default: one partition per core; `rdd = sc.parallelize(range(1000))`; Explicit: 4 partitions; `rdd = sc.parallelize(range(1000), numSlices=4)`

Must remember:

- `sc.parallelize(collection, numSlices=N)` converts a local Python collection into a distributed RDD
- Spark chops the collection into partitions and ships them from driver to workers
- Primary use cases: prototyping, testing, synthetic data, learning exercises
- Scale limit: data must fit in driver RAM - not suitable for large datasets
- Specify `numSlices` to control partition count and parallelism
- For production-scale data, use `textFile()` or `DataFrame` readers instead
- `parallelize()` does not benefit from data locality (unlike external file loading)

Formula / decision rule: Max data size for `parallelize()` ≤ Driver RAM

Example or contrast: Use Case: Prototyping; Example: Test a `.map()` / `.filter()` logic before running on production data Use Case: Unit testing; Example: Verify transformation correctness with known small inputs Use Case: Synthetic data; Example: Generate test datasets for simulations

Exam trap: Critical constraint: Data must fit in the driver machine's RAM before distribution Data Size: 100 elements; `parallelize()`: Works perfectly; `textFile()`: Overkill

BD-W05-T08 Loading External Data: HDFS, S3, and Local Filesystems

Tier A

Definition and why it matters: Most production datasets do not start as Python lists in memory. They live in distributed filesystems, cloud object stores, or databases - often at terabyte or petabyte scale. Spark's ability to read directly from these external data sources without pulling everything onto a single machine is what makes it viable for real-world pipelines

Core explanation: The core idea: when you call `sc.textFile(path)`, Spark does not load the entire file into the driver's RAM. It creates a distributed RDD whose partitions are read in parallel by cluster nodes, each responsible for its own slice of the data Concept flow: Local Filesystem\(\dev / small projects - Cloud Storage\(\nS3 Azure Blob GCS - HDFS\(\n(on-prem Hadoop clusters - SparkContext.textFile(path - Distributed RDD\(\n(partitions read in parallel - Node 1 reads slice - Node 2 reads slice - Node N reads slice Source: Local filesystem; Typical Use: Development, unit tests, small datasets on your laptop; Scale: MB-low GB Source: Cloud storage (S3, Azure Blob, GCS); Typical Use: Cloud-native data lakes, modern production pipelines; Scale: TB-PB Source: HDFS; Typical Use: Traditional on-premise Hadoop clusters, legacy enterprise data; Scale: TB-PB Paths like `/home/user/data/logs.txt` or `file:///path/to/data`. Suitable when the dataset fits comfortably on one machine and you are prototyping logic Amazon S3, Azure Blob Storage, and Google Cloud Storage buckets are the default home for modern data lakes. Spark reads via URI schemes such as `s3a://bucket/path/` (preferred over deprecated `s3n://`). Data stays in the cloud; executors pull only the blocks they need The classic standard for large on-premise clusters. Files are split into blocks (typically 128 MB) and replicated across DataNodes. Spark on YARN or a Hadoop cluster reads via `hdfs://namenode:8020/path/to/file` What happens under the hood: Spark parses the path and determines input format (text, by default one line per record) It computes how many partitions to create (based on file size, `minPartitions`, or cluster defaults) Each partition is assigned to an executor task that reads only its byte range from the source No single node holds the full dataset - work is spread across the cluster This design directly supports data locality: tasks run on (or near) nodes that already store the HDFS blocks, minimising network transfer

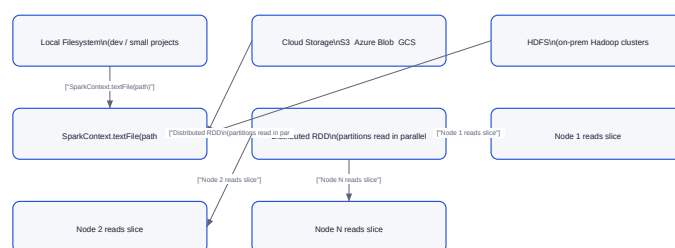
Must remember:

- Production data lives in local FS, cloud buckets (S3/Azure/GCS), or HDFS - not in driver memory
- `sc.textFile(path)` creates a distributed RDD; each node reads its own partition in parallel
- `parallelize`: data already in code; limited by driver RAM; good for testing
- External load: data on disk/object store; scales to petabytes; respects data locality
- For real pipelines, almost always load from S3 or HDFS, not `parallelize`
- External loading is faster at scale because executors read where data already lives
- Spark supports glob paths and configurable `minPartitions` for tuning parallelism
- The choice of creation method is fundamentally a scale and data-location decision, not a syntax preference

Example or contrast: Choosing the right creation method is one of the most common design decisions in Spark jobs Criterion: Data origin; `sc.parallelize(localList)`: Data already in driver memory as a Python/Scala collection; External load (`textFile`, etc.): Data stored as files on disk or object store Criterion: Scalability; `sc.parallelize(localList)`: Limited by driver RAM - a 1 TB list will crash the driver; External load (`textFile`, etc.): Scales to petabytes - work distributed across cluster

Exam trap: Assuming `textFile` loads everything into the driver - it creates a lazy RDD; data is read partition-by-partition when an action runs Using `parallelize` on large production datasets - driver memory becomes the hard ceiling; this is a classic exam trap contrasting the two creation methods

Loading External Data: HDFS, S3, and Local Filesystems



Vector reconstruction of the source Mermaid visual

Concept map: Loading External Data: HDFS, S3, and Local Filesystems

BD-W05-T09 Module Summary: RDD Fundamentals

Tier C

Definition and why it matters: A Resilient Distributed Dataset (RDD) is Spark's core abstraction - a virtual collection that behaves like a single list but is physically spread across the entire cluster

Core explanation: This module established the static foundation of Apache Spark - how data is represented, stored, partitioned, and brought into the cluster. Before manipulating data with transformations and actions (covered next), you need a solid mental model of what an RDD is and why Spark organises data the way it does Concept flow: 1. What is an RDD? - 2. Core Properties\n(resilience + immutability - 3. Internal Logic\n(partitioning + locality - 4. Creation Methods\n(parallelize vs external - Next: Transformations\n& Spark execution

Must remember:

- An RDD is a fault-tolerant, partitioned, immutable distributed collection
- Resilience comes from lineage - lost partitions are recomputed, not restored from replicas
- Immutability ensures consistency and safe parallel processing across the cluster
- Partitioning splits data so every CPU core can work in parallel
- Data locality schedules tasks where data already lives, minimising network transfer
- parallelize: in-memory collections, driver-RAM limited, for testing
- textFile / external sources: HDFS, S3, local files - production scale, distributed reads
- This module is the static foundation; next up is dynamic processing via transformations and Spark's execution engine

Formula / decision rule: Fault tolerance cost $\sim$ = recompute lost partitions, not replicate entire dataset; tasks, up to

Exam trap: Confusing resilience with replication - RDDs use lineage/recomputation, not 3x in-memory copies like HDFS blocks Thinking RDDs are mutable - operations like map return new RDDs; the original is unchanged

Week 5 Exam Lens**Likely questions**

- Define and explain Resilient Distributed Datasets (RDDs): Definition and Anatomy; include the mechanism and one limitation.
- Compare Resilient Distributed Datasets (RDDs): Definition and Anatomy with RDD Resilience and Distribution: Lineage vs Replication and state when each is preferred.
- Apply RDD Immutability: Why Spark Data Never Changes to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $RDD = \underbrace{\text{Collection of objects}}_{\text{Dataset}} \times \underbrace{\text{Spread across nodes}}_{\text{Distributed}} \times \underbrace{\text{Self-healing via lineage}}_{\text{Resilient}}$; Fault tolerance cost = $N_{\text{(copies)}} \times \text{Storage size} \times \text{Network bandwidth for sync}$; Recovery cost = Recompute lost partition only, not entire dataset; premium partition = $\text{filter}_{(>100)}(\text{map}_{(-10)}(\text{base_prices partition}))$; Memory cost $\sim$ = Cached/persisted RDDs only, not every intermediate; $\sim$ =; Time proportional to $(\text{Data size})/(\text{Partitions} \times \text{Processing rate per core})$; Transfer time = $(10 \text{ TB})/(1.25 \text{ GB/s}) \sim 2.3 \text{ hours}$

Mistakes to avoid: Trap: "RDDs are obsolete - only use DataFrames." DataFrames compile to RDDs internally; understanding RDDs is essential for debugging and performance tuning Trap: "Resilient means replicated." Resilience comes from lineage (recomputation), not storing multiple copies Trap: "Resilient = data is replicated in memory." Spark uses lineage, not in-memory replication, for RDD fault tolerance

Week 6: Spark Core II: Execution Model

Week roadmap: Spark Execution Engine: Architecture Overview → Lazy Evaluation: Why Spark Waits → Benefits of Lazy Evaluation: Optimisation and Pipelining → Narrow vs Wide Dependencies: The Performance Skeleton → Actions: Triggering Execution → Building the Directed Acyclic Graph (DAG) → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W06-T01 Spark Execution Engine: Architecture Overview

Tier A

Definition and why it matters: Writing Spark code is only half the story. The same logical pipeline can run in seconds or hours depending on how Spark plans, schedules, and executes work across a cluster. This module moves from "what is an RDD" to how Spark turns your code into a high-performance distributed machine

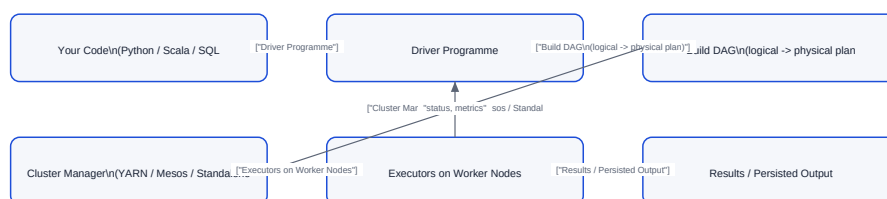
Core explanation: Three questions drive everything that follows: Objective: Lazy evaluation; What You Will Master: Why Spark delays execution and how that enables global optimisation Objective: Narrow vs wide dependencies; What You Will Master: Which operations stay local vs trigger expensive shuffles Objective: DAG and stage decomposition; What You Will Master: How the driver builds a plan and breaks it into parallel tasks Concept flow: Your Code\n(Python / Scala / SQL - Driver Programme - Build DAG\n(logical - physical plan - Cluster Manager\n(YARN / Mesos / Standalone - Executors on Worker Nodes - Results / Persisted Output Why does Spark wait before running transformations? What makes some operations slow (shuffles) while others are fast (pipelined)? How does code become stages and tasks that executors run in parallel? Core concepts: lazy evaluation, dependencies, and the DAG (Directed Acyclic Graph) with stages The execution engine is the central nervous system of a Spark application. It coordinates work across the entire cluster, transforming high-level Python, Scala, or SQL into a physical execution plan Hosts the SparkContext (or SparkSession in unified APIs) Orchestrates the entire job lifecycle Builds the DAG - the master plan of transformations and dependencies Converts actions into stages and tasks; monitors progress and handles failures Concept flow: SparkContext - DAG Scheduler - Task Scheduler - Executor 1\nTasks + Cache - Executor 2\nTasks + Cache - Executor N\nTasks + Cache Runs on a single machine (often called the "driver node") - not where bulk data processing happens Component: Driver; Role: Plan, schedule, coordinate, recover; Analogy: Orchestra conductor Allocates CPU, memory, and executors across the cluster Component: Cluster Manager; Role: Allocate cores and RAM; Analogy: Venue manager assigning stages Options: YARN (Hadoop ecosystem), Mesos, Kubernetes, or Spark's Standalone mode Component: Executor; Role: Run tasks, hold cached data; Analogy: Musicians playing their parts Does not understand Spark logic - it only provides hardware slots when the driver requests them JVM processes on individual cluster nodes Concept flow: Transformations\n(lazy - recorded - Logical Plan - Catalyst Optimizer - Physical Plan - Stages\n(shuffle boundaries - Tasks\n(one per partition - Action\n(trigger - Executors execute

Must remember:

- This module covers Spark's execution model: lazy evaluation, dependencies, DAG, stages, and tasks
- The driver hosts SparkContext, builds the DAG, and orchestrates the job
- The cluster manager (YARN/Mesos/Standalone/K8s) allocates CPU and RAM
- Executors run tasks, cache data, and report back to the driver
- The engine transforms high-level code into a physical execution plan executed in parallel
- Three core learning goals: lazy evaluation, narrow vs wide dependencies, DAG/stage decomposition
- Understanding architecture is prerequisite to diagnosing slow jobs and shuffle bottlenecks

Exam trap: Confusing driver with workers - the driver plans and coordinates; executors do the heavy data processing. Putting collect() on huge data overwhelms the driver, not executors Thinking the cluster manager runs Spark logic - it only allocates resources; the driver owns the DAG

Spark Execution Engine: Architecture Overview



Vector reconstruction of the source Mermaid visual

Concept map: Spark Execution Engine: Architecture Overview

BD-W06-T02 Lazy Evaluation: Why Spark Waits

Tier A

Definition and why it matters: In Spark, transformations are not executed immediately Concept flow: Source RDD - recorded - RDD2 (logical - recorded - RDD3 (logical - Waiting for action

Core explanation: Run a chain of Spark transformations - map, filter, flatMap on a large dataset - and the console returns almost instantly. No fan spin, no progress bar, no cluster activity. This is lazy evaluation in action: Spark has not touched your data yet Understanding laziness is essential because it explains both Spark's speed and its mental model. Transformations build a recipe; something else must order the kitchen to start cooking You tell the waiter: appetiser, main course, dessert. The kitchen does not start cooking after the first item. It waits for the complete order so it can coordinate timing, shared prep, and oven space. Spark waits for the full transformation chain (plus an action) before touching data If transformations are the recipe, actions are the order to serve Concept flow: map - filter - map - Logical plan stored\n(no execution - Action: count(- Optimizer - Physical execution\nacross cluster Actions are operations that either: Return a result to the driver (count, collect, take, reduce), or Write data to external storage (saveAsTextFile, save, etc.) The moment an action is invoked: The driver reads the accumulated logical plan (lineage) The Catalyst optimizer (for DataFrame/SQL) converts it into a physical execution plan The DAG Scheduler splits the plan into stages at shuffle boundaries Tasks are serialised and sent to executors for parallel execution Eager systems (execute each step immediately, often spilling to disk between steps) cannot see the full pipeline. Spark's delay enables whole-program optimisation

Must remember:

- Lazy evaluation: transformations are recorded, not executed immediately
- Spark builds a logical plan (lineage/recipe) while you chain transformations
- Actions (count, collect, save, etc.) trigger conversion to a physical plan and cluster execution
- Laziness enables global optimisation - predicate pushdown, filter reordering, join strategy selection
- Analogy: complete order before cooking, not dish-by-dish on partial orders
- Without an action, Spark holds an "unfulfilled promise" - no cluster work occurs
- Multiple actions without caching can recompute the full pipeline each time

Example or contrast: Code pattern: Written in this order;; raw = sc.textFile("hdfs://data/logs/"); parsed = raw.map(parseline); filtered = parsed.filter(lambda r: r["status"] == "ERROR"); result = filtered.count() Eager execution: Step 1 runs - materialise - Step 2 runs; Lazy execution: Entire chain visible before any step runs Eager execution: Local optimisations only; Lazy execution: Global reordering, fusion, pushdown

Exam trap: Expecting immediate feedback after transformations - nothing runs until an action; "instant" return means the plan was recorded, not executed Calling multiple actions on the same pipeline without caching - each action re-executes the entire lineage from scratch unless you cache() or persist()

BD-W06-T03 Benefits of Lazy Evaluation: Optimisation and Pipelining

Tier A

Definition and why it matters: Lazy evaluation is not procrastination - it is a deliberate architectural choice that unlocks four optimisation pillars impossible in naive eager systems. Each pillar exists because Spark waits until an action before executing

Core explanation: Concept flow: Lazy Evaluation\n(wait for action - 1. Query Optimisation - 2. Pipelining - 3. Fault Tolerance - 4. Catalyst Optimizer Without global view: Join full wide rows - then filter; With lazy global view: Push filter before join - smaller shuffle Without global view: Shuffle billions of columns; With lazy global view: Shuffle only needed columns/rows Because Spark sees the entire logical plan before execution, it can reorder and simplify operations globally Example scenario: You join two 1 TB tables but only need 3 columns from each, with a filter applied late in the script Concept flow: Read disk - map - Write disk - Read disk - filter - Write disk - Read once - map + filter\n(single memory pass Filter pushdown / predicate pushdown - apply filters as early as possible Projection pruning - read only required columns (especially in Parquet/ORC) Join reordering - pick smaller/build side for broadcast joins when beneficial This is smart execution: the written order of your code is not necessarily the executed order In traditional eager MapReduce-style systems, each transformation often writes intermediate results to disk, then the next step reads them back - read-map-write-read-filter-write Spark fuses multiple narrow transformations into a single in-memory pass within one stage: Checkpointing (traditional): Periodically snapshot full state to disk; Lineage (Spark): Store transformation graph metadata Checkpointing (traditional): Recovery = restore snapshot; Lineage (Spark): Recovery = re-run lost slice of graph Result: Eliminates unnecessary I/O. A chain of map → filter → map on the same partition runs as one fused iterator pipeline - data flows through without materialising intermediate RDDs to disk Checkpointing (traditional): High storage I/O; Lineage (Spark): Low metadata overhead; pay recompute cost on failure Rough speedup intuition: if eager execution takes $T(\text{eager}) = \text{sumi}(\text{readi} + \text{computei} + \text{writei})$, pipelining collapses many read/write pairs into one read and one write per stage boundary Lazy recording stores not just data pointers but the full recipe (lineage) to recreate any partition On worker failure: Identify lost partition(s) Walk lineage to nearest reliable source (HDFS block, checkpoint, cached parent) Recompute only the lost partition(s) Concept flow: Logical Plan - Rule-based rewrites - Physical plan candidates - Cost-based selection - Best physical plan Pillar: Query

optimisation; Primary savings: Less data shuffled and processed This pairs naturally with laziness: the plan always knows how to rebuild any partition without maintaining redundant copies of every intermediate Pillar: Pipelining; Primary savings: Less disk I/O per stage Catalyst is Spark's rule-based and cost-based optimizer engine (primary for Spark SQL / DataFrame API). It:

Must remember:

- Lazy evaluation enables four optimisation pillars: query optimisation, pipelining, fault tolerance, Catalyst
- Query optimisation: bird's-eye view → push filters early, prune columns, reorder joins
- Pipelining: fuse narrow transforms → single in-memory pass, no intermediate disk writes
- Fault tolerance: lineage stores the recipe → recompute lost partitions only
- Catalyst: automates logical → physical plan selection for DataFrame/SQL workloads
- Pipelining stops at shuffle boundaries (wide dependencies)
- Spark delays execution to optimise globally, not to defer work arbitrarily

Formula / decision rule: $T_{\text{(eager)}} = \sum_i (\text{read}_i + \text{compute}_i + \text{write}_i)$

Exam trap: Assuming pipelining applies across shuffles - pipelining works within a stage (narrow deps only); wide deps break the pipeline Thinking lineage eliminates all recovery cost - long lineage chains make recomputation expensive; `cache()` or `checkpoint()` may be needed

BD-W06-T04 Narrow vs Wide Dependencies: The Performance Skeleton

Tier B

Definition and why it matters: Prefer narrow chains where possible. When you must shuffle: Use `reduceByKey` instead of `groupByKey` + `map` (combines locally before shuffle)

Core explanation: When you chain transformations, each parent RDD connects to child RDDs through dependencies. The dependency type - narrow or wide - is the single most important predictor of job performance because it determines whether data stays local or must be shuffled across the network Concept flow: Transformation chain - Dependency type? - Local / pipelined (fast - Shuffle required) (slow - stage boundary Concept flow: P0 - P1 - P2 - C0 - C1 - C2 Operation: `map`; Dependency: Narrow (1:1); Shuffle?: No Operation: `filter`; Dependency: Narrow (1:1); Shuffle?: No In a narrow dependency, each partition of the parent RDD contributes to at most one partition of the child RDD Operation: `flatMap`; Dependency: Narrow (1:1); Shuffle?: No One-to-one: `map`, `filter`, `flatMap` - each parent partition maps to exactly one child partition Operation: `mapPartitions`; Dependency: Narrow (1:1); Shuffle?: No Many-to-one: `union` - multiple parent partitions may feed one child, but no parent partition splits across multiple children Operation: `union`; Dependency: Narrow (many:1); Shuffle?: No No shuffle - all required data is already in the parent partition Concept flow: P0 - P1 - C0 (all keys ending hash-0 - C1 Operations can be pipelined in a single memory sweep Tasks run in parallel, one per partition, with minimal network I/O Operation: `groupByKey`; Why wide?: All values for a key must colocate; Shuffle?: Yes Operation: `reduceByKey`; Why wide?: Aggregations require key colocation; Shuffle?: Yes Operation: `join`; Why wide?: Matching keys must be on same partition; Shuffle?: Yes Operation: `distinct`; Why wide?: Global dedup requires redistribution; Shuffle?: Yes Operation: `repartition(n)`; Why wide?: Explicit reshuffle; Shuffle?: Yes In a wide dependency, a single parent partition may be needed by multiple child partitions. Spark must redistribute data so related records (typically sharing a key) land on the same node

Must remember:

- Narrow dependency: each parent partition feeds at most one child partition - no shuffle, pipelined
- Wide dependency: parent data needed by multiple children - shuffle required, new stage boundary
- Examples narrow: `map`, `filter`, `flatMap`, `union`
- Examples wide: `groupByKey`, `reduceByKey`, `join`, `repartition`
- Shuffles are expensive: disk I/O + network + serialization
- Wide dependencies are the primary performance boundaries to watch in tuning
- Stage = maximal pipeline between shuffle boundaries

Example or contrast: Aspect: Parent - child partitions; Narrow: many children possible

Exam trap: Calling `groupByKey` when `reduceByKey` suffices - `groupByKey` shuffles all values; `reduceByKey` pre-aggregates locally first Assuming `union` causes a shuffle - `union` is narrow; no redistribution (but may produce uneven partition sizes)

BD-W06-T05 Actions: Triggering Execution

Tier B

Definition and why it matters: An action is any operation that either: Returns a value to the driver programme, or

Core explanation: Transformations build the plan lazily - a growing lineage graph stored in the driver's memory. But a plan that never runs produces no value. Actions are the spark plug: they tell Spark to stop recording and start

executing Without an action, the engine idles. With an action, the full pipeline - optimisation, stage construction, task dispatch - fires in one coordinated burst Concept flow: Transformations\n(lazy recipe - Engine idle\nplan in driver memory - Action called - DAG Scheduler - Stages - Tasks - Executors Action: count(); Returns to driver?: Yes (integer); Writes externally?: No; Typical use: Row/record count Action: collect(); Returns to driver?: Yes (full dataset); Writes externally?: No; Typical use: Dangerous at scale Action: take(n); Returns to driver?: Yes (n rows); Writes externally?: No; Typical use: Safe sampling Action: first(); Returns to driver?: Yes (one row); Writes externally?: No; Typical use: Peek at data Action: reduce(func); Returns to driver?: Yes (aggregated value); Writes externally?: No; Typical use: Global aggregate Action: saveAsTextFile(path); Returns to driver?: No; Writes externally?: Yes (HDFS/S3/local); Typical use: Production output Action: save(path); Returns to driver?: No; Writes externally?: Yes (Parquet, etc.); Typical use: Structured export Action: foreach(func); Returns to driver?: No; Writes externally?: Side effect per row; Typical use: External writes per record Concept flow: Action invoked - Driver reads lineage graph - Optimizer - physical plan - DAG Scheduler - Break into stages\n(at shuffle boundaries - Task Scheduler - Ship serialised tasks to executors - Driver monitors tasks - Action returns result / completes write

Must remember:

- Actions trigger execution; transformations alone keep the engine idle
- Actions return values to the driver (count, collect, take) or write externally (save)
- On action: driver → DAG Scheduler → stages → tasks → executors
- Transformations = path; actions = ignition
- Never collect() massive datasets - use take, save, or aggregates
- Production pipelines typically end with save, not collect
- Multiple actions without cache cause full lineage recomputation each time

Example or contrast: : Execution; Transformations: Lazy - recorded only; Actions: Eager - triggers job

Exam trap: Debugging with collect() on production-scale data - classic driver OOM cause Assuming transformations show data - you need an action (or show() on DataFrames) to see results

BD-W06-T06

Building the Directed Acyclic Graph (DAG)

Tier A

Definition and why it matters: While transformations accumulate lazily, Spark constructs an internal master plan - the Directed Acyclic Graph (DAG). This graph is the blueprint that turns your high-level code into parallel work on a cluster

Core explanation: Directed: data and dependencies flow in one direction (sources → transformations → actions) Concept flow: 1. Logical Plan\n(record lineage - 2. Physical Plan\n(Catalyst optimizes - 3. Stage Decomposition\n(shuffle boundaries - 4. Task Launch\n(one task per partition Concept flow: textFile - map - filter - join - target RDD Acyclic: no loops - you cannot have circular dependencies between RDDs. Each node is computed from its ancestors, never from its descendants When you write Python, Scala, or SQL, Spark parses it into an abstract representation of operations - not lines of text, but a tree/graph of what you want to compute At this stage: Spark knows what operations to apply (map, filter, join, etc.) It records lineage: the sequence of transformations from source to result Logical plan says: "Join tables A and B"; Physical plan decides: Broadcast small B vs shuffle both sides It has not decided how to execute (which join algorithm, which read strategy) Logical plan says: "Filter where status=ERROR"; Physical plan decides: Push filter to Parquet reader Logical plan says: "Select 3 columns"; Physical plan decides: Skip reading other columns The logical plan is the "intent layer" - pure semantics, no hardware binding yet The Catalyst optimizer converts the logical plan into a physical execution plan - a concrete blueprint specifying: Which algorithms to use (broadcast hash join vs sort-merge join) How data will be accessed (column pruning, partition filtering) Optimal operator ordering (filters before joins, etc.) Catalyst may generate multiple candidate physical plans, estimate costs, and pick the most efficient route Concept flow: Read - map - filter - Shuffle - Stage 1 - subgraph - reduceByKey - Stage 2 - map (format output Output: a specific, hardware-aware blueprint ready for scheduling The DAG Scheduler receives the physical plan and splits it into stages using shuffle boundaries Rule: Everything between two wide dependencies (shuffles) becomes one stage Within a stage: Narrow transformations are pipelined together Data stays in memory as long as possible before network movement Concept flow: User code - Logical Plan - Catalyst - Physical Plan - DAG Scheduler - Stages - Tasks - Executors - Driver: result / completion Why stages matter: tasks within a stage can fuse operations; crossing a stage always implies shuffle I/O Property: Directed; Implication: Clear data flow from sources to sinks Each stage is divided into tasks - the smallest unit of work in Spark Property: Acyclic; Implication: No infinite recomputation loops; lineage is a DAG

Must remember:

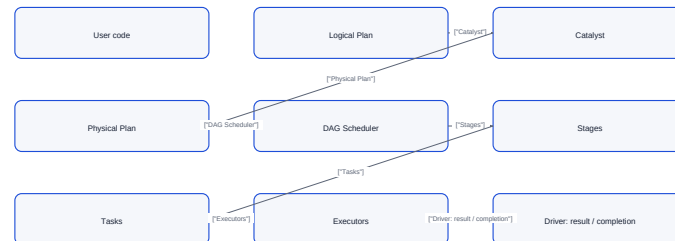
- The DAG is Spark's master execution plan: directed, acyclic, built during lazy transformation recording
- Step 1 - Logical plan: record operations and lineage (what to compute)
- Step 2 - Physical plan: Catalyst picks algorithms and access patterns (how to compute)
- Step 3 - Stages: DAG Scheduler splits at shuffle boundaries (wide dependencies)

- Step 4 - Tasks: one task per partition per stage, shipped to executors
- Stages enable pipelining of narrow ops; shuffles break stages
- Driver monitors tasks until the action completes

Formula / decision rule: Tasks in stage $\sim$ number of partitions processed in that stage

Exam trap: Confusing logical and physical plans - logical = what; physical = how Thinking the DAG executes incrementally per transformation - the DAG is built lazily but run only when an action fires

Building the Directed Acyclic Graph (DAG)



Vector reconstruction of the source Mermaid visual

Concept map: Building the Directed Acyclic Graph (DAG)

BD-W06-T07 Breaking the DAG into Stages and Tasks

Tier A

Definition and why it matters: The DAG is an abstract plan. To run on hardware, Spark must decompose it into stages (shuffle-separated pipeline chunks) and tasks (partition-sized units of work). This decomposition is what converts a lazy graph into thousands of parallel operations across executors

Core explanation: Every Spark command falls into one of three categories. How each interacts with the DAG determines performance

Property: Examples; Value: map, filter, flatMap, mapPartitions

Property: Lazy?; Value: Yes

- recorded, not executed

Property: DAG impact; Value: Pipelined into current stage

Property: Network; Value: None

- stays local to partition

Property: Efficiency; Value: High - fused in-memory

Property: Examples; Value: reduceByKey, groupByKey, join, repartition

Property: DAG impact; Value: Creates shuffle boundary - new stage

Property: Network; Value: Heavy - cross-node redistribution

Property: Efficiency; Value: Lower - pipeline stops, data moves

Concept flow: map - filter - flatMap - SHUFFLE \n (stage boundary) - Stage 1 - subgraph - reduceByKey

Property: Examples; Value: count, collect, save, take

Property: Lazy?; Value: No - immediate trigger

Property: DAG impact; Value: Forces optimisation + full execution

Property: Role; Value: Go signal for the entire DAG

The DAG Scheduler applies these rules:

- Start a new stage at each wide dependency (shuffle)
- Fuse all consecutive narrow transformations between shuffles into one stage
- Source reads (e.g., textFile) begin Stage 0 unless pushed down by optimizer
- The action runs after the final stage completes

Concept flow: Task 1 \n Partition 0 - Task 2 \n Partition 1 - Task 3 \n Partition 2 - Task 4 \n Partition 3 - Shuffle barrier \n (all tasks must finish - Stage 1 - Stage 2: 4 tasks - subgraph - Task 5

Counting stages (exam technique): Count wide dependencies in the pipeline. Typically: Number of stages = Number of shuffle boundaries + 1 (Adjust if multiple shuffles are consecutive or if certain operators combine.)

Within each stage: One task per partition (in most cases)

Each task executes the same transformation code on a different data slice

Tasks in the same stage can run fully in parallel (limited by available cores)

Tasks in later stages wait for all tasks in the prior stage to finish (barrier at shuffle)

Must remember:

- Operation type: map, filter; Lazy?: Yes; Creates shuffle?: No; New stage?: No; Parallelism: Per partition
- Operation type: count, save; Lazy?: No (action); Creates shuffle?: Depends on pipeline; New stage?: Triggers all; Parallelism: Orchestrates full job
- Spark decomposes the DAG into stages (at shuffles) and tasks (per partition)
- Eager systems spill to disk each step; Spark pipelines narrow ops within a stage
- Three categories: narrow transforms (pipelined), wide transforms (shuffle + new stage), actions (trigger)
- Wide deps stop the pipeline, redistribute data, and start a fresh stage
- Tasks are the smallest work units; parallel count $\sim$ partition count per stage
- Design jobs: filter early, minimise shuffles, tune partitions, read the Spark UI
- Actions are the go signal that optimises and executes the entire DAG

Formula / decision rule: Speedup $\sim$ $\frac{T_{\text{eager (disk-bound)}}}{T_{\text{Spark (pipelined)}}}$; Number of stages = Number of shuffle boundaries + 1

Example or contrast: Aspect: Intermediate results; Eager execution: Written to disk after each step; Spark lazy + staged execution: Pipelined in memory within a stage Aspect: I/O pattern; Eager execution: Read-write-read-write per operation; Spark lazy + staged execution: Read once per stage, write at shuffle boundary Aspect: Global optimisation; Eager execution: Not possible; Spark lazy + staged execution: Full plan visible before execution

Exam trap: Assuming all lazy ops are equal cost - narrow lazy ops are cheap to record; wide lazy ops imply future expensive shuffles Forgetting shuffle barrier synchronisation - one slow task in Stage 0 delays all of Stage 1 (straggler problem)

BD-W06-T08 Coding Walkthrough: Tracing a Job from Code to Tasks

Tier B

Definition and why it matters: This walkthrough traces a classic word count pipeline - the "Hello World" of distributed computing - from the first line of code through DAG construction, stage boundaries, shuffle, and final task execution. The goal is to make the abstract execution model concrete

Core explanation: Typical PySpark word count logic: Concept flow: `textFile` → `flatMap` → `split` → `map` → `reduceByKey` → `count` (Step: 1; Operation: `textFile`; Dependency type: Source read; Stage (when run): Stage 0 Step: 2; Operation: `flatMap`; Dependency type: Narrow; Stage (when run): Stage 0 (pipelined) Step: 4; Operation: `reduceByKey`; Dependency type: Wide (shuffle); Stage (when run): Stage 1 Step: 5; Operation: `count`; Dependency type: Action; Stage (when run): Triggers both stages Each line adds a node to the lineage graph. Nothing executes until `count()` As transformations are registered, Spark records the DAG without running it Concept flow: `count()` - Optimise plan - Stage 0 tasks - Shuffle → (group by word - Stage 1 tasks - Count returned to driver Key insight: `flatMap` and `map` are narrow - they chain in one pipeline. `reduceByKey` requires all values for the same key on one partition → shuffle boundary When `count()` is called: Driver hands lineage to DAG Scheduler Optimizer produces physical plan Graph splits into 2 stages separated by the shuffle before `reduceByKey` Tasks are created and dispatched to executors Code pattern: `lines = sc.textFile("hdfs://data/input.txt"); words = lines.flatMap(lambda line: line.split(" ")); pairs = words.map(lambda w: (w, 1)); counts = pairs.reduceByKey(lambda a, b: a + b); result = counts.count()` action - triggers execution Tasks in Stage 0 (one per input partition): Read assigned slice of `input.txt` from HDFS (data locality preferred) `flatMap`: split each line into words Concept flow: `hello-1` → `nword-1` - `hello-1` → `nspark-1` - Shuffle by key - Node C (hash slot - subgraph - `hello-1,1` → `nspark-1`)

Must remember:

- Word count: `textFile` → `flatMap` → `map` → `reduceByKey` → `count` (action)
- Stage 0: read + `flatMap` + `map` - all narrow, pipelined in one pass
- Shuffle before `reduceByKey` - wide dependency, redistributes by key
- Stage 1: `reduceByKey` + action completion logic
- DAG grows lazily; `count()` is the trigger that executes the full plan
- One simple script can already have 2 stages and many parallel tasks
- The shuffle is the performance bottleneck - not the `map` or `flatMap`

Exam trap: Drawing word count as one stage - `reduceByKey` always introduces a shuffle stage boundary Placing shuffle before `map` in word count - read → `flatMap` → `map` are all Stage 0; shuffle comes before reduce, not before `map`

BD-W06-T09 Module Summary: Spark Execution Model

Tier C

Definition and why it matters: This module moved from writing Spark code to understanding how Spark thinks - delaying work, optimising globally, splitting plans at shuffles, and firing parallel tasks across a cluster. Spark is not merely a data processor; it is an intelligent execution engine that compiles your logic into an optimised distributed program

Core explanation: Concept flow: Lazy Evaluation - Global Optimisation → (pushdown, pipelining, Catalyst - Dependencies → (narrow vs wide - DAG - Stages - Tasks - Parallel execution → non executors - Actions trigger

Must remember:

- Spark waits for actions before executing - laziness enables global optimisation
- Driver builds lineage/DAG; executors run tasks; cluster manager allocates resources
- Narrow deps stay local and pipelined; wide deps cause shuffles and new stages
- Execution path: logical plan → physical plan (Catalyst) → stages → tasks → executors
- Four pillars: query optimisation, pipelining, fault tolerance via lineage, Catalyst
- Actions (`count`, `save`, `collect`) trigger the entire engine; transformations do not
- Spark is an intelligent execution engine - every API call builds part of an optimised distributed craft
- Next: Spark SQL and deeper Catalyst mechanics

Example or contrast: Even minimal code exhibits the full model:

Exam trap: Treating Spark as eager - transformations return instantly because nothing ran yet Underestimating shuffle cost - wide deps dominate runtime in production jobs

Week 6 Exam Lens

Likely questions

- Define and explain Spark Execution Engine: Architecture Overview; include the mechanism and one limitation.
- Compare Spark Execution Engine: Architecture Overview with Lazy Evaluation: Why Spark Waits and state when each is preferred.
- Apply Benefits of Lazy Evaluation: Optimisation and Pipelining to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $T_{\text{eager}} = \sum_i (\text{read}_i + \text{compute}_i + \text{write}_i)$; Tasks in stage $\sim$ number of partitions processed in that stage; Speedup $\sim \frac{T_{\text{eager}}(\text{disk-bound})}{T_{\text{Spark}}(\text{pipelined})}$; Number of stages = Number of shuffle boundaries + 1

Mistakes to avoid: Confusing driver with workers - the driver plans and coordinates; executors do the heavy data processing. Putting `collect()` on huge data overwhelms the driver, not executors Expecting immediate feedback after transformations - nothing runs until an action; "instant" return means the plan was recorded, not executed Assuming pipelining applies across shuffles - pipelining works within a stage (narrow deps only); wide deps break the pipeline

Week 7: Resilience I: Lineage Mechanisms

Week roadmap: Why Resilience Matters in Big Data Systems → Challenges of Distributed Memory Recovery → Recomputing vs Replicating Data: Two Fault-Tolerance Philosophies → Anatomy of an RDD Lineage Graph → RDD Lineage Graph Walkthrough: Sales Analysis by Category → Recovery in Narrow Dependencies: Fast, Local, Isolated → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W07-T01 Why Resilience Matters in Big Data Systems

Tier B

Definition and why it matters: In distributed big data processing, the fundamental assumption must be that hardware will fail. A cluster with 1,000 nodes experiences failures daily - not occasionally. At scale, probability turns into certainty:

Core explanation: Hardware failures: disks crash, RAM fails, power supplies die, fans overheat Concept flow: 1000-node cluster - Daily failures? - Hardware / Network / Software - Lost partitions / Stranded tasks - Resilient system? - Auto-recovery, job continues - Full restart, hours wasted Property: Persistence; HDFS (disk): Survives node reboot; Spark (RAM): Lost instantly on failure Property: Speed; HDFS (disk): Milliseconds per read; Spark (RAM): Microseconds per read Property: Recovery strategy; HDFS (disk): Switch to replica; Spark (RAM): Must recompute or reload Network partitions: transient outages isolate nodes mid-computation, leaving jobs stranded Property: Cost of failure; HDFS (disk): Low (replica ready); Spark (RAM): High (data gone) Software crashes: JVM out-of-memory, executor timeouts, driver disconnects A system designed for "happy path only" cannot survive production workloads. Resilience - the ability to detect failure, recover automatically, and continue processing - is not an optional feature; it is the foundation of every serious big data platform A multi-hour Spark job processing terabytes of clickstream data cannot afford to restart from scratch every time one executor hiccups. Cloud ML pipelines training on petabyte-scale datasets would never complete. Resilience converts catastrophic failure into localized, recoverable interruption Traditional storage systems like HDFS keep data on persistent disk with replication. Spark's performance advantage comes from processing in RAM - but RAM is volatile: Concept flow: Fault Tolerance - Lineage Graph / DAG - Partition Recovery - Narrow: local, fast - Wide: shuffle, expensive

Must remember:

- At 1,000+ nodes, hardware failure is a daily statistical certainty, not an edge case
- Spark processes in volatile RAM, making recovery harder than disk-based systems like HDFS
- Resilience means self-healing: detect loss, recover automatically, continue the job
- Three pillars: fault tolerance, lineage graphs, partition recovery (narrow vs wide)
- Without resilience, multi-hour jobs restart from scratch on every node hiccup
- Network partitions can strand data mid-computation - systems must handle isolation
- The goal is localized recovery, not cluster-wide restart

Exam trap: Trap: "Adding more nodes eliminates failures." More nodes actually increases the probability of at least one failure occurring during a job Trap: "HDFS replication solves Spark's memory problem." HDFS replication protects disk blocks, not in-memory partitions during active computation

BD-W07-T02 Challenges of Distributed Memory Recovery

Tier A

Definition and why it matters: The defining challenge of in-memory distributed computing is volatility. When a node fails:

Core explanation: All RAM contents are wiped instantly - no graceful shutdown, no flush to disk Concept flow: Executor processing partition P - Node failure - RAM wiped instantly - Partition P lost - Has lineage recipe? - Recompute P from parents - Data permanently lost Factor: Data volume; Impact: Terabytes in RAM x 3 replicas = 3x memory consumption Factor: Network bandwidth; Impact: Every write multiplies cross-network traffic by replica count Factor: Job speed; Impact: Massive copying slows the primary computation Factor: Memory availability; Impact: Half or more of cluster RAM spent on safety copies, not computation Intermediate computation state vanishes Without a metadata-based "recipe" of how data was built, the system has no way to know where computation stopped or how to resume This is fundamentally different from disk-based systems where data persists across reboots. In Spark, every partition living only in executor memory is a ticking liability unless lineage provides a reconstruction path A natural question: why not just copy data like HDFS does? The answer reveals why Spark chose a different path For a cluster holding 10 TB in RAM across executors, triplicating that data requires 30 TB of memory capacity - most of which sits idle as insurance. This defeats the purpose of an in-memory engine whose speed comes from keeping memory lean for actual processing Approach: Replication; What's stored: Full data copies; Recovery method: Switch to replica; Memory footprint: Grows with data size Real-world analogy: Imagine a kitchen where every chef must maintain three identical copies of every dish in progress. The kitchen runs out of counter space, and chefs spend more time copying

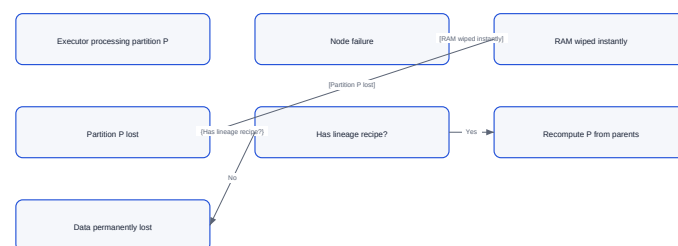
than cooking Approach: Lineage; What's stored: Metadata (kilobytes); Recovery method: Recompute from source; Memory footprint: Grows with logic complexity Even if replication were affordable, keeping memory replicas perfectly synchronized across a global cluster introduces: Heavy latency - every write must propagate to all replicas before proceeding Concept flow: Data Copy 1 - Data Copy 2 - Data Copy 3 - High RAM + Network cost - Source Data - Transform A - Transform B - Metadata only: recipe stored - On failure: recompute missing piece Complex locking - coordinating concurrent updates across nodes Split-brain scenarios - network partitions leave replicas inconsistent Coordination overhead - a central authority must track which replica is authoritative These consistency mechanisms work tolerably for small-scale databases but collapse under the throughput demands of big data pipelines processing millions of records per second

Must remember:

- RAM is volatile - node failure instantly wipes all in-memory state
- Replicating terabytes in RAM creates a bandwidth tax that slows jobs and wastes memory
- Memory replica synchronization introduces locking, latency, and consistency bottlenecks
- Spark stores metadata recipes (lineage) instead of data copies - kilobytes vs terabytes
- Recovery uses CPU recomputation instead of network replication - trading compute for bandwidth
- This design enables ~100x speedup over Hadoop while maintaining fault tolerance
- Lineage footprint grows with logic complexity, not data size

Exam trap: Trap: "Replication is always better for recovery speed." True for instant failover, but at terabyte scale in RAM, replication's bandwidth tax makes it impractical Trap: "Spark has no fault tolerance because it doesn't replicate." Spark is fault tolerant via recomputation, not replication

Challenges of Distributed Memory Recovery



Vector reconstruction of the source Mermaid visual

Concept map: Challenges of Distributed Memory Recovery

BD-W07-T03 Recomputing vs Replicating Data: Two Fault-Tolerance Philosophies Tier B

Definition and why it matters: Distributed systems recover from failure through one of two fundamental approaches:

Core explanation: Data replication - copy the data itself to multiple nodes Lineage-based recomputation - store the transformation recipe and rebuild lost data on demand Understanding why Spark chose recomputation requires contrasting these strategies across every dimension that matters in production Concept flow: Source Block - Node A - Node B - Node C - Source File - map - filter - reduce - Lineage DAG Strategy: Copy each data block to multiple nodes (typically 3 replicas) Dimension: Write overhead; Replication: 3x network + disk traffic per write; Lineage: Metadata logging only (kilobytes) Storage: Node A, Node B, Node C each hold an identical copy Dimension: Read overhead; Replication: Low (local replica); Lineage: Zero until failure occurs Recovery: If Node A fails, immediately read from Node B's replica Dimension: Steady-state cost; Replication: Continuous, every write; Lineage: Near-zero during normal execution Analogy: Photocopying every document and filing copies in three offices Dimension: Failure cost; Replication: Near-zero (switch replica); Lineage: Proportional to DAG depth Strategy: Store a DAG (Directed Acyclic Graph) describing how to recreate data from its source Storage: Only metadata - the "recipe" of transformations applied Recovery: Trace the lineage backward and re-execute transformations on source data Analogy: Storing a recipe card instead of pre-cooked meals in three kitchens Factor: Storage footprint; Replication: 3x data size (or more); Lineage: Metadata only - grows with logic, not data Factor: 1 GB data; Replication: Needs 3 GB across cluster; Lineage: Needs ~KB of lineage metadata Factor: 1 PB data; Replication: Needs 3 PB of storage; Lineage: Still ~KB-MB of metadata Replication is IO-heavy by design - every write multiplies traffic. In a high-speed RAM environment, this becomes a massive bottleneck. Spark's lineage logs only instructions, not data: moving terabytes vs recording a few kilobytes of metadata

Must remember:

- Replication copies data (HDFS: 3x storage, instant recovery); lineage stores recipes (metadata only, computed recovery)

- Replication multiplies IO on every write; lineage has near-zero steady-state overhead
- Replication recovers instantly; recomputation cost grows with DAG depth
- Lineage footprint scales with logic complexity, not data size - critical for petabyte workloads
- Spark's promise: re-execute lost partitions using CPU, keeping RAM lean for computation
- Immutability + determinism guarantee recomputation produces identical results
- Choosing compute-over-replicate is the foundation of Spark's speed advantage

Exam trap: Scenario: Node failure; Replication: Nearly instantaneous - switch to replica; Recomputation: Depends on DAG depth Scenario: Shallow DAG (1-2 transforms); Replication: Fast; Recomputation: Fast (nearly instant)

BD-W07-T04 Anatomy of an RDD Lineage Graph

Tier A

Definition and why it matters: When Spark loses a partition, it must know exactly what to recompute. The lineage graph is the logical map that answers this question. It is a Directed Acyclic Graph (DAG) that records every transformation applied to data - not the data itself, but the history of how data was built Concept flow: Source: HDFS/S3 file - map: parse CSV - filter: valid rows - join: by productid - reduceByKey: sum amounts - Action: save output

Core explanation: The time to recover a lost partition equals the sum of execution times for each operation in that partition's history, starting from the last checkpoint or source: Recovery Cost = $\sum_{i=1}^N T_i$ DAG Depth: 1-2 stages; Recovery Time: Milliseconds; Example: textFile - filter DAG Depth: 10-20 stages; Recovery Time: Seconds; Example: Multi-join ETL pipeline DAG Depth: 100+ stages; Recovery Time: Minutes-hours; Example: Iterative Page Rank, ALS N = number of transformation steps in the lineage chain for that partition T_i = execution time of the i -th operation Critical insight: recovery time is proportional to graph depth. A simple filter on a text file recovers nearly instantly; a 50-stage ML pipeline may take minutes to replay The lineage graph's reliability rests on three non-negotiable properties: Once an RDD is created, it is a fixed, read-only snapshot Transformations never modify existing RDDs - they produce new RDDs No risk of data changing midstream during recovery Each node in the DAG represents an immutable state Concept flow: Immutability: RDDs never change - Ancestry: Parent pointers - Determinism: Same input - same output - Recoverable Data Every RDD maintains a pointer to its parent RDD Pillar: Immutability; What it prevents: Mid-computation corruption; What it enables: Safe parallel reads This creates a chain of custody: RDDB knows it came from RDDA Pillar: Ancestry; What it prevents: Guessing parent data; What it enables: Targeted partition recovery On partition loss, the child knows exactly which parent to ask for input Pillar: Determinism; What it prevents: Inconsistent recomputation; What it enables: Trustworthy fault tolerance The DAG is built by linking these parent-child references recursively Rerunning the same transformation on the same input always produces the same output Without determinism, the lineage recipe is useless - recomputation might yield different data Non-deterministic operations (e.g., sample without fixed seed) break this guarantee Determinism is the "secret sauce" that makes recomputation trustworthy The Spark driver holds the complete lineage graph in memory: User defines transformations (lazy - no execution yet) Driver builds the DAG by recording parent-child RDD relationships An action triggers the DAG scheduler to break the graph into stages On failure, driver consults lineage to determine which tasks to reschedule

Must remember:

- Lineage graph = DAG tracking how data was built, not the data itself
- Recovery cost = sum of operation times from source/checkpoint to lost partition
- Recovery time grows linearly with DAG depth (N transformations)
- Three pillars: immutability (fixed snapshots), ancestry (parent pointers), determinism (reproducible results)
- Driver holds the lineage graph and uses it to reschedule failed tasks
- Shallow DAGs recover in milliseconds; deep iterative DAGs can take minutes
- Non-deterministic operations undermine the entire lineage-based recovery model

Formula / decision rule: Recovery Cost = $\sum_{i=1}^N T_i$; T_i

Exam trap: Trap: "Lineage stores actual data values." It stores transformation metadata only Trap: "Recovery is always fast because Spark is in-memory." Recovery speed depends on DAG depth - deep graphs are expensive

BD-W07-T05 RDD Lineage Graph Walkthrough: Sales Analysis by Category

Tier A

Definition and why it matters: Spark creates base RDDs representing file blocks in storage. These are the leaf nodes of the lineage graph:

Core explanation: RDDA1 $\leftarrow$ blocks from sales.csv RDDB1 $\leftarrow$ blocks from products.csv Each file block maps to one partition on one executor. No dependencies - data comes directly from HDFS/S3 Narrow dependency: each input row maps to exactly one output row (1:1) No data moves between machines - each executor processes its local partition independently RDDA2 sits on the same executor as RDDA1; RDDB2 on its own executor Wide dependency: records

with the same key must land on the same machine Triggers a shuffle - data physically moves across the network All productid = 42 records from both RDDs converge on one partition This is the most expensive step in the pipeline Reshapes tuples from (productid, (amount, category)) to (category, amount) Narrow dependency: happens locally on already-shuffled data Concept flow: sales.csv blocks: RDDA1 - map: parse: RDDA2 - products.csv blocks: RDDB1 - map: parse: RDDB2 - join by productid: RDDC: WIDE + SHUFFLE - map: reshape: RDDD: NARROW - reduceByKey: sum: RDDE: WIDE + SHUFFLE - saveAsTextFile: ACTION No cross-network movement - single-machine operation Step: 1; Operation: textFile; Dependency: None (leaf); Shuffle?: No; Recovery Cost: Read from source Step: 2; Operation: map (parse); Dependency: Narrow (1:1); Shuffle?: No; Recovery Cost: Recompute local partition Wide dependency: all records with the same category must be on the same partition Step: 3; Operation: join; Dependency: Wide (N:N); Shuffle?: Yes; Recovery Cost: Recompute all parents + reshuffle Code pattern: salesparsed = sales.map(parsesales) RDDA2; productsparsed = products.map(parseproducts) RDDB2 Requires another shuffle to group categories together Different categories may sit on different partitions across the cluster Step: 6; Operation: saveAsTextFile; Dependency: Action; Shuffle?: No; Recovery Cost: Triggers execution Triggers actual execution of the entire DAG Code pattern: joined = salesparsed.join(productsparsed) RDDC Results written to distributed storage Code pattern: remapped = joined.map(lambda x: (x[1][1], x[1][0])) (category, amount) The DAG scheduler recognizes that the sales branch and products branch are independent: Code pattern: totals = remapped.reduceByKey(lambda a, b: a + b) RDDA1 → RDDA2 and RDDB1 → RDDB2 run in parallel Both branches only converge at the join step This maximizes cluster utilization - two sets of tasks execute simultaneously before the shuffle barrier

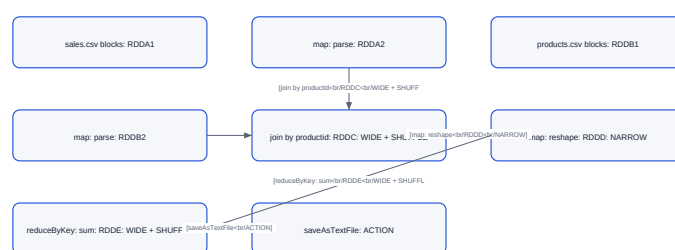
Must remember:

- Sales analysis pipeline: ingest → parse (narrow) → join (wide/shuffle) → reshape (narrow) → aggregate (wide/shuffle) → save
- Leaf nodes are base RDDs from file blocks; each transformation adds a DAG node
- Narrow dependencies (map, filter): 1:1 mapping, local recovery, no shuffle
- Wide dependencies (join, reduceByKey): N:N mapping, shuffle required, expensive recovery
- Independent branches (sales vs products) execute in parallel until a shuffle barrier
- Two shuffles in this pipeline: join and reduceByKey - both are recovery bottlenecks
- Actions trigger execution; transformations before them are lazy

Example or contrast: Consider a distributed sales analysis pipeline with two input datasets: File: sales.csv; Columns: trackingid, productid, amount; Role: Transaction records File: products.csv; Columns: productid, category; Role: Product metadata

Exam trap: Trap: "All map operations are narrow." True for 1:1 maps, but mapPartitions with side effects can behave differently Trap: "Join always requires shuffle." A broadcast join (small table) avoids shuffle by sending a copy to every node

RDD Lineage Graph Walkthrough: Sales Analysis by Category



Vector reconstruction of the source Mermaid visual

Concept map: RDD Lineage Graph Walkthrough: Sales Analysis by Category

BD-W07-T06

Recovery in Narrow Dependencies: Fast, Local, Isolated

Tier A

Definition and why it matters: A narrow dependency occurs when each parent partition maps to exactly one child partition - a 1:1 relationship. Common operations: Concept flow: Parent P1 - Child C1 - Parent P2 - Child C2 - Parent P3 - Child C3

Core explanation: Property: Partitions affected; Narrow Recovery: 1 child + 1 parent; Wide Recovery: 1 child + all parents Property: Other partitions; Narrow Recovery: Unaffected; Wide Recovery: May cascade Property: Cluster impact; Narrow Recovery: Minimal; Wide Recovery: Potentially global The failure is contained to a single parent-child pair. The rest of the cluster continues processing unaffected No data moves across the network Recovery is a local recomputation on one executor Spark simply reruns the transformation logic on the parent partition already available (or recomputed from its own parent) Recovery cost = T(transform) - a single operation's execution time Production

Spark pipelines deliberately chain narrow transformations: Each step in this chain has isolated, local recovery. If any partition is lost at any stage, recovery touches only one parent partition. Pipelining bonus: Spark fuses consecutive narrow transformations into a single stage, executing them in one pass per record without intermediate disk I/O. This means recovery of a fused stage replays all narrow ops together - still local, still fast. Recovery Cost(narrow) = T(single transform). This is the lower bound of recovery cost in Spark. Compare with wide dependencies where cost can cascade across the entire cluster. Narrow recovery is why Spark can "heal itself partition by partition without interrupting the rest of the cluster." Code pattern: Good: all narrow - fast recovery at every step; result = data.map(parse).filter(isvalid).map(extractfields).filter(nonnul)

Must remember:

- Narrow dependency = 1 parent partition → 1 child partition (map, filter)
- Lost partition C2 → driver traces lineage → finds sole parent P2 → recomputes locally
- Isolation: failure contained to one partition pair; rest of cluster unaffected
- No shuffle: recovery requires zero network data movement
- Chaining narrow transformations minimizes recovery cost at every pipeline stage
- Pipelining fuses consecutive narrow ops into one stage - still local recovery
- Recovery cost = single transformation time - the cheapest possible recovery in Spark

Formula / decision rule: $T_{\text{transform}}; \text{Recovery Cost}_{\text{narrow}} = T_{\text{single transform}}$

Example or contrast: Suppose the node holding child partition C2 fails. Here is exactly what happens: Concept flow: Node holding C2 fails - Driver checks lineage - C2 derived from P2 only - Isolate: ignore P1, P3, C1, C3 - Reschedule task on healthy executor - Re-execute transform on P2 - C2 restored. Driver detects failure - executor heartbeat timeout or task failure notification

Exam trap: Trap: "Narrow means no failure can occur." Narrow means recovery is cheap, not that failures are impossible. Trap: "flatMap is always narrow." flatMap is narrow for lineage purposes (each parent partition produces child partitions locally), but output size can vary wildly

BD-W07-T07 The Complexity of Wide Dependency Failures

Tier A

Definition and why it matters: Wide dependencies create many-to-many mappings between parent and child partitions. Operations that trigger them:

Core explanation: groupByKey / reduceByKey - aggregate by key. Concept flow: Parent P1 - Child C2 - Parent P2 - Parent P3 - Child C1 join - combine two datasets by key repartition / coalesce (with shuffle) - redistribute partitions. Concept flow: C2 LOST - P1 in memory? - P2 in memory? - P3 in memory? - Recompute P1 - Recompute P2 - Recompute P3 - Re-execute shuffle - Reconstruct C2 sortByKey - global sort requires shuffle. Child partition C2 contains data from multiple parent partitions (P1, P2, P3). This many-to-many relationship is the root of recovery complexity. When partition C2 is lost in a wide dependency: Driver cannot rerun a single local task - C2 needs data from P1, P2, and P3. Spark must fetch relevant data from every parent partition across the cluster. If parent partitions are no longer in memory, the failure cascades upward. Failure Point: During narrow transform; Recovery Scope: 1 partition; Cost: Low. Spark must recompute all parent partitions before it can even begin fixing C2. Failure Point: After shuffle completes; Recovery Scope: 1 partition + shuffle re-read; Cost: Medium. Failure Point: Mid-shuffle; Recovery Scope: Entire preceding stage + full reshuffle; Cost: High. Cascading recovery means a single lost partition can trigger recomputation of an entire upstream stage - potentially every partition in the preceding stage. Shuffles are the most expensive operation in Spark because they physically move data across the network to group keys together. When failure occurs mid-shuffle: Partially shuffled data may be inconsistent or incomplete. The system must re-execute the entire preceding stage to ensure data being shuffled is consistent and complete. Network bandwidth is consumed again for the full shuffle. All tasks in the failed stage are rescheduled, not just the lost partition. Concept flow: Source - Stage 1 - Stage 2: Shuffle - CHECKPOINT on disk - Stage 3 - Stage 4: Shuffle - Output. Because wide dependency failures are so expensive, Spark provides checkpointing to break the lineage chain: Save the state of an RDD after a shuffle to reliable disk storage (HDFS/S3). This creates a save point - a new leaf node in the DAG. On failure, Spark recovers from the checkpoint instead of cascading all the way to the source.

Must remember:

- Wide dependencies (join, groupByKey) create many-to-many parent-child mappings
- Losing one child partition requires data from all contributing parent partitions
- Cascading recovery: if parents are gone from memory, entire upstream stages recompute
- Mid-shuffle failures force full preceding stage replay - the most expensive recovery scenario
- Shuffles move data across the network; replaying them dominates recovery time
- Checkpointing after expensive shuffles creates save points that truncate cascading recovery
- Design principle: chain narrow ops freely; checkpoint strategically after wide/shuffle stages

Formula / decision rule: $T_{\text{one op}}; \text{sum } T_{\text{all parents}} + T_{\text{shuffle}}$

Example or contrast: Dimension: Parent partitions needed; Narrow: 1; Wide: Multiple (all contributors) Dimension: Network traffic during recovery; Narrow: None; Wide: Full shuffle replay Dimension: Failure scope; Narrow: Isolated partition; Wide: Entire upstream stage

Exam trap: Trap: "Wide dependency recovery is the same as narrow, just slower." Wide recovery is qualitatively different - it requires all parent partitions and may replay entire stages Trap: "Checkpointing eliminates shuffle cost." Checkpointing reduces recovery cost; the shuffle still happens during normal execution

BD-W07-T08 Spark Execution Mechanics: Stages, Pipelining, and Shuffle Skew

Tier A

Definition and why it matters: When a partition is lost in a real Spark cluster, the recovery flow is:

Core explanation: Executor failure detected (heartbeat timeout) Concept flow: Parse sales - Parse products - Join by productid - Reshape tuples - reduceByKey sum - Stage1 - Stage2 Driver consults the lineage graph for the failed task's partition Driver identifies the exact parent data needed Only the failed task set is requeued - the entire job does not restart A healthy executor recomputes the missing partition This surgical recovery is what makes Spark production-viable. A 6-hour job losing one partition does not become a 6-hour restart The DAG scheduler cuts the lineage graph at shuffle boundaries (wide dependencies). Each segment between shuffles becomes one stage: Within a single stage, consecutive narrow transformations (map, filter) are fused into one pass: Spark executes $\text{map} \rightarrow \text{filter} \rightarrow \text{map}$ as a single task per partition No intermediate materialization to disk One record flows through all transformations before the next record is processed This minimizes I/O and maximizes throughput The DAG scheduler recognizes independent branches: Concept flow: P0: mixed categories - P1: mixed categories - P2: mixed categories - P0: ALL Electronics (hotspot - P1: Books only - P2: Clothing only - Before - After Sales parsing and products parsing have no dependency on each other Both branches launch tasks in parallel to maximize cluster utilization They only synchronize at the join (shuffle barrier) A shuffle physically moves data across the network so all records with the same key end up on the same partition Each partition has a roughly equal mix of categories - work is balanced All "Electronics" records converge on one partition. If Electronics is the most popular category, Partition 0 becomes a hotspot - it holds far more data than the others Data skew occurs when one key has disproportionately more records than others. In the real world, keys are rarely uniformly distributed: Optimization: Stage fusion; What it does: Combines narrow ops in one stage; Benefit: Fewer task launches, less I/O E-commerce: a few bestselling SKUs dominate sales Optimization: Parallel branches; What it does: Runs independent DAG branches simultaneously; Benefit: Maximizes cluster utilization Social media: celebrity accounts have millions of followers Optimization: Task requeue on failure; What it does: Reschedules only failed tasks; Benefit: Surgical recovery, not full restart Finance: major trading days concentrate volume Optimization: Speculative execution; What it does: Launches backup task for stragglers; Benefit: Mitigates skew impact When skewed data lands on one partition after a shuffle:

Must remember:

- On failure, Spark requeues only the failed task set - not the entire job
- Stages are created at shuffle boundaries; narrow ops within a stage are pipelined (fused)
- Independent DAG branches execute in parallel until a shuffle barrier synchronizes them
- Shuffles move data across the network to co-locate keys - the most expensive operation
- Post-shuffle data skew creates hotspots where one partition holds most of a popular key
- Stragglers (slowest task) determine stage completion time - the job waits for the slowest partition
- Real-world key distributions are rarely uniform; skew is a production reality, not an edge case

Formula / decision rule: 100], [Books.; 200], [Clothing,

Exam trap: Trap: "Spark restarts the entire job on failure." Only the failed task set is requeued Trap: "Each transformation is a separate stage." Only shuffle boundaries create new stages; narrow ops are fused

BD-W07-T09 Spark Resilience: Module Summary and Key Takeaways

Tier C

Definition and why it matters: Transformations like map and filter form the backbone of high-performance pipelines:

Core explanation: 1:1 mapping: each parent partition $\rightarrow$ exactly one child partition

Must remember:

- Spark stores lineage (recipes), not data copies - saving RAM and network bandwidth
- Narrow deps (map, filter): 1:1 mapping, local recovery, no shuffle - the efficiency building blocks
- Wide deps (join, groupByKey): many-to-many, cascading recovery, shuffle replay - necessary but expensive
- Resilience rests on immutability (fixed snapshots) and determinism (reproducible results)
- Use persistence for reuse within a session; checkpointing for long iterative pipelines
- Design pipelines: chain narrow ops freely, checkpoint after expensive shuffles

- The recipe (lineage) never expires - lost data can always be recreated from source + transformations

Example or contrast: Spark's core architectural decision: prioritize metadata over data Aspect: What's stored; Replication (HDFS): Full data copies (3x); Lineage (Spark): Transformation recipe (DAG)

Exam trap: Trap: "Spark doesn't need fault tolerance because it's fast." Speed comes from in-memory processing, which requires fault tolerance Trap: "All recovery is equally fast in Spark." Narrow = milliseconds; wide = potentially entire stage replay

Week 7 Exam Lens

Likely questions

- Define and explain Challenges of Distributed Memory Recovery; include the mechanism and one limitation.
- Compare Challenges of Distributed Memory Recovery with Anatomy of an RDD Lineage Graph and state when each is preferred.
- Apply RDD Lineage Graph Walkthrough: Sales Analysis by Category to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $\text{Recovery Cost} = \sum_{i=1}^N T_i; T_i; T_{\text{(transform)}}; \text{Recovery Cost}_{\text{(narrow)}} = T_{\text{(single transform)}}; T_{\text{(one op)}}; \sum T_{\text{(all parents)}} + T_{\text{(shuffle)}}; 100], [\text{Books},; 200], [\text{Clothing},$

Mistakes to avoid: Trap: "Adding more nodes eliminates failures." More nodes actually increases the probability of at least one failure occurring during a job Trap: "Replication is always better for recovery speed." True for instant failover, but at terabyte scale in RAM, replication's bandwidth tax makes it impractical Scenario: Node failure; Replication: Nearly instantaneous - switch to replica; Recomputation: Depends on DAG depth

Week 8: Resilience II: Checkpointing

Week roadmap: When Lineage Becomes a Liability → Stack Overflow and Performance Degradation from Deep Lineage → Breaking the Family Tree: Strategic Truncation via Checkpointing → Caching vs Checkpointing: Two Critical Resilience Mechanisms → Writing Checkpoints to Reliable Storage: HDFS and S3 → Internal Mechanics: How Spark Truncates the DAG During Checkpointing → 2 more topics.

Coverage: 8 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W08-T01 When Lineage Becomes a Liability

Tier A

Definition and why it matters: RDD lineage is Spark's greatest resilience asset - until it isn't. In shallow DAGs (a few transformations), lineage enables near-instant recovery with minimal metadata overhead. But as graphs grow deeper through iterative algorithms, machine learning pipelines, or complex ETL chains, lineage transforms from an asset into a liability

Core explanation: The tipping point arrives when the cost of tracking and recovering through deep lineage exceeds the cost of simply writing intermediate results to disk DAG Depth: Stage 5; Recovery on Failure: Replay 4 transformations Recovery time grows linearly with DAG depth: Recovery Time proportional to N (number of stages) Concept flow: Stage 1 - Stage 2 - Stage 3 - Stage 4 - Stage 5 - Small metadata - Stage 1 - ... - Stage 99 Losing a partition at stage 100 means recomputing 99 previous steps. For iterative algorithms running hundreds of iterations, this is catastrophic Every transformation step must be tracked by the Spark driver: Each RDD object holds a reference to its parent, which holds a reference to its parent, and so on Metadata for the entire "family tree" lives in driver memory Signal: Recovery time; Shallow DAG (Asset): Milliseconds; Deep DAG (Liability): Minutes to hours As graph complexity grows, driver memory and CPU are consumed by dependency tracking rather than task scheduling Signal: Driver memory; Shallow DAG (Asset): Stable; Deep DAG (Liability): Growing, approaching OOM Eventually, the driver becomes the bottleneck - slowing down the entire cluster Signal: Iteration count; Shallow DAG (Asset): < 10; Deep DAG (Liability): 100+ (PageRank, ALS, ML) Signal: Failure impact; Shallow DAG (Asset): Localized; Deep DAG (Liability): Cascading across stages Lineage provides reliability - but a very long chain creates more points of failure: Signal: Metadata size; Shallow DAG (Asset): Kilobytes; Deep DAG (Liability): Megabytes Longer pipeline = higher statistical probability that recomputation will be triggered Each recomputation event is more expensive (deeper replay) The system designed for fault tolerance becomes more likely to need fault recovery, and each recovery is costlier This paradox: the longer the lineage, the more "reliable" the system claims to be, but the more painful each failure becomes Concept flow: Stages 1-20 - CHECKPOINT - Stages 21-40 - CHECKPOINT - Stages 41-60 Common triggers for deep lineage: Iterative graph algorithms (PageRank: 100+ iterations) Alternating Least Squares (ALS) for recommendation systems Gradient descent loops in distributed ML Multi-stage ETL with 50+ transformation steps The question this module answers: How do we break long lineage chains to ensure stability without losing Spark's functional model? The answer is checkpointing - strategically truncating the DAG by saving intermediate state to reliable storage and severing parent references. This converts a 100-stage recovery into a 1-stage recovery

Must remember:

- Lineage is an asset in shallow DAGs but becomes a liability in deep ones (100+ stages)
- Recovery time grows linearly with DAG depth - stage 100 failure replays 99 steps
- Driver stores entire family tree in memory - deep graphs cause driver bottleneck
- Reliability paradox: longer chains mean more failure points and costlier recoveries
- Iterative algorithms (PageRank, ALS, gradient descent) are primary deep-lineage triggers
- Solution: checkpointing truncates the DAG at strategic points
- The tipping point is when metadata/recovery cost exceeds the cost of disk writes

Formula / decision rule: Recovery Time proportional to N (number of stages)

Exam trap: Trap: "Lineage is always beneficial." Lineage is beneficial up to a depth threshold; beyond that, metadata and recovery costs dominate Trap: "Driver bottleneck only happens with large data." Driver bottleneck is about metadata volume (number of RDD objects), not data size

BD-W08-T02 Stack Overflow and Performance Degradation from Deep Lineage

Tier A

Definition and why it matters: When lineage chains grow beyond ~100 stages, Spark doesn't just slow down - the system can crash. Two failure modes emerge: hard crashes (stack overflow) and soft degradation (latency spikes). Both stem from the same root cause: recursive dependency tracking

Core explanation: When the Spark driver sends a task to an executor, it must serialize the RDD object. Because each RDD holds a reference to its parent, which holds a reference to its parent, serialization is inherently recursive: Algorithm: PageRank; Typical Iterations: 100-500; Lineage Depth: 100-500 stages Algorithm: ALS (recommendations); Typical

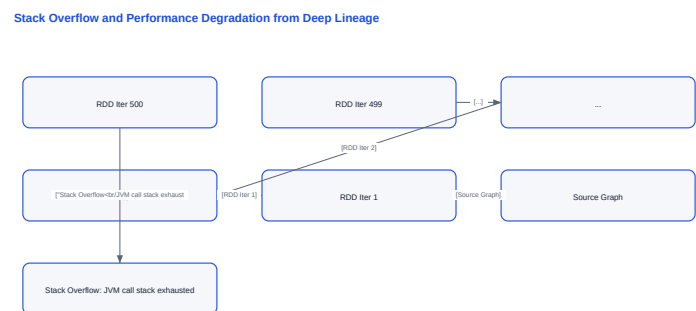
Iterations: 50-200; Lineage Depth: 50-200 stages Algorithm: Gradient descent; Typical Iterations: 100-1000; Lineage Depth: 100-1000 stages Algorithm: k-means (distributed); Typical Iterations: 20-100; Lineage Depth: 20-100 stages Concept flow: RDD Iter 500 - RDD Iter 499 - ... - RDD Iter 2 - RDD Iter 1 - Source Graph - Stack Overflow: JVM call stack exhausted The JVM traverses this chain recursively during serialization. At lineage depth of 100+, the JVM's call stack runs out of space before it can finish describing the RDD's history Lineage Depth: 5 stages; Normal Task Recovery: < 1 second; Deep Lineage Recovery: < 1 second Each iteration adds a new layer to the lineage. PageRank updating state 500 times creates a 500-deep recursive chain - a guaranteed stack overflow without checkpointing Lineage Depth: 500 stages; Normal Task Recovery: N/A (crashed); Deep Lineage Recovery: Hours or crash Even without a hard crash, deep lineage causes severe performance problems: Every task failure requires traversing the entire family tree to find the root Driver spends more time calculating dependencies than scheduling actual work Task scheduling latency increases with lineage depth Code pattern: RDD100 - parent: RDD99 - parent: RDD98 - ... - parent: RDD1 - parent: Source What should be a quick fix (recompute one partition) becomes a massive recomputation effort stalling the entire pipeline for minutes or hours Deep lineage doesn't just threaten the call stack - it consumes driver memory: Each RDD in the chain is a Java object with parent references, partition info, and dependency metadata 500 iterations = 500 RDD objects linked in a chain, all held in driver memory These objects cannot be garbage collected because the chain maintains references Driver memory fills with metadata, leaving less room for scheduling and coordination Driver Memory Pressure proportional to Lineage Depth x Metadata per RDD This is not about how much data you store - it's about how much metadata the system must manage A job with deep lineage that runs successfully for 50 iterations may: Iterations 1-50: runs fine, lineage grows silently Iteration 51: first task failure triggers deep recomputation - noticeable slowdown

Must remember:

- Deep lineage (100+ stages) causes stack overflow via recursive RDD serialization on the JVM
- PageRank, ALS, and gradient descent are primary victims - each iteration adds a lineage layer
- Even without crashing, deep lineage causes driver CPU overhead and recovery latency spikes
- Driver memory fills with RDD metadata objects that cannot be garbage collected
- Jobs appear healthy until sudden failure at high iteration counts - a "ticking time bomb"
- Solution: checkpointing breaks the recursive chain at strategic intervals
- The limit is metadata manageability, not data storage capacity

Formula / decision rule: Driver Memory Pressure proportional to Lineage Depth x Metadata per RDD

Exam trap: Trap: "Stack overflow is a data size problem." It's a lineage depth problem - metadata recursion, not data volume
Trap: "The job runs fine for 90 iterations, so checkpointing is unnecessary." Iteration 99 crashes without it - the failure is sudden, not gradual



Vector reconstruction of the source Mermaid statue

Concept map: Stack Overflow and Performance Degradation from Deep Lineage

BD-W08-T03 Breaking the Family Tree: Strategic Truncation via Checkpointing

Tier A

Definition and why it matters: A lineage family tree that grows too long is the problem. Checkpointing is the solution - strategic truncation of the DAG that converts an unstable deep graph into a series of manageable segments

Core explanation: If lineage is a recipe book growing one page per transformation, checkpointing is photographing the finished dish and throwing away the earlier recipe pages. You can always start again from the photograph Concept flow: Source - Transform 1 - Transform 2 - Transform 3 - Current RDD - Checkpoint on HDFS/S3: NEW LEAF NODE - Transform 4 - Transform 5 When you checkpoint an RDD, three things happen: Physical save: Spark writes the RDD's data partitions to reliable storage (HDFS or S3) Lineage truncation: Spark removes all references to parent RDDs - the checkpointed RDD becomes a new leaf node Stability anchor: The checkpointed RDD is a fresh starting point with no history Failure at: Stage 25; Without checkpoint: Replay 24 stages; With checkpoint: Replay 5 stages (from checkpoint

at 20) In the driver's eyes, the checkpointed RDD no longer has a history. The entire family tree behind it is pruned away Without checkpointing, losing a partition at stage 100 requires replaying 99 stages. With checkpointing at stage 20 and stage 40: Concept flow: Iteration 1-10 - Checkpoint - Iteration 11-20 - Checkpoint - Iteration 21-30 - Checkpoint - Complete 1000 iterations Recovery time resets from linear growth to a bounded constant determined by checkpoint frequency Recovery Time(with checkpoint) $\leq$ stages since last checkpoint Checkpointing is not optional for iterative workloads - it is mandatory: PageRank: state updated hundreds of times; without checkpointing, lineage depth = iteration count $\rightarrow$ stack overflow ALS: matrix factorization with 50-200 iterations; each adds a lineage layer Gradient descent: convergence loops create unbounded lineage growth Without checkpointing, these jobs will eventually fail - not might fail, will fail. Checkpointing is the only mechanism that keeps metadata small enough for the driver to manage

Must remember:

- Checkpointing = write RDD to reliable storage + sever all parent references
- Checkpointed RDD becomes a new leaf node - no history, no cascading recovery
- Unlike caching, checkpointing truncates lineage - the defining difference
- Recovery time becomes bounded by stages since last checkpoint, not total DAG depth
- Mandatory for iterative algorithms (PageRank, ALS) - without it, jobs WILL fail
- Checkpointing is a stability strategy, not a performance optimization
- Think of it as "photographing the dish" and discarding earlier recipe pages

Formula / decision rule: Recovery Time_(with checkpoint) $\leq$ stages since last checkpoint

Example or contrast: Property: Stores data copy; Caching (cache() / persist()): Yes (in memory / local disk); Checkpointing (checkpoint()): Yes (on HDFS / S3) Property: Preserves lineage; Caching (cache() / persist()): Yes - parents still referenced; Checkpointing (checkpoint()): No - parents severed Property: On cache loss; Caching (cache() / persist()): Recompute from lineage (full chain); Checkpointing (checkpoint()): Read from checkpoint file

Exam trap: Trap: "Checkpointing is the same as caching to disk." Caching preserves lineage; checkpointing severs it Trap: "Checkpoint once at the start is enough." Checkpointing must be periodic in iterative loops - one checkpoint doesn't help iteration 200

BD-W08-T04

Caching vs Checkpointing: Two Critical Resilience Mechanisms

Tier A

Definition and why it matters: Spark provides two mechanisms to avoid recomputing data - caching and checkpointing. Both store data copies, but they serve fundamentally different purposes and behave differently under failure. Choosing the wrong one can leave a pipeline unstable or wastefully slow

Core explanation: Code pattern: `data = sc.textFile("input.txt").map(parse).filter(valid); data.cache()` stores computed data in memory Concept flow: Source - Transform - Cached RDD - Recompute full chain - Source - Transform - Checkpoint on HDFS - No lineage - Read from HDFS Spark keeps a copy of the computed data and the full lineage graph If the cached data is lost (executor failure), Spark uses the preserved lineage to recompute from the original source Recovery may cascade through the entire DAG - caching does not limit recovery depth Spark writes data to reliable storage, then removes all parent references The checkpointed RDD becomes a leaf node - no history exists On failure, Spark reads from the checkpoint file - recovery depth = stages since checkpoint only Lives in executor memory or local disk - bound to specific nodes Scenario: Reusing an intermediate result multiple times in one job; Use Caching: Yes; Use Checkpointing: No Lazy: data is materialized only when an action first accesses the cached RDD Scenario: Iterative algorithm (PageRank, ALS, k-means); Use Caching: No; Use Checkpointing: Yes Session-bound: when the Spark session ends, all cached data is gone Scenario: DAG depth < 20 stages; Use Caching: Yes; Use Checkpointing: Optional Fast to access (microseconds from RAM) but vulnerable to node failure Writes to HDFS or S3 - distributed, replicated, durable Scenario: Need data to survive session restart; Use Caching: No; Use Checkpointing: Yes Eager: triggers an immediate background job to write data Scenario: Interactive exploration / ad-hoc queries; Use Caching: Yes; Use Checkpointing: No Session-independent: checkpoint files survive session termination Scenario: Driver approaching memory limits; Use Caching: No; Use Checkpointing: Yes A new Spark session can resume by reading checkpoint files Higher latency for writes (network + disk IO) but survives any single-node failure Code pattern: `sc.setCheckpointDir("/hdfs/checkpoints"); data = sc.textFile("input.txt").map(parse).filter(valid); data.checkpoint()` writes to HDFS, severs parents Rule of thumb: Cache for speed in reuse. Checkpoint for stability in long iterative pipelines

Must remember:

- Caching: preserves lineage, stores in memory/local disk, lazy, session-scoped - for speed and reuse
- Checkpointing: truncates lineage, stores on HDFS/S3, eager, cross-session - for stability
- On cache loss: recompute from full lineage chain; on checkpoint loss: read from distributed storage
- Caching does not solve deep lineage problems - only checkpointing severs the chain
- Decision: cache for short pipelines with reuse; checkpoint for iterative/long pipelines
- Checkpointing frees driver memory by making parent RDDs eligible for garbage collection

Example or contrast: Dimension: Lineage; Caching / Persistence: Preserved; Checkpointing: Truncated (destroyed)
 Dimension: Storage; Caching / Persistence: Memory or local executor disk; Checkpointing: HDFS or S3 (distributed)
 Dimension: Execution; Caching / Persistence: Lazy (on first action); Checkpointing: Eager (immediate background job)

Exam trap: Trap: "Caching and checkpointing are interchangeable." They solve different problems - caching for speed, checkpointing for stability
 Trap: "Caching protects against deep lineage." Caching preserves lineage; if cache is lost, full DAG replay still occurs

BD-W08-T05 Writing Checkpoints to Reliable Storage: HDFS and S3

Tier A

Definition and why it matters: Checkpointing only works if the saved data survives failures. Writing to a single executor's local disk is insufficient - if that node dies, the checkpoint is lost and lineage was already truncated. Checkpoints must go to reliable distributed storage that outlives any single node or session

Core explanation: The two dominant backends: HDFS (on-premise) and S3 (cloud-native)
 Property: Data locality; Benefit: Co-located with Spark workers - low network IO
 Property: Consistent access speed; Benefit: Predictable throughput on dedicated hardware
 Property: High throughput; Benefit: Optimized for large sequential writes
 Property: Integrated with Hadoop ecosystem; Benefit: Native support in on-premise clusters
 Property: Global durability; Benefit: 99.99999999% (11 nines) object durability
 Property: Decoupled from compute; Benefit: Cluster can be shut down; checkpoints persist
 Property: Cost efficiency; Benefit: Pay only for storage; spin down compute between jobs
 Property: Cloud-native integration; Benefit: Native on EMR, Databricks, GCP Dataproc
 On-premise Hadoop clusters where Spark workers and HDFS datanodes share the same rack
 Workloads where write speed for checkpoints is critical
 Environments with dedicated hardware and stable infrastructure
 Cloud-native Spark deployments (EMR, Databricks)
 Ephemeral clusters that spin up/down for cost savings
 Cross-session recovery - start a new cluster and resume from checkpoints
 Long-term persistence of intermediate computation state
 Before calling `RDD.checkpoint()`, you must configure the checkpoint directory: Without `setCheckpointDir()`, Spark has no destination for checkpoint files and the call will fail. This is a mandatory one-time configuration per `SparkContext`. Spark initiates a background job to materialize the RDD. Data partitions are written to the configured directory on HDFS/S3. Once write completes, parent references are severed. The checkpoint directory contains one folder per checkpointed RDD with partition files.
 Code pattern: Required setup - must precede any `checkpoint()` call; `sc.setCheckpointDir("hdfs:///checkpoints/my-app");` or; `sc.setCheckpointDir("s3://my-bucket/checkpoints/my-app");` Now checkpointing will work; `result = longpipeline.compute();`
`result.checkpoint()`

Must remember:

- Checkpoints must go to reliable distributed storage - not local executor disk
- HDFS: on-premise, data locality, fast writes, tied to cluster lifecycle
- S3: cloud-native, 11-nines durability, decoupled from compute, survives cluster shutdown
- `sc.setCheckpointDir(path)` is mandatory before any `checkpoint()` call
- HDFS for fast on-premise writes; S3 for cloud durability and ephemeral clusters
- Checkpoint directory should be unique per application/job
- The choice depends on infrastructure: on-premise → HDFS; cloud → S3

Example or contrast: Dimension: Deployment; HDFS: On-premise; S3: Cloud
 Dimension: Write latency; HDFS: Lower (locality); S3: Higher (network)
 Dimension: Durability; HDFS: Rack-level replication; S3: 11-nines object durability

Exam trap: Tied to the lifecycle of the compute cluster - if the Hadoop cluster is decommissioned, checkpoints are lost
 Not portable across different infrastructure setups

BD-W08-T06 Internal Mechanics: How Spark Truncates the DAG During Checkpointing

Tier A

Definition and why it matters: Understanding the internal checkpointing process reveals why it is so much more effective than caching for managing driver memory and job stability. The process involves four precise steps executed by the Spark driver and executors

Core explanation: When `RDD.checkpoint()` is called, Spark marks the RDD but does not immediately truncate lineage. The truncation only happens after data is safely persisted. Spark initiates an eager background job: Purpose: materialize the RDD and write all partitions to configured reliable storage (HDFS/S3). This is not lazy - unlike caching, checkpointing triggers immediate execution. Spark ensures data is physically safe on disk before proceeding to truncation.
 Concept flow: RDD B - RDD E - RDD C - RDD D - 1. Background job writes E to HDFS - 2. Create `ReliableCheckpoint RDD` - 3. Set `deps = empty list` - 4. Parents B, C, D eligible for GC - RDD E: Leaf node: `deps = empty`. All partitions are written as files in the checkpoint directory. Once the write completes, Spark performs the critical transformation: State: Normal RDD; `deps` contents: `[NarrowDependency(B), WideDependency(C)]`; Meaning: Knows its parents. Creates a

ReliableCheckpointRDD object to replace the original RDD State: After checkpoint; deps contents: [] (empty list); Meaning: No parents - leaf node Updates the internal deps field (dependency list) to an empty list The RDD no longer references any parent RDDs Parent RDDs (B, C, D) are no longer referenced by the checkpointed RDD (E) If no other RDD in the active DAG references them, they become eligible for garbage collection Driver memory previously occupied by thousands of metadata objects is reclaimed Metric: RDD objects in driver; Before Checkpoint (100 iterations): 100 linked objects; After Checkpoint: 1 leaf object + new transforms Metric: Serialization depth; Before Checkpoint (100 iterations): 100 (stack overflow risk); After Checkpoint: 0 for checkpointed RDD Every RDD internally maintains a deps field - a list of parent dependencies: Metric: GC-eligible objects; Before Checkpoint (100 iterations): 0 (all referenced); After Checkpoint: 99 (parents dereferenced) Metric: Driver memory for metadata; Before Checkpoint (100 iterations): Growing linearly; After Checkpoint: Reset to baseline This is not a cosmetic change. The empty deps list is what makes the checkpointed RDD a true leaf node. Spark's DAG scheduler, lineage traversal, and serialization all use this field. An empty list means: Serialization depth = 0 (no recursive parent chain)

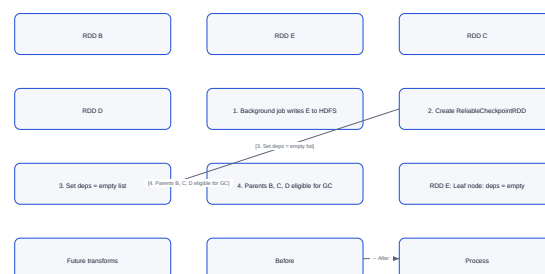
Must remember:

- Checkpointing: write data to HDFS/S3 → create ReliableCheckpointRDD → set deps = [] → parents eligible for GC
- The deps field going empty is what makes the RDD a true leaf node
- Background job is eager - data must be safely on disk before truncation
- Parent RDD metadata is physically removed from driver memory, not just hidden
- This solves driver bottleneck (metadata growth) and stack overflow (serialization depth)
- Caching does none of this - it preserves the full parent chain and deps field
- GC reclaims thousands of metadata objects, resetting driver memory to baseline

Example or contrast: Internal Action: Materialize data; cache(): On first action (lazy); checkpoint(): Immediately (eager) Internal Action: Write destination; cache(): Executor memory/disk; checkpoint(): HDFS/S3 Internal Action: Update deps field; cache(): No change; checkpoint(): Set to empty

Exam trap: Trap: "Checkpointing hides the lineage." Lineage is not hidden - it is physically removed and parents are garbage collected Trap: "Checkpoint happens instantly." The background write job must complete before truncation occurs - checkpointing is not atomic

Internal Mechanics: How Spark Truncates the DAG During Checkpointing



Vector reconstruction of the source MemSQL visual

Concept map: Internal Mechanics: How Spark Truncates the DAG During Checkpointing

BD-W08-T07

Recomputation Cost vs Storage IO Latency: The Checkpointing Trade-off

Tier A

Definition and why it matters: Checkpointing is a lifesaver for long-running Spark jobs - but every truncation and disk write carries a price. Understanding the trade-off between recomputation cost (no checkpoint) and storage IO latency (with checkpoint) is essential for deciding when and how often to checkpoint

Core explanation: Two curves describe failure recovery behavior: Concept flow: N At N = 100: must replay 100 stages of logic At N = 500: recovery may take longer than the job's total runtime Formula: Recovery Time = $\sum_{i=1}^N T_i$ Starts slightly higher than red (fixed IO cost per checkpoint) As lineage depth increases, recovery time levels off At N = 500 with checkpoints every 20 stages: recovery requires at most 20 stages of replay Formula: Recovery Time $\leq \sum_{i=1}^K T_i$ where K = stages since last checkpoint Every checkpoint incurs: Disk IO: writing all partitions to HDFS/S3 Network bandwidth: data movement from executors to storage Compute time: materializing the RDD before writing Driver coordination: scheduling the background checkpoint job Factor: Iteration count; Favor more frequent checkpointing: High (100+); Favor less frequent checkpointing: Low (< 20) This is a fixed upfront cost paid during normal execution, whether or not a failure occurs Factor: Failure probability; Favor more frequent checkpointing: High (unstable cluster); Favor less frequent checkpointing: Low (stable cluster) Factor: Stage compute cost; Favor more frequent checkpointing: Expensive stages; Favor less frequent checkpointing: Cheap stages Factor:

IO bandwidth; Favor more frequent checkpointing: Abundant; Favor less frequent checkpointing: Constrained Factor: Driver memory pressure; Favor more frequent checkpointing: High (deep lineage); Favor less frequent checkpointing: Low (shallow lineage) The optimal checkpoint frequency balances write cost against risk cost: Checkpoint every 10-20 iterations Job Type: PageRank (100+ iter); Recommended Checkpoint Interval: Every 10 iterations; Rationale: Deep lineage, expensive graph ops Frequent enough to keep recovery time predictable and driver memory safe Job Type: ALS (50-200 iter); Recommended Checkpoint Interval: Every 10-20 iterations; Rationale: Matrix ops are costly to replay Not so frequent that IO overhead slows the job to a crawl Job Type: ETL pipeline (no loop); Recommended Checkpoint Interval: After major shuffles only; Rationale: Shallow lineage, shuffle is the bottleneck At 10-20 iteration intervals, recovery replays at most 10-20 stages Job Type: Interactive query; Recommended Checkpoint Interval: No checkpointing needed; Rationale: Shallow DAG, cache instead Checkpointing is insurance against failure: Premium = IO cost of writing checkpoints during normal execution Coverage = bounded recovery time if failure occurs Deductible = stages since last checkpoint that must be replayed The more expensive the recomputation (deeper lineage, costlier stages), the more you should be willing to pay the IO premium

Must remember:

- Without checkpointing: recovery time grows linearly with lineage depth
- With checkpointing: recovery time is bounded by stages since last checkpoint
- Checkpointing adds a fixed IO write cost during normal execution (the "premium")
- Not checkpointing risks catastrophic recomputation cost on failure (the "risk")
- Rule of thumb: checkpoint every 10-20 iterations for iterative algorithms
- Balance write cost (IO overhead) against risk cost (recovery time on failure)
- In large-scale systems, stability is often more valuable than raw speed

Formula / decision rule: $N = 100$; $N = 500$

Exam trap: If a failure occurs without checkpointing: Recomputation time: replaying all stages from source or last checkpoint

BD-W08-T08

Advanced Resilience Strategies: Module Summary

Tier C

Definition and why it matters: This module moved beyond Spark's default lineage-based fault tolerance to explore the advanced resilience strategies required for complex, real-world big data processing - particularly iterative algorithms and long-running pipelines

Core explanation: Depth: < 20 stages; Behavior: Near-instant recovery, minimal metadata; Risk: Low

Must remember:

- Deep lineage (100+ stages) causes stack overflow, driver bottleneck, and expensive recovery
- Checkpointing truncates the DAG: write to HDFS/S3, sever parents, create leaf node
- Internal mechanics: eager write $\rightarrow$ ReliableCheckpointRDD $\rightarrow$ deps = [] $\rightarrow$ GC parents
- Caching = speed/reuse (preserves lineage); Checkpointing = stability (truncates lineage)
- Recovery curve: raw lineage grows linearly; checkpointed recovery is bounded
- Checkpoint every 10-20 iterations for iterative algorithms
- Checkpointing is a stability strategy, not just a recovery mechanism
- The difference between a job that crashes at iteration 99 and one that completes iteration 1000

Formula / decision rule: proportional to N ; $N = 500$

Example or contrast: : Purpose; Caching: Speed and reuse; Checkpointing: Stability and lineage management

Exam trap: Raw lineage: recovery time proportional to N (linear growth) With checkpoints: recovery time bounded by checkpoint interval

Week 8 Exam Lens

Likely questions

- Define and explain When Lineage Becomes a Liability; include the mechanism and one limitation.
- Compare When Lineage Becomes a Liability with Stack Overflow and Performance Degradation from Deep Lineage and state when each is preferred.
- Apply Breaking the Family Tree: Strategic Truncation via Checkpointing to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Recovery Time proportional to N (number of stages); Driver Memory Pressure proportional to Lineage Depth x Metadata per RDD; Recovery Time__(with checkpoint) $\leq$ stages since last checkpoint; N = 100; N = 500; proportional to N

Mistakes to avoid: Trap: "Lineage is always beneficial." Lineage is beneficial up to a depth threshold; beyond that, metadata and recovery costs dominate Trap: "Stack overflow is a data size problem." It's a lineage depth problem - metadata recursion, not data volume Trap: "Checkpointing is the same as caching to disk." Caching preserves lineage; checkpointing severs it

Week 9: Data Partitioning I: Strategies and Structures

Week roadmap: Why Partitioning Is the Secret to Horizontal Scale → Partitioning vs Replication: Complementary but Distinct → How Partitions Enable Parallel Execution → Hash Partitioning Mechanics: The Modulo Operator and Uniformity → Use Cases: When to Choose Hash Partitioning → Range Partitioning Mechanics: Key Ranges and Sorted Data → 4 more topics.

Coverage: 10 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W09-T01 Why Partitioning Is the Secret to Horizontal Scale

Tier A

Definition and why it matters: The naive scaling intuition: "My database is slow, so I'll buy more servers." In distributed systems like Spark and Hadoop, simply adding machines does not automatically make jobs faster. Without a strategy for how data is physically arranged across nodes, you get a cluster where one node does 90% of the work while the rest sit idle

Core explanation: Partitioning is the architectural mechanism that makes horizontal scaling actually work. It answers the question: where does each piece of data live, and how is work divided across the cluster? Even the most powerful server has finite disk space and memory. A 100 TB dataset cannot fit on one machine. Partitioning breaks massive datasets into smaller, manageable chunks distributed across multiple nodes: A 100 TB file → 1,000 partitions of ~100 GB each → spread across 100 nodes. No single machine's capacity is the bottleneck. Storage and processing capacity grow linearly with cluster size. Overcome the single-machine storage and compute limit. Distribute data chunks across the cluster. Process datasets larger than any one computer could hold. Scale out by adding nodes, each holding unique data partitions. Concept flow: Scalability: Break data into chunks: across nodes - Parallelism: Each partition = unit of work: N partitions - N cores - Data Locality: Process where data lives: Minimize shuffles - Horizontal Scale Works. Each partition is a unit of work in Spark. Pillar: Scalability; Problem it solves: Dataset exceeds single machine; Without it: Cannot store or process 100 partitions + 100 CPU cores = theoretical 100x throughput vs single core. Pillar: Parallelism; Problem it solves: Single-threaded bottleneck; Without it: One core does all work. Multiple CPUs process different partitions simultaneously. Pillar: Data locality; Problem it solves: Network shuffle overhead; Without it: Data constantly crossing network. Partitioning is the bridge between a static dataset and parallel execution. The network is the biggest bottleneck in distributed systems. Strategy: Hash partitioning; How it works: $P = \text{hash}(\text{key}) \bmod N$; Best for: Uniform distribution, point lookups, joins. Moving data between servers (shuffle) is expensive in time and resources. Strategy: Range partitioning; How it works: Continuous ranges on sortable key; Best for: Range queries, chronological data. Effective partitioning keeps processing logic close to the physical data. Strategy: Custom partitioning; How it works: Domain-specific logic; Best for: Skewed data, geographic, hierarchical IDs. Minimizing network shuffles maximizes cluster throughput. This module covers the major strategies, each balancing the three pillars differently. Every strategy is designed to balance scalability, parallelism, and data locality - but each excels in different scenarios and fails in others.

Must remember:

- Adding machines without partitioning strategy leaves most nodes idle
- Partitioning breaks datasets into chunks distributed across the cluster
- Three pillars: scalability (exceed single-machine limits), parallelism (N partitions → N cores), data locality (minimize shuffles)
- Each partition is a unit of work in Spark - partition count sets the parallelism ceiling
- Strategies: hash (scatter), range (organize), custom (domain-specific)
- Poor partitioning creates hotspots where one node does 90% of the work
- Partitioning is the architectural prerequisite for horizontal scale-out

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N$

Exam trap: Trap: "More nodes = faster jobs." Without partitioning, adding nodes does nothing - data may still sit on one machine. Trap: "Partitioning and replication are the same thing." Partitioning splits data for speed; replication copies data for safety (covered next)

BD-W09-T02 Partitioning vs Replication: Complementary but Distinct

Tier A

Definition and why it matters: In distributed big data systems, partitioning and replication are constantly mentioned together - but they serve completely different purposes. Confusing them leads to architectural mistakes. A healthy distributed system requires both, but for different reasons

Core explanation: Replication creates identical copies of the same data on different nodes. Property: What it does; Detail: Copies the same data block to Node A, B, and C. Property: Why; Detail: Hardware failure is a statistical certainty at scale. Property: On failure; Detail: System switches to a replica - user notices nothing. Property: Analogy;

Detail: Keeping three photocopies of every important document Property: Cost; Detail: Storage multiplied by replica count (typically 3x) Property: What it does; Detail: Customer A-M on Node 1, N-Z on Node 2 Property: Why; Detail: Enable parallel processing across the cluster Property: On scale-out; Detail: Add nodes to hold more unique data partitions Property: Analogy; Detail: Dividing a library catalog across multiple buildings by letter range Property: Cost; Detail: Requires careful strategy to avoid skew If Node A crashes and was the only machine holding a specific data block, that data is gone. With replicas on Node B and Node C, the system simply redirects reads to a surviving copy Replication is about safety Partitioning splits a dataset into different subsets across nodes - not copies, but unique pieces Each node holds a different portion of the data. Together, all partitions form the complete dataset. No node has the full dataset - they each have their slice Partitioning is about speed and capacity Layer: Logical; Strategy: Partitioning (100 chunks); Benefit: Parallel processing, scale-out Layer: Physical; Strategy: Replication (3x per chunk); Benefit: Fault tolerance, durability In real-world Spark/Hadoop environments, both strategies operate in tandem: Partition the dataset into N unique chunks for parallel processing Replicate each partition 3x across different nodes for fault tolerance This gives you the performance of partitioning with the safety of replication Code pattern: Dataset (1 TB) - 100 partitions (10 GB each) - each partition replicated 3x; Total storage: $100 \times 10 \text{ GB} \times 3 = 3 \text{ TB}$; Parallelism: 100 partitions - up to 100 parallel tasks; Fault tolerance: any single node failure - replica available

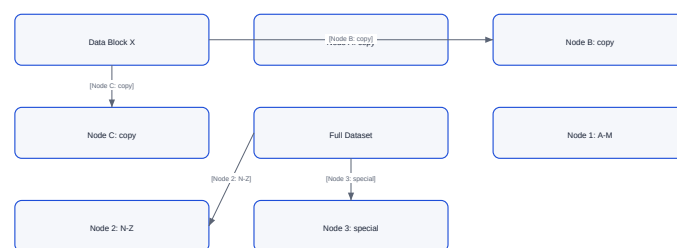
Must remember:

- Replication = identical copies for fault tolerance (insurance policy)
- Partitioning = unique subsets for parallel processing (engine)
- Replication protects against failure; partitioning enables speed and scale
- Production systems use both: partition for parallelism, replicate for safety
- A single partition is often replicated 3x - performance + durability
- Replication multiplies storage (3x); partitioning divides work (N-way parallelism)
- Confusing the two leads to architectural mistakes in distributed system design

Example or contrast: Dimension: Goal; Replication: Fault tolerance, durability; Partitioning: Performance, scalability Dimension: Data on each node; Replication: Identical copy; Partitioning: Unique subset Dimension: Adds capacity?; Replication: No (same data, more copies); Partitioning: Yes (more unique data per node)

Exam trap: Trap: "Replication makes the system faster." Replication adds safety, not speed - it multiplies storage without adding parallelism Trap: "Partitioning protects against data loss." Partitioning splits data; if the only node holding a partition fails, that data is lost unless replicated

Partitioning vs Replication: Complementary but Distinct



Vector reconstruction of the source Mermaid visual

Concept map: Partitioning vs Replication: Complementary but Distinct

BD-W09-T03

How Partitions Enable Parallel Execution

Tier A

Definition and why it matters: A straggler is a task that takes significantly longer than all others to complete. In distributed systems, a job finishes only when every task finishes - the slowest task determines total runtime Scenario: 99 tasks finish in 1 minute, 1 straggler takes 10 minutes; Result: Entire job takes 10 minutes

Core explanation: In a distributed engine like Spark, partitioning defines the unit of parallelism. You can deploy the most powerful cluster available, but if all data sits in a single partition, only one CPU core can work on it at a time. The number of partitions effectively sets the speed limit for how much work the cluster can perform simultaneously Partitioning is not merely about where data physically resides - it is the blueprint for how work is divided across the cluster Rule: One partition - one task; Meaning: Each partition maps to exactly one task Rule: Task - worker node; Meaning: Each task is dispatched to a worker for independent execution Rule: Worker thread executes; Meaning: A thread on the worker runs the task without coordinating with other partitions Concept flow: Dataset: 4 partitions - Task 1 - Task 2 - Task 3 - Task 4 - Worker Node A - Worker Node B - Worker Node C - Worker Node D When a job is submitted, the engine decomposes it into tasks. The mapping rule is strict: 400 partitions → up to 400 concurrent tasks With 400 cores available, theoretical throughput scales linearly with partition count Partitioning is the bridge between

a static dataset and parallel cluster utilization. Simply having many partitions is not enough. Optimal partitioning ensures cluster resources are utilized evenly. Understanding that partitions drive parallelism sets up the next question: how should data be assigned to partitions? Hash partitioning - scatter keys uniformly using $P = \text{hash}(\text{key}) \bmod N$. Range partitioning - group contiguous key ranges together. Custom partitioners - domain-specific logic when defaults fail. Each strategy affects both parallelism (how many tasks run) and balance (whether stragglers appear).

Must remember:

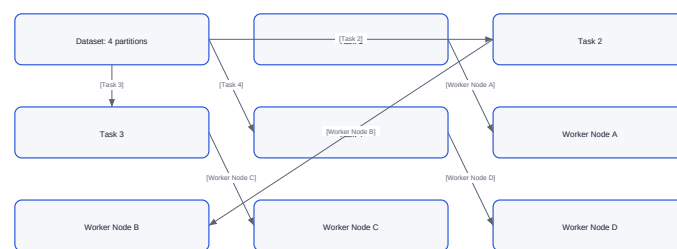
- In Spark, one partition = one task - the fundamental unit of parallelism
- Partition count sets the upper bound on simultaneous work across the cluster
- 4 partitions + 4 cores = 4 tasks running in true parallel
- Stragglers (slowest task) determine total job runtime, not average task speed
- Skewed partitions (90% in one partition) waste cluster resources and create bottlenecks
- Optimal partitioning balances partition count with even data distribution
- Partitions define where work is divided; partitioning strategy defines how data is assigned
- Next step: hash partitioning uses the modulo operator for uniform distribution

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N$

Example or contrast: Dataset has 4 partitions → engine creates 4 tasks. Cluster has 4 available CPU cores → all 4 tasks run at the same time. This is parallel execution: static data becomes dynamic, concurrent work.

Exam trap: Trap: "More machines automatically means faster jobs." Without enough partitions, added nodes sit idle - parallelism is bounded by partition count, not node count. Trap: "More partitions always means better performance." Excessive partitions add scheduling overhead; balanced partitions matter more than raw count.

How Partitions Enable Parallel Execution



Vector reconstruction of the source Mermaid visual

Concept map: How Partitions Enable Parallel Execution

BD-W09-T04 Hash Partitioning Mechanics: The Modulo Operator and Uniformity Tier A

Definition and why it matters: Manually deciding where every row in a petabyte-scale dataset should live is impossible. Distributed systems need an automated, mathematical way to scatter data so every node receives a fair share. Hash partitioning is the industry standard for achieving a pseudorandom but predictable spread - deterministic mapping that balances load without human intervention.

Core explanation: The partition assignment formula is: Component: Key; Role: Business attribute that identifies a record. Component: Hash function; Role: Maps arbitrary key types to a large integer space. Component: Modulo (mod N); Role: Scatters the integer across partition indices 0 to N-1. Component: Result P; Role: Deterministic partition address for that key $P = \text{hash}(\text{key}) \bmod N$. Key - the attribute chosen for partitioning (e.g., userid, orderid). hash(key) - transforms any input (string, number, date) into a large consistent integer. N - total number of partitions desired. P - resulting partition index (target address). Property: Deterministic; Benefit: Same key - same partition every time. Property: Automated; Benefit: No manual assignment needed. Property: Scattered; Benefit: Keys spread across all partitions. Property: Predictable; Benefit: Enables direct lookup without full scan. The modulo operator naturally scatters values across the range 0 to N-1. When keys are diverse and the hash function distributes well, roughly equal amounts of data land on every partition. Analogy - dealing a deck of cards. Condition: 50% of records share the same key (e.g., null, "UNKNOWN"); What Happens: Hash produces identical integer for all - modulo sends all to one partition. The hash function shuffles the keys into a random-looking order. Condition: Low-cardinality key (e.g., country with 3 values); What Happens: Only 3 distinct hash outputs - only 3 partitions used, rest empty. The modulo operator deals them one by one to N partition "hands". Condition: Popular category dominates; What Happens: Hot spot / straggler - one node overwhelmed. With a fair shuffle and many cards, each hand gets approximately the same count. Concept flow: key A - Partition 3 - key B - Partition 7 - key C - Partition 1 - key D - Partition 9 - null. Property: Deterministic; Requirement: Same input - same output, always. Property: Uniform distribution; Requirement: Spreads keys evenly.

across integer space Property: Avalanche effect; Requirement: Small key change - large hash change Hash partitioning assumes diverse keys. When keys are not diverse, the math collapses: Property: Fast computation; Requirement: Called for every record at scale This is not a flaw in the formula - it is a data distribution problem. Hash partitioning works perfectly under normal conditions but cannot fix inherently skewed keys

Must remember:

- Hash partitioning formula: $P = \text{hash}(\text{key}) \bmod N$
- Deterministic: same key always maps to the same partition - enables direct lookup
- Modulo operator scatters hash integers across partitions 0 to N-1
- Works like dealing shuffled cards - fair spread when keys are diverse
- Key skew (duplicate/null keys) sends disproportionate data to one partition → hot spot
- Hash preserves no ordering - numerically adjacent keys land on unrelated partitions
- Best for uniform key distribution; fails when keys are concentrated on few values
- Next: strategic use cases where hash partitioning excels (point lookups, joins)

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N$

Example or contrast: Concept flow: userid: 12345 - hash function: - 847291837 - mod N (N=10 - Partition 7 Choose a key: userid = 12345 Apply hash: $\text{hash}(12345) \rightarrow 847291837$ (same input always yields same output)

Exam trap: Trap: "Hash partitioning guarantees equal partition sizes." It guarantees probabilistic balance only when keys are diverse - skewed keys break uniformity Trap: "Hash of 101 and 102 will land on adjacent partitions." Numerically close keys produce completely different hashes - hash scatters, it does not preserve order

BD-W09-T05 Use Cases: When to Choose Hash Partitioning

Tier A

Definition and why it matters: Hash partitioning is the default in many distributed systems, but default does not mean always correct. Choosing a partitioning strategy is an architectural decision driven by query patterns, key cardinality, and join behavior. Four key scenarios guide when hash partitioning is the right choice - and one critical scenario where it is the wrong choice

Core explanation: Requirement: High-cardinality keys; Hash Partitioning: Best choice Requirement: Point lookups by ID; Hash Partitioning: Best choice Requirement: Co-located joins; Hash Partitioning: Best choice (same hash + N); Alternative: Broadcast join (small table) Requirement: Chronological queries; Hash Partitioning: Avoid; Alternative: Range partitioning Requirement: Alphabetical range scans; Hash Partitioning: Avoid; Alternative: Range partitioning Requirement: Skewed keys; Hash Partitioning: Avoid (hot spots); Alternative: Salting, custom partitioner

Must remember:

- Hash partitioning excels with high-cardinality, diverse keys for uniform distribution
- Point lookups are near-instant - deterministic mapping goes directly to one node
- Co-located joins: partition both tables on same key + same hash + same N → shuffle-less join
- Shuffle-less joins can improve performance by orders of magnitude
- Red flag: hash is terrible for range scans (BETWEEN, , <) - forces full cluster scan
- Hash scatters data with no ordering - numerically close keys land on different nodes
- For chronological or alphabetical queries, use range partitioning instead
- Strategy choice depends on query patterns, not just data shape

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N$

Example or contrast: When: Dataset has high cardinality - many unique keys (Social Security numbers, UUIDs, transaction IDs) Key Type: UUID; Cardinality: Very high (unique per record); Hash Suitability: Excellent Key Type: User ID (millions); Cardinality: High; Hash Suitability: Excellent

Exam trap: Trap: "Hash partitioning is always the best default." It fails for range scans and skewed keys - context determines the right strategy Trap: "Co-located join works if both tables are hash-partitioned." Both must use the same key, same hash function, and same N - any mismatch triggers a shuffle

BD-W09-T06 Range Partitioning Mechanics: Key Ranges and Sorted Data

Tier A

Definition and why it matters: Range partitioning divides data into continuous, non-overlapping ranges based on a sortable key. Each range maps to a specific node or partition Partition: Node 1; Key Range: A - G; Example (names): Adams, Garcia

Core explanation: Hash partitioning excels at scattering data for balance and point lookups, but it is terrible for range-based queries. When the business needs to analyze data within specific time frames, numeric bands, or alphabetical segments, a different strategy is required: range partitioning Range partitioning optimizes for sorted data

and continuous queries - it organizes data instead of scrambling it The primary advantage of range partitioning is query efficiency through partition pruning Assumption: Names evenly spread A-Z; Reality: Regional datasets skew toward certain letters Assumption: Dates evenly spread across months; Reality: Seasonal spikes, event-driven bursts Assumption: Revenue evenly spread across brackets; Reality: Power-law distribution - few high-value records Use Case: Time-series analytics; Why Range Wins: Date-range queries prune to relevant partitions Use Case: Chronological reporting; Why Range Wins: Monthly/quarterly reports hit specific nodes Use Case: Alphabetical browsing; Why Range Wins: Name-range queries are localized Range partitioning sounds ideal, but it relies on a dangerous assumption: data is evenly distributed across the defined ranges Use Case: Ordered aggregations; Why Range Wins: Sorted data within partitions enables efficient scans Library analogy: If every author's name started with "S", the S shelf collapses under the weight while A-R and T-Z shelves stand empty Use Case: Log analysis by timestamp; Why Range Wins: Recent vs historical queries target different ranges When data concentrates in one range, that partition becomes a hot spot - the exact straggler problem hash partitioning tries to avoid, but through a different mechanism

Must remember:

- Range partitioning divides data into continuous, non-overlapping ranges on a sortable key
- Data within each partition is typically sorted - like books on a library shelf
- Query pruning: range queries (dates, names) target only relevant nodes - massive IO savings
- Hash scatters; range organizes - opposite strategies for opposite query patterns
- Critical assumption: data must be evenly distributed across ranges - otherwise hot spots form
- Range skew (e.g., all names starting with "S") creates stragglers identical to hash hot spots
- Best for time-series, chronological reports, and alphabetical range scans
- Next: boundary skew and dynamic range challenges in real-world data

Example or contrast: Query: All records from March 1 to March 15 Strategy: Hash partitioning; System Behavior: Records scattered everywhere - no metadata about date location; Nodes Scanned: All nodes Strategy: Range partitioning; System Behavior: Metadata says "March dates live on Node 4"; Nodes Scanned: Node 4 only

Exam trap: Trap: "Range partitioning is always better because it's organized." Uneven data distribution creates hot spots worse than hash skew in some cases Trap: "Range partitioning helps point lookups." Point lookups by non-range-key attributes still require scanning; range helps range queries on the partition key

BD-W09-T07 Handling Dynamic Ranges and Boundary Issues

Tier A

Definition and why it matters: Range partitioning creates an elegant, ordered library of data - but real-world datasets rarely conform to neat boundaries. The single biggest risk of range partitioning is data skew, also known as hot spots. While range partitioning is efficient for queries, it is incredibly vulnerable to uneven data distribution across defined ranges

Core explanation: Consider alphabetical partitioning across three nodes: Node: Node 1; Range: A - L; Expected Load: ~33% Node: Node 1 (A-L); Actual Load: ~25%; Status: Underutilized Node: Node 2 (M-S); Actual Load: ~60%; Status: Hot spot - overwhelmed Concept flow: Node 1: A-L: 25% load: Idle - Node 2: M-S: 60% load: STRAGGLER - Node 3: T-Z: 15% load: Idle - Entire job waits: for Node 2 On paper, this is an even split. But if the dataset represents users from a region where names starting with S are disproportionately common: Time: Every new record; Write Target: Latest partition (today's date); Problem: All writes hit one node Time: All other partitions; Write Target: Historical - read-only; Problem: Zero write load Impact: Write throughput; Consequence: Capped by single-node capacity - cannot scale out writes Node 2 becomes a straggler - the entire job slows to the pace of that one overloaded node while Node 1 and Node 3 sit idle Impact: Query latency; Consequence: Latest partition grows unboundedly - reads slow down Impact: Fault tolerance; Consequence: Single point of failure for all incoming data This is range skew - boundaries that appear balanced in design but fail in practice because the underlying data distribution is non-uniform Impact: Resource waste; Consequence: Cluster write capacity underutilized by N-1 nodes Beyond volume skew, boundary skew is a distinct and often more damaging problem - especially with time-series data Concept flow: Write stream - Today's partition: Node 4: ALL writes - Write stream - Write stream - Yesterday: Node 1: Idle - Last week: Node 2: Idle When data is partitioned by date and new records continuously append to the most recent date range: Every single write operation targets the same partition. One node handles all ingestion while others do nothing Strategy: Analyze distribution first; How It Helps: Set boundaries based on actual data histograms; Trade-off: Requires upfront analysis; boundaries may drift Strategy: Dynamic re-partitioning; How It Helps: Split overloaded ranges, merge empty ones; Trade-off: Operational complexity, rebalancing cost

Must remember:

- Range skew: data concentrates in one range despite seemingly balanced boundaries → straggler at query time
- Boundary skew: all new writes hit the latest range partition → write bottleneck at ingestion time
- Alphabetical ranges fail when name distribution is regional or cultural (e.g., many "S" names)
- Time-series append workloads create severe boundary skew on the "today" partition

- Boundary skew prevents horizontal scaling of write capacity - critical production failure
- Fixed ranges require constant vigilance and understanding of actual data distribution
- When hash and range both fail, custom partitioners provide domain-specific control
- Partitioning strategy must evolve as data grows and distribution shifts

Formula / decision rule: N-1

Example or contrast: Type: Range skew; Cause: Uneven data density within fixed ranges; When It Appears: Query time; Primary Impact: Straggler tasks, slow reads Type: Boundary skew; Cause: All new data appends to latest range boundary; When It Appears: Ingestion time; Primary Impact: Write bottleneck, cannot scale writes Both create hot spots, but boundary skew is particularly insidious because it affects the live ingestion pipeline, not just batch analytics

Exam trap: Trap: "Equal letter ranges = equal load." Alphabetical ranges assume uniform name distribution - regional datasets violate this assumption Trap: "Boundary skew only affects reads." Boundary skew primarily cripples write throughput - all ingestion hits one partition

BD-W09-T08 When Default Partitioners Fail: Custom Partitioning Logic

Tier A

Definition and why it matters: When default partitioners fail to distribute data evenly, frameworks like Spark allow defining a partitioner class or custom function. This function: Concept flow: Input Key - Default: Partitioner - Random spread: May break locality - Custom: Partitioner - Targeted routing: Preserves locality

Core explanation: Hash and range partitioning are off-the-shelf solutions that handle roughly 80% of use cases well. But standard strategies struggle when data becomes highly skewed or follows a complex, nonlinear distribution that generic math cannot capture. This is the territory of custom partitioners - moving from general-purpose formulas to domain-specific structures Failure Mode: Geographical data; Why Defaults Break: Hash puts neighboring addresses on different nodes; Custom Solution: Group by geo-hash or map tile Failure Mode: Hierarchical IDs; Why Defaults Break: Hash ignores business structure in ID prefix; Custom Solution: Route by department/country prefix Failure Mode: Known skew; Why Defaults Break: 50% null keys - one hash partition; Custom Solution: Explicit null routing or salting Failure Mode: Spatial queries; Why Defaults Break: Need neighbors co-located for local computation; Custom Solution: Tile-based or grid partitioning Failure Mode: Multi-tenant data; Why Defaults Break: Each tenant needs isolated partition; Custom Solution: Tenant-ID-based routing

Must remember:

- Default hash/range partitioners handle ~80% of cases; custom partitioners handle the rest
- Custom partitioner: function (key) → partitionindex replacing generic hash/modulo logic
- Geographical data: group by geo-hash/map tiles for spatial locality
- Hierarchical IDs: route by prefix (country, department) to keep related records co-located
- Custom partitioners preserve data locality - processing near where data lives
- Use when data has domain-specific structure that generic math cannot capture
- Requires fail-safe defaults for unknown/messy keys
- Next: PySpark implementation walkthrough with geographic region routing

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N; [0, N)$

Example or contrast: Problem: Processing delivery or traffic data where spatial locality matters Approach: Standard hash; Behavior: Address A - Node 7, Address B (next door) - Node 2 Approach: Geo-hash partitioner; Behavior: Address A - Tile 9q8yy, Address B - Tile 9q8yy - Same node

Exam trap: Trap: "Custom partitioners are always better." They add complexity - use only when defaults demonstrably fail for your data shape Trap: "Custom partitioners fix skew automatically." You must design the logic to handle skew - custom logic that ignores distribution still creates hot spots

BD-W09-T09 Building Custom Partitioning Logic in PySpark

Tier A

Definition and why it matters: When default hash and range partitioners cannot handle specific data distributions, PySpark allows explicit control over data routing through custom partition functions. This walkthrough builds a geographic region partitioner from scratch - routing transaction records to partitions based on business region keys (US, EU, APEC)

Core explanation: Every PySpark application begins with a Spark session. The sample data is a list of key-value pairs: Key (Region): "US"; Value (Transaction ID): "txn-001" Key (Region): "APEC"; Value (Transaction ID): "txn-003" Key (Region): "LATAM"; Value (Transaction ID): "txn-004" Key: "US"; Return Value: 0; Target Partition: Partition 0 Key: "LATAM" (unknown); Return Value: 0 (fail-safe default); Target Partition: Partition 0 When parallelized with defaults, Spark assigns a standard number of partitions using hash-based routing - regions may be scattered unpredictably The core logic maps region keys to specific partition indices: Argument: numPartitions; Value: 3; Purpose: Total partition

count Argument: partitionFunc; Value: geographicpartitioner; Purpose: Custom routing function Real data is messy - not every key appears in the mapping. Using Python's dict.get(key, default) prevents pipeline crashes: Concept flow: Original RDD: Mixed regions - partitionBy: 3, geographicpartitioner - Partition 0: US, LATAM - Partition 1: EU - Partition 2: APEC Known keys route to their designated partition Unknown keys (e.g., "LATAM") fall back to partition 0 The job continues instead of throwing an exception Verification: US + LATAM in Partition 0; Confirms: US mapped correctly; LATAM hit fail-safe default partitionBy requires two arguments: Verification: EU in Partition 1; Confirms: EU mapped correctly Code pattern: from pyspark.sql import SparkSession; spark = SparkSession.builder.appName("CustomPartitioner").getOrCreate(); sc = spark.sparkContext; data = [("US", "txn-001"), ("EU", "txn-002"), ("APEC", "txn-003"), ("LATAM", "txn-004")], This triggers a shuffle - Spark redistributes all records according to the custom function. After this transformation, data is physically separated by region Without Custom Partitioner: Regions scattered by hash; With Custom Partitioner: All US transactions on one node Without Custom Partitioner: Cross-region aggregations require shuffle; With Custom Partitioner: Region-level aggregations are local Code pattern: numpartitions = 3; regionmap = { "US": 0, "EU": 1, "APEC": 2, }; def geographicpartitioner(key): return regionmap.get(key, 0) unknown keys - partition 0 The glom() action collects all elements within each partition into a list - making partition contents visible: Without Custom Partitioner: Network overhead for every join; With Custom Partitioner: Minimal network movement Extension: More regions; Implementation: Add entries to regionmap Extension: More partitions; Implementation: Increase numpartitions, expand mapping Extension: Hierarchical routing; Implementation: Nested dict or prefix parsing in partition function Extension: DataFrame API; Implementation: Use repartition(num, cols) or partitionBy(cols) for DataFrames

Must remember:

- Custom partitioner: Python function key → int passed to RDD.partitionBy(N, func)
- Geographic example maps "US"→0, "EU"→1, "APEC"→2 for region-based routing
- Fail-safe: dict.get(key, 0) routes unknown keys to default partition - prevents crashes
- partitionBy triggers a shuffle to physically redistribute data by custom logic
- glom() verifies partition contents - collects each partition's elements into a list
- Custom partitioners achieve data locality - processing near where data lives
- Reduces expensive network shuffles for domain-structured workloads
- Extend by expanding the mapping dict and adjusting numpartitions

Formula / decision rule: [0, N)

Exam trap: Trap: "Custom partition function can return any integer." Must return values in range [0, N) where N = numPartitions - out-of-range values cause errors Trap: Forgetting fail-safe for unknown keys - production pipelines crash on unexpected data without .get(key, default)

BD-W09-T10

Data Partitioning Strategies: Module Summary

Tier C

Definition and why it matters: Data partitioning is the architectural foundation that makes horizontal scaling work in big data platforms. Simply adding machines does not improve performance - a deliberate strategy for dividing data and work across the cluster is required. This module covered the why, the how, and the manual control of data distribution

Core explanation: Adding machines without a partitioning strategy wastes hardware. One node doing 90% of the work while others idle is a common failure mode

Must remember:

- Partitioning enables scalability, parallelism, and data locality - replication does not
- One partition = one task; job finishes when the slowest task (straggler) completes
- Hash: $P = \text{hash}(\text{key}) \bmod N$ - scatter for balance, point lookups, co-located joins
- Range: continuous sorted ranges - fast range queries, vulnerable to skew
- Hash fails on range scans; range fails on write-heavy append workloads (boundary skew)
- Custom partitioners: domain-specific routing when defaults cannot handle data shape
- PySpark: partitionBy(N, func) + fail-safe defaults + glom() for verification
- Partitioning strategy is iterative - evolves as data and queries change

Formula / decision rule: $P = \text{hash}(\text{key}) \bmod N$

Example or contrast: : Purpose; Partitioning: Performance and scale; Replication: Safety and fault tolerance

Exam trap: Trap: "Replication and partitioning are interchangeable." Replication = safety; partitioning = speed - complementary, not substitutable Trap: "Hash is always the default choice." Range queries, time-series, and skewed keys require different strategies

Week 9 Exam Lens

Likely questions

- Define and explain Why Partitioning Is the Secret to Horizontal Scale; include the mechanism and one limitation.
- Compare Why Partitioning Is the Secret to Horizontal Scale with Partitioning vs Replication: Complementary but Distinct and state when each is preferred.
- Apply How Partitions Enable Parallel Execution to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $P = \text{hash}(\text{key}) \bmod N; \bmod N; N-1; [0, N)$

Mistakes to avoid: Trap: "More nodes = faster jobs." Without partitioning, adding nodes does nothing - data may still sit on one machine Trap: "Replication makes the system faster." Replication adds safety, not speed - it multiplies storage without adding parallelism Trap: "More machines automatically means faster jobs." Without enough partitions, added nodes sit idle - parallelism is bounded by partition count, not node count

Week 10: Data Partitioning II: Optimization and Skew

Week roadmap: Advanced Data Partitioning: Optimisation and Skew → Uniform Distribution versus Data Skew in Distributed Clusters → Identifying Data Skew and Hotspots in Production Pipelines → Salting: Breaking Up Skewed Partitions for Balanced Execution → The Shuffle Problem in Large-Scale Joins → Broadcast Joins: Eliminating Network Shuffle for Asymmetric Joins → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W10-T01 Advanced Data Partitioning: Optimisation and Skew

Tier A

Definition and why it matters: Distributed systems promise parallelism: add more workers, finish faster. That promise only holds when work is evenly distributed. Real-world data is rarely uniform - a few keys dominate volume (e.g., one country code with millions of transactions, others with dozens). When distribution is skewed, one or two workers carry most of the load while the rest sit idle. Jobs that should finish in minutes stretch to hours, and cloud spend buys idle capacity instead of throughput

Core explanation: This module moves beyond default partitioning settings to address optimisation and skew - the advanced challenges that separate operators who merely run jobs from engineers who tune production pipelines
 Concept flow: Evenly distributed data - All workers finish together - Skewed / hotspot data - Straggler worker delays entire job - Cluster runs at speed of slowest task
 Skew is diagnosed like a detective investigates a crime scene - through logs and the Spark UI
 Job reaches 99% completion quickly, then stalls on the final task for many minutes
 Uneven CPU/memory across cluster nodes (one node at 100%, others at 5-15%)
 Topic: Uniform distribution; Problem addressed: Why balance matters; Strategy: Foundation for all tuning
 Large gap between max task time and median task time in stage metrics
 Topic: Skew detection; Problem addressed: Hotspots, stragglers; Strategy: Spark UI, cluster monitoring
 When a single key (e.g., USA in a country-code join) concentrates all records in one partition, salting appends random suffixes (USA1, USA2, ...) to artificially split the key across partitions. The small table must be replicated/exploded to match salted keys so the join still produces correct results
 Topic: Salting; Problem addressed: Heavy keys; Strategy: Artificial key splitting
 Standard joins trigger expensive shuffles - moving data across the network so matching keys land on the same node. When one table is small enough to fit in executor memory, a broadcast join sends a copy of the small table to every worker. The large table stays put; the join runs locally at memory speed
 Topic: Broadcast join; Problem addressed: Expensive shuffle on asymmetric joins; Strategy: Replicate small table locally
 Reactive fixes (salting, broadcasting) help after the fact. Co-location is proactive: related datasets that are frequently joined are stored with the same partitioner and partition count from the start, so matching keys land on the same node without a network shuffle
 Topic: Co-location; Problem addressed: Repeated shuffle on related tables; Strategy: Harmonised partitioners at ingest
 Shuffle is often the slowest phase of a pipeline. Key metrics:

Must remember:

- Real data is skewed; default hash partitioning creates hotspots on dominant keys
- Five competencies: detect skew, salt heavy keys, broadcast small tables, co-locate related data, analyse shuffle metrics
- Hotspots = one worker overloaded; stragglers delay the entire stage
- Salting breaks up heavy keys; broadcast eliminates shuffle for small-table joins
- Co-location replaces shuffle joins with local joins when partitioners match
- Shuffle metrics (bytes, fetch wait, remote reads) guide partition-count tuning
- Performance tuning is iterative - adjust, measure, refine

Exam trap: Assuming more workers always means faster jobs - skew means the cluster runs at the speed of the slowest partition, not the average
 Confusing salting with random partitioning - salting is targeted at known heavy keys and requires join logic on both sides; it is not a blanket random shuffle

BD-W10-T02 Uniform Distribution versus Data Skew in Distributed Clusters

Tier A

Definition and why it matters: The core value proposition of distributed computing is parallelism: divide work across N workers and finish in roughly 1/N the time. With 10 workers and a 10-minute sequential task, a perfectly balanced job finishes in about 1 minute

Core explanation: That ideal requires uniform distribution - each worker receives roughly the same number of rows or the same computational load. When distribution is skewed, parallelism collapses
 Concept flow: Worker 1: small load - Worker 2: MASSIVE load - Worker 3: small load - Worker 4: small load - Finishes fast, idles - Finishes fast, idles - Straggler - delays entire job
 Concept flow: Worker 1: equal load - Worker 2: equal load - Worker 3: equal load - Worker 4: equal load - All finish ~same time
 In a skewed scenario, one partition holds vastly more data than the others
 Critical principle: The speed of the entire job is limited by the slowest task (Amdahl's law applied to data partitions). Workers 1, 3, and 4 finish quickly and sit idle while Worker 2 processes a mountain of data.

You pay for a large cluster but get the throughput of a single machine
 Jobs delayed from minutes to hours
 Expensive cloud resources wasted on idle CPU cycles
 Pipeline hits a performance wall instead of scaling linearly with added nodes
 In a balanced cluster, work is sliced evenly. Every worker processes a comparable volume
 Hardware utilisation approaches 100%
 No idle CPU cycles between fast and slow workers
 Job finishes as fast as physically possible for the given data volume
 Pipeline scales linearly as nodes are added
 Partitioning decisions made at ingest or transformation time propagate through every downstream join, aggregation, and shuffle. A single dominant key - a popular user ID, a default null value, a large-market country code - can negate the power of a thousand-node cluster
 The objective throughout advanced partitioning work is to move data from the skewed hotspot state to the uniform balanced state, using detection, mitigation (salting, broadcast), and proactive design (co-location)

Must remember:

- Parallelism only works when work is evenly distributed across partitions
- Skew = a few partitions much larger than others; creates hotspots and stragglers
- Entire job speed = speed of slowest task, not fastest or average
- Uniform distribution = all workers finish together; maximum cluster utilisation
- Skewed clusters waste money - idle workers while one node struggles
- Goal: move from skewed hotspot state to balanced uniform state
- This principle underpins all subsequent skew mitigation techniques

Formula / decision rule: workers and finish in roughly

Example or contrast: Aspect: Partition sizes; Skewed distribution: Few partitions much larger than rest; Uniform distribution: All partitions roughly equal
 Aspect: Worker utilisation; Skewed distribution: One hotspot; others idle; Uniform distribution: All workers active until completion
 Aspect: Job completion time; Skewed distribution: Bounded by slowest partition; Uniform distribution: Bounded by average partition time

Exam trap: Believing "I have 100 executors so I'm 100x faster" - with skew, effective parallelism may be 1
 Optimising average task time instead of max task time - the max (straggler) determines stage completion

BD-W10-T03

Identifying Data Skew and Hotspots in Production Pipelines

Tier A

Definition and why it matters: Data skew is a condition where a small number of data partitions are significantly larger than the rest. Distributed engines like Spark process one partition per core at a time; an oversized partition leaves one worker processing a mountain of data while others finish small hills and go idle
 Skew is a silent killer of big data performance - jobs appear healthy until the final straggler task reveals the imbalance

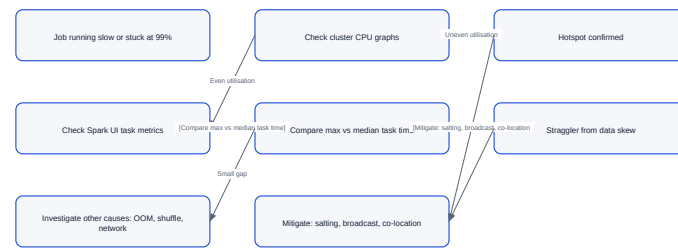
Core explanation: Monitor resource usage through tools like Ganglia, CloudWatch, or the Spark UI
 executors tab
 Healthy job: CPU and memory graphs across nodes move together like a synchronised heartbeat - all rise and fall in unison
 Concept flow: Node CPU graphs sync - Node A: 100% CPU sustained - Nodes B-D: 5-15% idle
 Metric: Median task duration; Balanced job: ~2 seconds; Skewed job: ~2 seconds
 Specific nodes at 100% CPU or maxed memory
 Metric: Max vs median ratio; Balanced job: Close (e.g., 1.5x); Skewed job: Very large (e.g., 150x)
 Other nodes in the same cluster at 5-15% utilisation
 One spike stays high while others drop off
 Concept flow: Job running slow or stuck at 99%
 - Check cluster CPU graphs - Hotspot confirmed - Check Spark UI task metrics - Compare max vs median task time - Straggler from data skew - Investigate other causes: OOM, shuffle, network - Mitigate: salting, broadcast, co-location
 A classic user-facing symptom: the Spark progress bar shoots to 99% in 2 minutes, then stays on the last 1% for another 10+ minutes
 This is tail latency - the final straggler task holding up the entire stage
 Navigate to the stage's Summary Metrics for tasks: A large gap between max and median task time identifies the straggler. This single slow task almost always traces back to a skewed partition containing a disproportionate share of data for one key
 Identification is step one. Mitigation strategies include: Salting - split heavy keys with random suffixes
 Broadcast join - when one join table is small
 Custom partitioning - explicit partition assignment for known skew
 Co-location - proactive design for frequently joined tables

Must remember:

- Data skew = few partitions much larger than others; silent performance killer
- Hotspot = uneven CPU/memory across nodes; one node overloaded, others idle
- Tail latency = job stuck at 99%; caused by a straggler task on a skewed partition
- Spark UI: compare max vs median task time - large gap confirms skew
- Healthy jobs show synchronised resource graphs across all nodes
- Heavy keys (popular values, nulls, defaults) are the usual root cause
- Detection precedes mitigation: salting, broadcast, custom partitioning, co-location

Exam trap: Blaming network or disk when max/median gap is huge - skew is the most common cause of the 99%-stuck pattern
 Looking only at stage duration, not per-task distribution - stage averages hide a single outlier task

Identifying Data Skew and Hotspots in Production Pipelines



Vector reconstruction of the source Memmail visual

Concept map: Identifying Data Skew and Hotspots in Production Pipelines

BD-W10-T04

Salting: Breaking Up Skewed Partitions for Balanced Execution

Tier A

Definition and why it matters: Real-world data is naturally imbalanced. One city may have millions of users while others have hundreds; one country code may dominate transaction logs. You cannot change the data - but you can change how the processing engine sees the keys

Core explanation: Salting is an engineering technique that artificially breaks up large skewed partitions by appending a random element to the key, forcing redistribution across more workers. Concept flow: Key: USA x 1M records - Single partition / Worker 2 - USA1 - Partition A - USA2 - Partition B - USA3 - Partition C - USA4. Original small table: USA, metadata; Exploded small table: USA1, metadata. Consider a join on countrycode. If key USA appears in millions of records, Spark's default hash partitioner sends every USA record to the same partition. One worker processes all USA data; others finish quickly and wait. Exploded small table: USA2, metadata. Exploded small table: USA3, metadata. Recognise the heavy key causing the hotspot (e.g., USA in a country-code join). Exploded small table: USA4, metadata. During processing, append a random suffix to the key on the large (skewed) table: Concept flow: skewed - LT - ST - C. Choose salt range N based on how many workers/partitions you want to spread across. One key becomes N distinct keys, tricking the partitioner into treating them as separate entities. After salting, the shuffle sends USA1, USA2, etc. to different partitions. Work that crushed one node is now distributed across N nodes processing in parallel. Critical join requirement: The small table must be exploded/replicated to match every salt value: Without this, salted keys on the large table will not find matches on the small table. Code pattern: USA - USA1, USA2, USA3, USA4 (random suffix in range [0, N-1]).

Must remember:

- Salting appends random suffixes to skewed keys to force redistribution across partitions
- Three steps: identify heavy key → add salt on large table → explode small table to match
- One heavy key becomes N distinct keys; shuffle spreads work across N workers
- Trade-off: more code complexity for balanced execution and pipeline stability
- Salt range should align with desired parallelism for the heavy key
- Always replicate/explode the small table - salting is a two-sided join fix
- Prefer broadcast join when one table is genuinely small

Formula / decision rule: based on how many workers/partitions you want to spread across. One key becomes

Example or contrast: Scenario: Heavy key on large table, both tables large; Best strategy: Salting. Scenario: One table small enough for memory; Best strategy: Broadcast join (simpler). Scenario: Tables frequently joined, known upfront; Best strategy: Co-location (proactive).

Exam trap: Advantage: Balanced execution across workers; Disadvantage: Increased code complexity. Advantage: Faster completion on skewed keys; Disadvantage: Must salt large table AND explode small table.

BD-W10-T05

The Shuffle Problem in Large-Scale Joins

Tier A

Definition and why it matters: Even with perfectly balanced data, shuffle remains a major bottleneck. In distributed systems like Spark, joining two large datasets almost always triggers a shuffle join - a wide dependency that reorganises data across the cluster.

Core explanation: Skew makes shuffles worse; but shuffles are expensive even when data is uniform. Concept flow: Node 1: key A rows + key B rows - Node 2: key C rows - Node 3: key A rows + key D rows - Shuffle: serialize, network transfer, deserialize - Before - After shuffle: all key A rows on same node. Component: CPU; Relative speed: Millions of instructions per second. A join requires all rows with matching keys to reside on the same physical node. Data is initially partitioned independently across nodes. Node 1 may hold some rows with key A; Node 3 holds the rest. Component: RAM / local memory; Relative speed: Very fast. Component: Network (inter-node); Relative speed: Slowest bottleneck. During the shuffle phase, Spark must: Component: Disk (spill); Relative speed: Slower than RAM,

often involved in large shuffles Serialise partition data Send it over the network (all-to-all communication) Deserialise on target nodes Optionally spill to disk if data exceeds RAM Every network arrow represents latency, serialisation overhead, and potential disk I/O Concept flow: Node 1 - Node 2 - Node 3 Modern hardware can compute far faster than the network can move data. A job may spend 80% of its time shuffling - waiting for data to travel across the data centre rather than processing locally Physical reality: The cable connecting Node A to Node B is the constraint. Shuffle joins are necessary for large-scale joins but rely on the slowest cluster component Shuffle creates an all-to-all pattern: every node sends data to every other node. As data volume grows: Network bandwidth saturates Serialisation/deserialisation CPU overhead increases Memory pressure causes disk spills Fetch wait times rise (workers idle waiting for remote data) The optimisation hierarchy for joins: Avoid the shuffle entirely - broadcast (small table) or co-location (proactive design) Reduce shuffle volume - filter before join, select only needed columns, partition pruning Make shuffle cheaper - right partition count, salting for skew, fast network (NVLink, InfiniBand) What if we didn't have to move both tables? What if one table was small enough to give a copy to everyone?

Must remember:

- Shuffle join reorganises data so matching keys land on the same node
- Network I/O is typically the slowest part of a big data pipeline - often 80% of job time
- Shuffle involves serialise → network transfer → deserialise; may spill to disk
- All-to-all communication pattern saturates bandwidth on large joins
- Shuffle is necessary for two large unco-located tables but should be avoided when possible
- Optimisation priority: avoid shuffle (broadcast/co-location) → reduce volume → tune shuffle
- Broadcast join is the shortcut when one table is small enough to replicate

Example or contrast: Situation: Two large tables, no co-location; Shuffle required?: Yes - shuffle join Situation: One table small, fits in memory; Shuffle required?: No - broadcast join Situation: Tables co-located with same partitioner; Shuffle required?: No - local join

Exam trap: Assuming a join on indexed data avoids shuffle in Spark - Spark's default join strategy still shuffles unless broadcast or co-location applies Ignoring shuffle when data looks balanced - uniformity fixes skew but not the fundamental cost of moving terabytes across the network

BD-W10-T06 Broadcast Joins: Eliminating Network Shuffle for Asymmetric Joins Tier A

Definition and why it matters: An asymmetric join pairs one very large table with one relatively small table - a pattern ubiquitous in analytics:

Core explanation: Billions of transactions joined to a product lookup table User event logs joined to a metadata dimension table Clickstream data joined to category codes Instead of moving both datasets across the network (shuffle join), a broadcast join sends a complete copy of the small table to every node. The large table never moves Spark identifies the smaller table (or the table is explicitly hinted with `broadcast()`) The small table is collected and broadcast to every executor's local memory Each executor joins its local partition of the large table with the in-memory copy of the small table No shuffle of the large table occurs The join runs at memory speed rather than network speed - often turning minutes into seconds When to use: One table is small enough to fit entirely in executor RAM Typical default threshold in Spark: ~10 MB (`spark.sql.autoBroadcastJoinThreshold`), tunable upward Ideal for lookup tables: product names, category codes, user metadata, country mappings When NOT to use: Small table exceeds executor memory → `OutOfMemoryError` Both tables are large → use shuffle join, salting, or co-location instead Trade-off: A small amount of memory replicated on every node buys a massive reduction in network traffic and execution time. With 100 executors, a 10 MB table uses ~1 GB total cluster memory - usually a excellent trade Spark SQL may auto-detect small tables below the broadcast threshold. Explicit hints are recommended when auto-detection fails due to statistics or threshold settings Code pattern: `from pyspark.sql.functions import broadcast; Explicit broadcast hint; result = largedf.join(broadcast(smallerdf), on="productid")`

Must remember:

- Broadcast join replicates the small table to every node; large table stays local
- Eliminates all-to-all shuffle; join runs at memory speed
- Best for asymmetric joins: large fact + small dimension/lookup table
- Small table must fit in executor RAM; default threshold ~10 MB (tunable)
- Trade memory on each node for massive network and time savings
- Can achieve 10x+ speedup vs shuffle join for suitable workloads
- Use `broadcast()` hint when auto-detection does not trigger

Example or contrast: Concept flow: Large table A - Network all-to-all - Large table B - Join on target nodes - Small table - Worker 1 - Worker 2 - Worker 3 - Large table - stays local Aspect: Data movement; Shuffle join: Both tables shuffled; Broadcast join: Only small table broadcast Aspect: Network pattern; Shuffle join: All-to-all; Broadcast join: One-to-all (small table)

Exam trap: Broadcasting a table that only looks small - compressed on disk but expands in memory; always consider deserialized size Setting threshold too high - broadcasting a 500 MB table to 200 executors causes OOM across the cluster

BD-W10-T07 Data Co-location: Proactive Shuffle-Free Join Architecture

Tier A

Definition and why it matters: Salting and broadcast joins fix problems after data has landed in suboptimal locations. Data co-location is a proactive architectural strategy: design the system so that datasets frequently joined together are physically stored or partitioned on the same nodes from the start

Core explanation: If related data is already in the same place, no shuffle is needed Concept flow: Orders: user 123, 456 - Users: user 123, 456 - Orders: user 789 - Users: user 789 - Local join - no network - Node1 - Local join - no network - Node2 Two massive datasets - e.g., orders and users - are normally partitioned independently. Their partition files may differ in count and hash distribution. A join forces a shuffle to co-locate matching keys Co-location ensures both datasets use: The exact same partitioner (e.g., hash on userid) The exact same number of partitions Step: 1; Action: Identify tables frequently joined together When both conditions hold, Spark's hash logic becomes predictable: userid = 123 in the orders table and userid = 123 in the users table always land in partition X on the same machine Step: 2; Action: Choose a common partition key (the join key) Step: 3; Action: Write both datasets with partition By(joinkey) and identical partition count When orders.join(users, on="userid") is called: Step: 4; Action: Use the same hash partitioner (Spark default HashPartitioner) Spark examines the execution plan Step: 5; Action: Maintain partition count consistency in downstream ETL Detects both datasets share partitioning on userid with equal partition count Recognises matching keys are already co-located on the same nodes Executes a shuffle-free (or local-only) join - no network transfer required During ETL/ingest: Both must use the same partition count (e.g., 200) and partition on the same key Drastically reduced network traffic - no cross-node data movement for joins No serialisation/deserialisation overhead from shuffle Lower risk of the 99%-stuck problem - no shuffle stragglers from join Predictable performance for recurring join patterns in analytics pipelines Code pattern: Both written with same partitions and partition key; ordersdf.write.partitionBy("userid").parquet("orders/"); usersdf.repartition(200, "userid").write.parquet("users/")

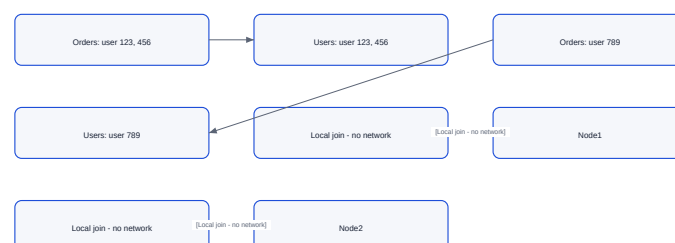
Must remember:

- Co-location = related datasets stored on same nodes via matching partitioner and partition count
- Proactive strategy: design at ETL time, not fix at query time
- Same join key + same partition count → predictable hash placement
- Spark detects co-location and executes shuffle-free local joins
- Eliminates network traffic, serialisation overhead, and shuffle stragglers for designed joins
- Requires knowing data access patterns upfront; less flexible if join keys change
- Holy grail of join optimisation: shuffle join → shuffle-less join

Example or contrast: Strategy: Co-location; Timing: Proactive (at ingest); Shuffle avoided?: Yes, for designed joins; Flexibility: Low - fixed join key Strategy: Broadcast join; Timing: Reactive (at query time); Shuffle avoided?: Yes, for small table; Flexibility: Medium - any small table Strategy: Salting; Timing: Reactive (at query time); Shuffle avoided?: No - still shuffles, but balanced; Flexibility: High - handles skew

Exam trap: Advantage: Shuffle-free joins; Requirement / cost: Must know join patterns ahead of time Advantage: Best downstream analytics performance; Requirement / cost: Planning during ETL/ingest phase

Data Co-location: Proactive Shuffle-Free Join Architecture



Vector reconstruction of the source Mermaid visual

Concept map: Data Co-location: Proactive Shuffle-Free Join Architecture

BD-W10-T08 Analysing Shuffle Performance and the Partition Count Dilemma

Tier A

Definition and why it matters: "How many partitions should I have?" has no universal answer. Partition count is a key tuning parameter that depends on data volume, cluster size, and workload type. Getting it wrong in either direction degrades performance

Core explanation: Concept flow: Too few partitions - Large chunks - disk spill / OOM - Optimal count - Balanced tasks, no spill, low overhead - Too many partitions - Small file problem - scheduling overhead When partition count is too low, each partition holds a large data chunk Disk spill - if a single partition exceeds available executor RAM, Spark writes temporary data to disk; disk I/O is orders of magnitude slower than memory Out-of-memory errors - partitions too massive for the worker to handle even with spill Underutilised cluster - fewer tasks than available cores; workers sit idle Residual skew amplified - fewer partitions means heavier keys concentrate even more Rule of thumb: Target partition sizes of roughly 128 MB-200 MB for HDFS-aligned workloads, adjusted for memory constraints Counterintuitively, thousands of tiny partitions also hurt performance Small file problem - each partition is a file; thousands of tiny files burden metadata management (NameNode in HDFS, catalog in cloud storage) Task scheduling overhead - if tasks run for milliseconds but take seconds to schedule, most cluster time is administrative overhead, not computation Shuffle overhead - more partitions = more shuffle files and network connections Symptom: High task count, low task duration, poor overall throughput Measures the sheer volume of data moved during shuffle operations Compare against input data size If shuffle bytes are massive relative to input, investigate: Unnecessary shuffles (missing broadcast opportunity) Inefficient joins (large-large shuffle when co-location possible) Wide transformations on unfiltered data Metric: Shuffle read/write bytes; What it reveals: Volume of data moved; Red flag: Massive vs input size Measures how long tasks sit idle waiting for remote shuffle data to arrive over the network Metric: Fetch wait time; What it reveals: Network congestion; Red flag: High wait, low compute High fetch wait = network bottleneck Metric: Remote bytes read; What it reveals: Cross-node traffic; Red flag: High when co-location expected Workers are ready to compute but the "highway is congested" Concept flow: Run job, open Spark UI - Check shuffle metrics - Fetch wait high? - Investigate network / co-location / broadcast - Disk spills present? - Increase partitions or executor memory - Tasks too short / many? - Decrease partition count - Configuration near optimal Indicates: saturated network, poor co-location, or oversized shuffle Direct measure of network traffic for data reads. Compare against local bytes read Factor: Cluster cores; Guidance: Start with 2-4x total cores across executors In a perfectly co-located join: remote bytes should be very low Factor: Data volume; Guidance: Aim for ~128-200 MB per partition

Must remember:

- Partition count is a tuning parameter - no one-size-fits-all answer
- Too few partitions → disk spill, OOM, underutilised cluster
- Too many partitions → small file problem, scheduling overhead
- Three key metrics: shuffle read/write bytes, fetch wait time, remote bytes read
- High fetch wait = network bottleneck; high remote bytes = unnecessary cross-node traffic
- Co-located/broadcast joins should show very low remote bytes
- Tuning is iterative: adjust partitions, measure metrics, refine
- Re-tune as data volume and cluster configuration evolve

Exam trap: Setting partition count once and never revisiting - optimal count changes with data growth and cluster resizing Using default 200 partitions for all workloads - may be too few for TB-scale or too many for MB-scale

BD-W10-T09 Data Partitioning Optimisation: Module Synthesis

Tier A

Definition and why it matters: This module progressed from identifying distribution problems to mastering production-grade optimisation. Four principles govern effective partitioning strategy

Core explanation: A single straggler task can negate the power of a thousand-node cluster. Skew leads to hotspots, hotspots lead to memory errors, and memory errors lead to downtime Concept flow: Data skew - Hotspot node - Memory errors - Pipeline failure Detection is non-negotiable: Spark UI: gap between median and max task time Cluster monitoring: uneven CPU/memory (synchronised heartbeat vs isolated spike) Symptom: job stuck at 99% completion Real-world data is skewed by nature. When heavy keys cause hotspots, salting artificially forces redistribution by appending random suffixes to keys Add salt to the large (skewed) table Explode/replicate the small table to match Metric: Shuffle read/write bytes; What it tells you: Volume of data movement Trade code complexity for balanced execution and on-time job completion Metric: Fetch wait time; What it tells you: Network congestion Network I/O is the slowest cluster component. For asymmetric joins where one table fits in memory: do not shuffle - broadcast Metric: Remote bytes read; What it tells you: Whether co-location/broadcast is working Send small table to every executor Problem: Heavy key concentration; Symptom: Hotspot, straggler; Solution: Salting Large table stays local Problem: Large + small table join; Symptom: Expensive shuffle; Solution: Broadcast join Achieve 10x+ speedups for lookup/dimension joins Problem: Recurring joins on same key; Symptom: Repeated shuffle cost; Solution: Co-location at ETL Most effective shortcut in the Spark developer's toolkit Problem: Wrong partition count; Symptom: Disk spill or scheduling overhead; Solution: Tune partition count There is no permanent magic number for partitions. As data grows and clusters change, partitioning strategy must evolve Problem: General shuffle cost; Symptom: High fetch wait, remote bytes; Solution: Co-location, broadcast, or filter early Guide decisions with metrics, not guesses: By the end of this module, a practitioner should be able to: Identify data skew from logs and Spark UI (hotspots, tail latency, max vs median) Apply salting with correct salt range and two-sided join logic Evaluate broadcast join eligibility based on table

size and memory Implement co-location via harmonised partitioners at ingest Analyse shuffle metrics to tune partition count iteratively

Must remember:

- Skew is the silent killer: one straggler negates massive clusters; detect via Spark UI and monitoring
- Salting = primary manual fix for heavy keys; requires two-sided join logic
- Broadcast join = most effective shuffle elimination for small-table joins; 10x+ speedups possible
- Co-location = proactive shuffle-free joins via matching partitioners at ETL time
- Partition count has no magic number; tune iteratively with shuffle metrics
- Four commandments: detect skew, salt when needed, broadcast when possible, measure and tune
- Move from running jobs to engineering optimised production pipelines

Exam trap: Treating the four commandments as independent - they work together: detect skew, choose the right join strategy, then verify with metrics Skipping detection and jumping to salting - confirm skew first; shuffle without skew may be the real bottleneck

Week 10 Exam Lens

Likely questions

- Define and explain Advanced Data Partitioning: Optimisation and Skew; include the mechanism and one limitation.
- Compare Advanced Data Partitioning: Optimisation and Skew with Uniform Distribution versus Data Skew in Distributed Clusters and state when each is preferred.
- Apply Identifying Data Skew and Hotspots in Production Pipelines to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: workers and finish in roughly; based on how many workers/partitions you want to spread across. One key becomes

Mistakes to avoid: Assuming more workers always means faster jobs - skew means the cluster runs at the speed of the slowest partition, not the average Believing "I have 100 executors so I'm 100x faster" - with skew, effective parallelism may be 1 Blaming network or disk when max/median gap is huge - skew is the most common cause of the 99%-stuck pattern

Week 11: Distributed ML I: Concepts and Algorithms

Week roadmap: Distributed Machine Learning: Concepts and Algorithms Overview → Communication Overhead and Data Movement in Distributed ML → Synchronisation, Stragglers, and Convergence in Distributed Training → Data Parallelism: Sharding the Dataset Across Workers → Model Parallelism: Splitting the Network Across Devices → Stochastic Gradient Descent: The Optimisation Engine of ML → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W11-T01 Distributed Machine Learning: Concepts and Algorithms Overview Tier A

Definition and why it matters: Machine learning models - from deep neural networks to advanced classifiers - have been trained largely on single machines. Real-world constraints break that assumption:

Core explanation: Datasets too massive to fit on one disk (petabytes of logs, images, transactions) Concept flow: Distributed ML Models too complex to fit in even the most powerful GPU memory (LLMs with hundreds of billions of parameters) Distributed machine learning breaks massive tasks into coordinated pieces across multiple machines that work as a single powerful unit Adding hardware does not automatically halve training time. Distribution introduces overhead in communication, data movement, and synchronisation that can slow training if unmanaged. Up to 70% of training time in large clusters may be spent on communication rather than computation Scaling is not just adding computers. It introduces: Constant gradient and parameter movement across nodes every training iteration Network latency and bandwidth limits Coordination timing (stragglers, convergence) Concept flow: Scaling challenges - Parallelism strategies - Distributed SGD mechanics - Aggregation strategies - Production ML at scale SGD is the heartbeat of modern ML training. The mathematics of optimisation changes when gradients are calculated across a cluster of workers rather than on a single machine Update rule on one machine: $\theta(t+1) = \theta_t - \eta \nabla L(\theta_t)$ In distributed settings, ∇L must be aggregated from multiple workers before the global update Once workers compute local gradients, they must be combined. Strategies differ in timing: Synchronous - wait for all workers; mathematically pure, straggler-sensitive Asynchronous - update immediately; fast but stale gradients Local SGD - many local steps before synchronising; reduces communication

Must remember:

- Distributed ML coordinates multiple machines when data or models exceed single-node capacity
- Four areas: scaling challenges, parallelism strategies, distributed SGD, aggregation methods
- Data parallelism splits data; model parallelism splits the model
- SGD remains the core optimiser; gradients are computed locally and aggregated globally
- Communication, not computation, often dominates training time (up to 70%)
- Aggregation strategy (sync/async/local) trades speed against convergence stability
- Foundation for understanding how modern AI is trained at scale

Formula / decision rule: $\theta_{t+1} = \theta_t - \eta * \nabla L(\theta_t)$; ∇L

Example or contrast: Two primary blueprints for distribution: Strategy: Data parallelism; What is split: Dataset (shards); When to use: Model fits in one GPU; data is huge Strategy: Model parallelism; What is split: Model layers/tensors; When to use: Model too large for one device

Exam trap: Assuming 2x hardware = 2x speed - communication overhead often dominates; can even slow training Confusing data parallelism with model parallelism - data parallelism duplicates the model; model parallelism splits it

BD-W11-T02 Communication Overhead and Data Movement in Distributed ML Tier A

Definition and why it matters: Doubling hardware rarely halves training time. Distributed machine learning introduces three hidden costs that often become the speed limit of ML infrastructure:

Core explanation: Data movement overhead Concept flow: notify Understanding these costs is prerequisite to designing systems that actually benefit from scale On a single node, the GPU/CPU accesses data via high-speed local memory buses (NVLink, PCIe). Once work is distributed, the network enters the picture - and the network is orders of magnitude slower than local memory Condition: Communication time < compute time; Outcome: Adding GPUs helps GPU processing may be a small fraction of total iteration time; the rest is consumed by notify and synchronise phases Condition: Communication time > compute time; Outcome: Adding GPUs does not help - may slow training In distributed ML, data is not moved once - it is continuously moved every training iteration: Condition: Network saturated; Outcome: All workers wait; GPUs idle Workers push local gradients to a central or collective point Updated parameters are pulled/broadcast back to all workers This creates persistent overhead driven by three factors: The delay in sending a message, even for tiny packets. Time to traverse wires and switches adds up across thousands of workers communicating every iteration Network pipes have maximum capacity. Syncing a model with billions of parameters across 100 nodes can saturate network bandwidth, causing everything to grind to a halt Some algorithms require re-organising data across the cluster (all-to-all communication) so each node gets a representative sample. Shuffle in ML

training mirrors the expensive shuffle in Spark - often the slowest pipeline phase Key insight: If communication takes longer than computation, more GPUs waste money on idle hardware waiting for gradients Prioritise fast interconnects - InfiniBand, NVLink, high-bandwidth Ethernet for multi-node training Reduce communication frequency - local SGD, gradient compression, larger mini-batches before sync Choose aggregation strategy wisely - async avoids sync waits but introduces staleness Right-size clusters - more nodes is not always better; find the point where communication dominates

Must remember:

- Distribution introduces communication, data movement, and synchronisation overhead
- Network is much slower than local memory - the primary bottleneck in multi-node training
- Gradients and parameters move every iteration, not just once
- Three drivers: latency (per-message delay), bandwidth (pipe capacity), shuffle (all-to-all cost)
- If communication compute, adding GPUs does not speed up training
- GPU time may be minority of iteration; notify/sync phases dominate
- Design for interconnect speed and minimise communication frequency

Example or contrast: Aspect: Data access; Single node: Local memory bus (fast); Distributed cluster: Network (slow) Aspect: Gradient sync; Single node: Not needed; Distributed cluster: Every iteration Aspect: Bandwidth; Single node: Internal bus (high); Distributed cluster: Shared network (limited)

Exam trap: Benchmarking only compute FLOPs when selecting hardware - interconnect speed matters equally Scaling to 100 nodes without measuring communication ratio - may get slower than 10 nodes

BD-W11-T03 Synchronisation, Stragglers, and Convergence in Distributed Training Tier A

Definition and why it matters: Even with infinite network bandwidth, distributed training faces a logical challenge: coordinating workers that finish at different times. Scaling is not just a networking problem - it is a timing and mathematical problem

Core explanation: The speed of the entire cluster is dictated by its slowest member Concept flow: 99 workers: done in 1s - Idle, waiting to sync - 1 straggler: 10s - Global update only after slowest finishes Scenario: 100 GPUs work in parallel. 99 finish in 1 second; one takes 10 seconds due to a network glitch, background OS process, or hardware variance Mode: Synchronous; Straggler impact: All wait for slowest; Gradient freshness: Always fresh Mode: Asynchronous; Straggler impact: No waiting; Gradient freshness: Potentially stale In synchronous training, the 99 fast GPUs sit idle - paying for power and hardware that performs no useful work. This is massive compute resource waste Connection to data skew: The straggler problem in ML training mirrors the straggler task in Spark - the slowest unit determines overall completion time To avoid waiting for stragglers, some systems use asynchronous updates - workers push results as soon as they are ready, without waiting for peers New problem: gradient staleness. A slow worker's gradient may arrive after the model has been updated 10 times by faster workers. Using this old (stale) gradient can cause: Numerical instability - optimisation steps based on outdated loss landscape Failed convergence - model may not reach optimal loss Oscillation - conflicting gradient directions from workers at different model versions Concept flow: V100: fast - A100: faster - CPU: slow - Older GPU: medium - Load balancing challenge - Inevitable stragglers without careful scheduling Real clusters are rarely uniform: Mix of older and newer GPU generations Combination of CPUs and GPUs in the same cluster Different memory capacities per node Problem: Stragglers (sync); Mitigation: Backup workers, speculative execution, bounded wait timeouts Variable network paths (rack topology, cross-AZ latency) Problem: Stale gradients (async); Mitigation: Staleness-aware learning rates, gradient clipping, periodic sync Load balancing - giving each node the right amount of work so all finish simultaneously - becomes extremely difficult with heterogeneous hardware Problem: Heterogeneity; Mitigation: Work partitioning by device capability, dedicated GPU pools Problem: Both; Mitigation: Local SGD - many steps locally, sync less frequently

Must remember:

- Cluster speed = speed of slowest worker (straggler problem)
- Synchronous training: all wait → idle GPUs, but fresh gradients
- Asynchronous training: no waiting → high utilisation, but stale gradients
- Stale gradients cause instability and convergence failure
- Hardware heterogeneity makes load balancing extremely difficult
- Scaling is a timing AND mathematical problem, not just networking
- Trade-off: speed (async) vs gradient accuracy (sync); Local SGD is middle ground

Exam trap: Concept flow: Need for speed - Need for accurate gradients - Asynchronous updates - Synchronous updates - Gradient staleness risk - Straggler idle time Distributed training must balance:

Synchronisation, Stragglers, and Convergence in Distributed Training



Vector reconstruction of the source Mermaid visual

Concept map: Synchronisation, Stragglers, and Convergence in Distributed Training

BD-W11-T04

Data Parallelism: Sharding the Dataset Across Workers

Tier A

Definition and why it matters: Data parallelism is the industry-standard approach when the model fits in a single GPU's memory but the dataset is so large that single-machine training would take weeks or months

Core explanation: The core concept: shard the dataset - split it into partitions, assign each partition to a different worker
 Concept flow: GPU 1: shard 1 + full model copy - GPU 2: shard 2 + full model copy - GPU 3: shard 3 + full model copy - GPU 4: shard 4 + full model copy - Full dataset - Gradient aggregation - Global model update applied to all copies
 Step: 1. Local forward pass; What happens: Each worker processes a mini-batch from its shard
 Step: 2. Local backward pass; What happens: Each worker computes gradients on its local data
 Step: 3. Independent compute; What happens: Workers operate in parallel - no communication yet
 Step: 4. Gradient aggregation; What happens: Local gradients are averaged (or summed) into one global gradient
 Step: 5. Global update; What happens: Aggregated gradient applied to model on every worker
 Step: 6. Synchronise weights; What happens: All workers now have identical updated weights
 Step: 7. Next iteration; What happens: Repeat with next mini-batch from each shard
 Critical distinction: Every worker maintains a full copy of the model architecture - same layers, same weights, same configuration. Only the data is split, not the model
 Condition: Model fits in one GPU RAM; Data parallelism suitable?: Yes
 Condition: Dataset too large for one machine; Data parallelism suitable?: Yes
 Effective batch size scales with workers - 4 workers x batch size 32 = effective batch of 128
 Condition: Model has hundreds of billions of parameters; Data parallelism suitable?: No - need model parallelism
 Larger effective batch can speed up convergence on massive datasets
 Condition: Communication bandwidth is high; Data parallelism suitable?: Yes - aggregation is affordable
 Straightforward to implement - same model code, different data shards
 Condition: Communication bandwidth is low; Data parallelism suitable?: Challenging - consider local SGD
 Most robust and widely supported strategy in frameworks (TensorFlow, PyTorch)
 With W workers each processing mini-batch size B : Effective batch size = $W \times B$
 Larger effective batch sizes generally provide more stable gradient estimates but may require learning rate adjustment (often linear scaling rule: scale eta with batch size)

Must remember:

- Data parallelism: shard dataset, replicate full model on every worker
- Each worker computes local gradients on its data shard independently
- Gradients are aggregated (averaged) and applied globally to all model copies
- Effective batch size = workers x per-worker batch size
- Industry standard for big datasets where model fits in one GPU
- Communication happens at gradient aggregation step (sync mode)
- Does not work when model itself exceeds single-device memory

Formula / decision rule: workers each processing mini-batch size; Effective batch size = $W \times B$

Example or contrast: Aspect: What is split; Data parallelism: Dataset; Model parallelism: Model layers/tensors
 Aspect: Model per worker; Data parallelism: Full copy; Model parallelism: Partial (subset of layers)
 Aspect: Communication; Data parallelism: At end of batch (gradient sync); Model parallelism: During forward/backward pass

Exam trap: Claiming data parallelism splits the model - it splits data; model is replicated on every worker
 Ignoring communication at aggregation step - data parallelism still requires gradient sync every step (in sync mode)

BD-W11-T05

Model Parallelism: Splitting the Network Across Devices

Tier A

Definition and why it matters: Data parallelism fails when the model itself is too large for a single GPU. Modern LLMs with hundreds of billions of parameters require hundreds of gigabytes of memory for a single copy - far exceeding any one device's capacity

Core explanation: Model parallelism splits the actual layers (or tensor operations) of the network across multiple devices
 Concept flow: Input batch - Node A: Layers 1-10 - Node B: Layers 11-20 - Node C: Layers 21-30 - Output / loss

Instead of every GPU holding the full model, the architecture is divided across nodes: Node A handles layers 1-10 Aspect: When communication occurs; Data parallelism: End of batch (gradient sync); Model parallelism: During forward AND backward pass Aspect: Communication pattern; Data parallelism: All-reduce gradients; Model parallelism: Pass activations/gradients between stages Aspect: Idle risk; Data parallelism: Stragglers at sync; Model parallelism: Pipeline bubbles (stages waiting) Data flows sequentially through the split - layer 11 cannot start until layer 10 finishes. This is pipeline parallelism Aspect: Complexity; Data parallelism: Lower; Model parallelism: Higher - timeline management critical A more granular approach: instead of splitting by layer, split the large matrix operations (tensors) themselves across GPUs Multiple GPUs compute a single layer simultaneously Each GPU holds a portion of the weight matrix Concept flow: naive Partial results are combined via communication within the layer Useful when individual layers are too large for one device, even if the layer count could fit Strategy: Data parallelism; Role: Different data shards across replica groups Strategy: Pipeline parallelism; Role: Layers split across GPUs within a group Strategy: Tensor parallelism; Role: Large layers split across GPUs Concept flow: Data parallelism across replica groups - Pipeline parallelism within group - Tensor parallelism within layer In model parallelism, nodes must communicate during the forward and backward passes because layer $N+1$ depends on layer N 's output. Managing the timeline and data transfer between stages is critical to avoid leaving GPUs idle Scenario: Huge dataset, model fits one GPU; Strategy: Data parallelism In naive pipeline parallelism, GPUs spend time idle waiting for upstream layers: Scenario: Model too large for one GPU; Strategy: Model parallelism Scenario: Both huge data and huge model; Strategy: Combined data + model parallelism Advanced techniques (micro-batching, GPipe, PipeDream) reduce pipeline bubbles by overlapping forward/backward passes across micro-batches Scenario: Single layer too large; Strategy: Tensor parallelism State-of-the-art systems (training GPT-class models) combine both:

Must remember:

- Model parallelism splits the network across devices when model exceeds single-GPU memory
- Pipeline parallelism: layers assigned to different nodes; sequential data flow
- Tensor parallelism: matrix operations split across GPUs within a single layer
- Communication occurs during forward/backward passes, not just at batch end
- Pipeline bubbles cause GPU idle time; advanced scheduling reduces this
- LLM training combines data + pipeline + tensor parallelism
- Use data parallelism for big data; model parallelism for big networks

Formula / decision rule: $N+1$

Exam trap: Using data parallelism for LLM-scale models - model won't fit; need model parallelism Assuming model parallelism has less communication - it has more frequent communication, just smaller messages per step

BD-W11-T06 Stochastic Gradient Descent: The Optimisation Engine of ML

Tier A

Definition and why it matters: Infrastructure (networking, parallelism strategies) enables scale, but stochastic gradient descent (SGD) is the mathematical engine that makes learning possible. Every distributed training system ultimately implements a variant of: compute gradients locally, aggregate, update weights globally

Core explanation: SGD minimises a loss function $L(\theta)$ by iteratively updating model weights θ Concept flow: Current weights θ - Compute $L(\theta)$ - Step opposite to gradient - Updated weights $\{\theta+1\}$ Symbol: θ_t ; Meaning: Current model weights at step t Symbol: $\nabla L(\theta_t)$; Meaning: Gradient (slope of loss at current position) Symbol: η ; Meaning: Learning rate - controls step size Symbol: θ_{t+1} ; Meaning: Updated weights after one step Intuition: Imagine a 3D loss landscape. The goal is to find the lowest point (global minimum). At each step, compute the gradient - the direction of steepest ascent - and take a step in the opposite direction (steepest descent) $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$ Learning rate η : Too large $\rightarrow$ overshoot minimum, diverge. Too small $\rightarrow$ slow convergence, may get stuck in local minima Sample a small mini-batch of data (not the full dataset - that would be batch gradient descent; not one sample - that would be pure SGD) Compute loss on the mini-batch Calculate gradient $\nabla L(\theta_t)$ via backpropagation Update weights using the formula above Repeat thousands or millions of times until loss converges "Stochastic" refers to the randomness introduced by sampling mini-batches - each gradient estimate is noisy but unbiased on average On a single machine, one gradient is computed per step. In distributed ML: Multiple workers stand on different parts of the data Each computes its own local gradient simultaneously The challenge: combine local gradients into one accurate global step $\nabla L(\text{global}) \approx (1/W) \sum_{i=1}^W \nabla L_i(\theta_t)$ where W is the number of workers and ∇L_i is the gradient computed on worker i 's mini-batch The distributed SGD cycle: Each worker computes local gradient on its shard Gradients are pushed to central/collective point Global gradient computed (average) Master weights updated Updated weights pulled to all workers The mathematics is the same; the engineering challenge is steps 2-5 happening fast enough that workers do not idle

Must remember:

- SGD minimises loss by iteratively stepping opposite to the gradient

- Update rule: $\theta(t+1) = \theta(t) - \eta \nabla L(\theta(t))$
- η = learning rate; controls step size and convergence stability
- Single machine: one mini-batch, one gradient, one update per step
- Distributed: multiple local gradients aggregated into one global update
- Global gradient $\approx$ average of local gradients across workers
- SGD is the mathematical foundation underlying all distributed training cycles

Formula / decision rule: $L(\theta); \theta$

Example or contrast: Variant: Full batch; Data per step: Entire dataset; Gradient quality: Exact; Speed per step: Slow Variant: Mini-batch SGD; Data per step: Small batch (32-512); Gradient quality: Noisy but useful; Speed per step: Fast Variant: Pure SGD; Data per step: One sample; Gradient quality: Very noisy; Speed per step: Fastest per step

Exam trap: Confusing SGD with gradient descent on full data - SGD uses mini-batches; "stochastic" means sampled Forgetting learning rate when scaling workers - effective batch size changes; may need to scale η

BD-W11-T07 Distributing Gradient Calculation: The Push-Update-Pull Cycle

Tier A

Definition and why it matters: In a distributed system, the gradient is not computed in one place - it is computed in pieces across the cluster. Every training iteration follows a four-step cycle that transforms independent local work into a synchronised global model update

Core explanation: Concept flow: $W_1 - W_2 - \dots - W_N - PS$ Each worker takes its own mini-batch from its data shard. Using the current model weights, it performs: Forward pass - compute predictions and loss Backward pass - compute local gradient ∇L_i This gradient represents the best improvement direction based only on the data that worker saw Local gradients are sent to a central or collective aggregation point: Architecture: Parameter server; Push mechanism: Workers - central server; Pull mechanism: Server - all workers Parameter server architecture - workers push to dedicated server(s) Architecture: Ring AllReduce; Push mechanism: Neighbour-to-neighbour exchange; Pull mechanism: Result available on all nodes after ring traversal AllReduce architecture - workers exchange gradients directly in a ring or tree pattern Architecture: NCCL (single-node multi-GPU); Push mechanism: GPU-to-GPU via NVLink; Pull mechanism: Synchronous update on all replicas Goal: move local gradient data to where it can be combined The system combines all local gradients (typically by averaging): $\nabla L(\text{global}) = (1/W) \sum_{i=1}^W \nabla L_i$ The central parameter server (or collective result) applies the SGD update: $\theta(t+1) = \theta(t) - \eta \nabla L(\text{global})$ Updated weights are broadcast back to every worker. Workers previously held stale (old) weights; now all have identical, updated $\theta(t+1)$. The cycle restarts with a new mini-batch The cycle is only as fast as the network: $\text{Iteration time} = t(\text{compute}) + t(\text{push}) + t(\text{update}) + t(\text{pull})$ If $t(\text{push}) + t(\text{pull}) \gg t(\text{compute})$, workers spend more time waiting than calculating. This is why communication dominates in large clusters Despite the overhead, this cycle allows a distributed system to behave like one giant computer: Each worker sees different data $\rightarrow$ broader gradient estimate Aggregation produces a more accurate global direction than any single worker alone All workers stay synchronised (in sync mode) $\rightarrow$ consistent model across cluster The four-step cycle describes what happens. The next decision is when: Synchronous - wait for all workers before Step 3 Asynchronous - update immediately when any worker pushes Local SGD - skip Steps 2-4 for many local iterations

Must remember:

- Four steps: local calculation $\rightarrow$ push $\rightarrow$ global update $\rightarrow$ pull
- Local gradient = improvement direction from worker's mini-batch only
- Push sends gradients to central/collective point; pull broadcasts updated weights
- Global gradient = average of all local gradients; then standard SGD update applied
- Cycle speed bounded by network (push + pull), not just GPU compute
- Parameter server and AllReduce are two architectures implementing the same cycle
- Timing of Steps 2-4 (sync/async/local) is the next design decision

Formula / decision rule: $\nabla L_i; \nabla L(\text{global}) = (1/W) \sum_{i=1}^W \nabla L_i$

Exam trap: Skipping the pull step in analysis - workers with stale weights compute wrong local gradients Assuming push and pull are instant - they are the primary bottleneck in multi-node training

BD-W11-T08 Synchronous vs Asynchronous vs Local SGD Aggregation Strategies

Tier A

Definition and why it matters: The four-step gradient cycle (compute $\rightarrow$ push $\rightarrow$ update $\rightarrow$ pull) works identically in structure. The critical design choice is how strictly to enforce timing - when to perform the global update relative to worker completion

Core explanation: Not every worker finishes at the same microsecond. The aggregation strategy determines whether the system waits or proceeds Concept flow: How to handle timing differences? - Synchronous: wait for all - Asynchronous: update immediately - Local SGD: sync periodically Pro: Mathematical purity - complete picture of data; Con: Straggler problem - slowest worker delays all Pro: Guaranteed convergence (true SGD at scale); Con: Expensive idle time on fast workers Pro: All workers use same model version; Con: Cluster speed = slowest node Pro: Incredible speed and throughput; Con: Gradient staleness Pro: GPUs never sit idle; Con: Convergence issues and instability Pro: ~95%+ worker utilisation; Con: Slow worker may submit gradient 10 steps old Rule: Wait for every worker to finish local calculation and push its gradient before performing the global update Pro: High steps per second; Con: May fail to reach optimal loss Best for: Stable high-speed networks, homogeneous hardware, models where convergence quality is paramount Pro: Drastically reduced communication cost; Con: Model drift between sync points Rule: Do not wait. As soon as any worker finishes, it pushes its gradient. The global model updates immediately. That worker pulls new weights and continues Pro: Great for slow networks; Con: Workers diverge in different directions Pro: Balances speed and stability; Con: Harder to reconcile at sync time Gradient staleness: A slow worker's gradient is computed on an outdated model version. Applying stale gradients can cause numerical instability or prevent convergence Best for: Unreliable nodes, varying hardware speeds, when raw throughput matters more than peak accuracy Rule: Workers perform many local training steps independently before synchronising with the cluster Model drift: Because workers train locally for extended periods without communicating, their local models start to diverge. Reconciling them into one accurate global model at sync time is challenging Best for: Bandwidth-constrained environments, geo-distributed training, federated learning scenarios Concept flow: Choose aggregation strategy - Stable, fast network? - Homogeneous hardware? - Synchronous - Sync with backup workers or Local SGD - Accuracy critical? - Local SGD with periodic sync - Asynchronous

Must remember:

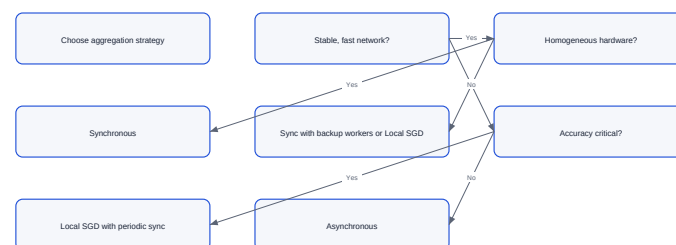
- Three strategies: synchronous (wait for all), asynchronous (update immediately), local SGD (periodic sync)
- Synchronous: best convergence, straggler-sensitive, ~70% efficiency
- Asynchronous: highest throughput, stale gradient risk, ~95% efficiency
- Local SGD: reduces communication, risks model drift between syncs
- Aggregation = trade-off between speed and stability
- Choose based on network stability, hardware uniformity, and accuracy requirements
- No single best strategy - match to infrastructure constraints

Formula / decision rule: eta

Example or contrast: Strategy: Synchronous; Waits for all?: Yes; Communication frequency: Every step; Convergence: Best; Speed: ~70% efficiency; Staleness risk: None Strategy: Asynchronous; Waits for all?: No; Communication frequency: Every step (per worker); Convergence: Good with tuning; Speed: ~95% efficiency; Staleness risk: High Strategy: Local SGD; Waits for all?: Periodically; Communication frequency: Every K steps; Convergence: Moderate; Speed: High (low comm); Staleness risk: Drift, not staleness

Exam trap: Claiming async is always better because it is faster - convergence quality may suffer significantly Using synchronous on heterogeneous/unreliable clusters without mitigation - stragglers destroy efficiency

Synchronous vs Asynchronous vs Local SGD Aggregation Strategies



Vector reconstruction of the source Mermaid visual

Concept map: Synchronous vs Asynchronous vs Local SGD Aggregation Strategies

BD-W11-T09

Module Summary: Distributed Machine Learning at Scale

Tier C

Definition and why it matters: In large-scale ML clusters, up to 70% of total training time is spent on communication rather than computation. This single statistic reframes how distributed training must be designed

Core explanation: Why it matters: Training time is dominated by data movement - moving gradients, weights, and activations across the network - not by the underlying math. The fastest GPUs in the world become idle if the interconnect is congested

Must remember:

- Up to 70% of training time in large clusters is communication, not computation
- Balanced systems (matched compute + network) outperform fast-GPU/slow-network setups
- Data parallelism: shard data, replicate model - default for most workloads
- Model parallelism: split model across devices - required for billion-parameter LLMs
- Pipeline and tensor parallelism break the single-device memory wall
- Design must account for bandwidth, latency, and synchronisation alongside FLOPs
- Fast GPUs with congested networks spend most time waiting, not computing
- Principles of communication, parallelism, and aggregation guide all distributed ML architecture

Exam trap: Assuming faster GPUs always mean faster training - network congestion can dominate; 70% communication overhead is realistic Using data parallelism for LLM-scale models - model won't fit in one GPU; model parallelism is required

Week 11 Exam Lens**Likely questions**

- Define and explain Distributed Machine Learning: Concepts and Algorithms Overview; include the mechanism and one limitation.
- Compare Distributed Machine Learning: Concepts and Algorithms Overview with Communication Overhead and Data Movement in Distributed ML and state when each is preferred.
- Apply Synchronisation, Stragglers, and Convergence in Distributed Training to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $\theta_{t+1} = \theta_t - \eta * \nabla L(\theta_t)$; ∇L ; workers each processing mini-batch size; Effective batch size = $W \times B$; $N+1$; $L(\theta)$; θ ; ∇L_i

Mistakes to avoid: Assuming 2x hardware = 2x speed - communication overhead often dominates; can even slow training Benchmarking only compute FLOPs when selecting hardware - interconnect speed matters equally Concept flow: Need for speed - Need for accurate gradients - Asynchronous updates - Synchronous updates - Gradient staleness risk - Straggler idle time

Week 12: Distributed ML II: Architectures

Week roadmap: Architecting for Distributed Intelligence → Centralised Learning: The Parameter Server Model → Decentralised Learning: The Ring All-Reduce Strategy → Synchronous Training: Consistency and the Straggler Problem → Asynchronous Training: Speed vs Convergence Quality → MirroredStrategy: Single-Node Multi-GPU Training → 3 more topics.

Coverage: 9 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W12-T01 Architecting for Distributed Intelligence

Tier A

Definition and why it matters: Modern ML models - large language models, massive recommendation engines - involve billions of parameters operating on petabytes of data. No single device, regardless of power, can handle this alone

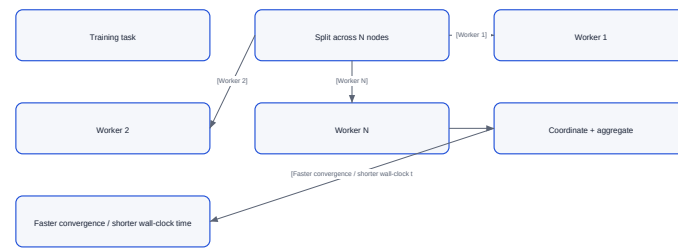
Core explanation: The industry has shifted from single-device intelligence to multi-node intelligence: orchestrating dozens or hundreds of computing resources to work as one cohesive unit Concept flow: Training task - Split across N nodes - Worker 1 - Worker 2 - Worker N - Coordinate + aggregate - Faster convergence / shorter wall-clock time Metric: Wall-clock training time; Single device: Weeks; Well-orchestrated cluster: Hours Metric: Data processed per step; Single device: Limited by one GPU; Well-orchestrated cluster: Scales with worker count Metric: Risk of slowdown; Single device: None; Well-orchestrated cluster: High if communication dominates Pillar: Sync vs async training; Focus: Consistency vs speed trade-offs The core challenge is orchestration: How do you split a training task across many nodes in a way that actually speeds up the process? Done poorly, communication overhead can make distributed training slower than a single machine Pillar: TensorFlow strategies; Focus: MirroredStrategy (single node) and MultiWorker MirroredStrategy (multi-node) The ultimate objective of distributed architecture is to increase throughput and reduce training time - turning processes that once took weeks into tasks completed in hours Pillar: Computational graphs; Focus: Device placement and graph partitioning This module covers the practical architectures and strategies for scaling ML across nodes, centred on TensorFlow's distributed APIs Concept flow: Centralised - Decentralised - Synchronous - Asynchronous Distributed ML clusters can be structured along two independent axes: Scenario: Petabyte-scale training data; Distributed benefit: Data parallelism shards dataset Axis 1 - Where weights live: Scenario: Billion-parameter models; Distributed benefit: Model parallelism + multi-node Centralised: Parameter server holds global model state Scenario: Iterative deep learning; Distributed benefit: Gradient sync across replicas Decentralised: Workers collectively aggregate via ring all-reduce Scenario: Production SLA on training time; Distributed benefit: Wall-clock reduction from parallelism Axis 2 - When weights update: Synchronous: All workers finish before global update (barrier) Asynchronous: Each worker updates independently as it finishes Real-world example: A recommendation system with sparse billion-parameter embedding tables often uses a centralised parameter server with asynchronous updates. A computer vision model on a homogeneous GPU cluster typically uses decentralised ring all-reduce with synchronous training When it does NOT pay off: Small datasets, models that fit comfortably on one GPU, or clusters with severely mismatched hardware and poor interconnects

Must remember:

- Modern AI requires multi-node intelligence - single devices cannot handle billion-parameter, petabyte-scale workloads
- Orchestration is the core challenge: split work without communication dominating
- Goal is throughput - reduce wall-clock training from weeks to hours
- Four pillars: centralised/decentralised, sync/async, TF strategies, graph partitioning
- Poor orchestration can make distributed training slower than single-machine
- Architecture choice depends on model sparsity, cluster topology, and consistency requirements
- Parameter server = centralised; ring all-reduce = decentralised
- TensorFlow provides strategy APIs to implement these patterns with minimal code changes

Exam trap: Assuming more nodes always means faster training - communication overhead can make it slower than single-machine training Conflating data parallelism with model parallelism - data parallelism splits the dataset; model parallelism splits the model

Architecting for Distributed Intelligence



Vector reconstruction of the source Memmalt visual

Concept map: Architecting for Distributed Intelligence

BD-W12-T02

Centralised Learning: The Parameter Server Model

Tier A

Definition and why it matters: In centralised learning (parameter server architecture), computing resources are divided into two distinct roles:

Core explanation: Role: Parameter server; Function: Stores current model weights/parameters; Analogy: Brain / central repository Role: Workers; Function: Process data, compute gradients; Analogy: Muscles / compute engines Concept flow: PS - W - e.g. SGD Step: 1. Pull; Action: Worker fetches latest parameters from server Step: 2. Compute; Action: Worker runs forward/backward pass on a data batch Step: 3. Push; Action: Worker sends gradients back to parameter server Step: 4. Update; Action: Server applies optimisation algorithm (SGD, Adam, etc.) to update global weights The parameter server is the single source of truth for model state. Workers are stateless compute units that pull weights, train locally, and push gradients back Model characteristic: Sparse updates (few params touched per step); Parameter server fit: Excellent Model characteristic: Massive embedding tables; Parameter server fit: Excellent Model characteristic: Dense updates (all params every step); Parameter server fit: Poor - bottleneck risk Model characteristic: Few workers (< 10); Parameter server fit: Good The parameter server model excels for large, sparse models with billions of parameters where only a small fraction of parameters are updated in any given step Model characteristic: Many workers (> 50); Parameter server fit: Risk of server bottleneck Classic use case: large-scale recommendation systems Massive embedding tables (billions of entries) Concept flow: Worker 1 - Parameter Server - Worker 2 - Worker 3 - Worker N - Training slows down Each training step touches only a tiny subset of embeddings A dedicated server efficiently manages sparse state updates As worker count scales up, the central parameter server becomes a communication bottleneck Why it happens: All workers push gradients to one server simultaneously Server must receive, aggregate, and redistribute weights for every step Network bandwidth to a single node is finite Shard parameters across multiple parameter servers Use asynchronous updates to reduce barrier pressure Switch to decentralised aggregation (ring all-reduce) for dense models

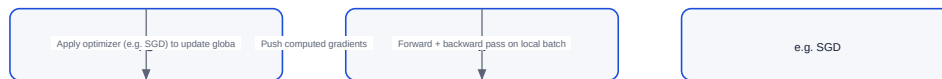
Must remember:

- Parameter server = centralised architecture with dedicated weight storage
- Workers pull parameters, compute gradients, push gradients back
- Server applies optimizer to update global weights
- Ideal for large sparse models - recommendation systems with massive embedding tables
- Only a small fraction of parameters updated per step - server manages sparse state efficiently
- Bottleneck risk when too many workers talk to one server simultaneously
- Decentralised alternative (ring all-reduce) eliminates central bottleneck for dense models
- Architecture choice depends on model sparsity, not just dataset size

Example or contrast: Aspect: Architecture; Parameter server: Centralised; Ring all-reduce: Decentralised Aspect: Weight storage; Parameter server: Dedicated server(s); Ring all-reduce: No central store Aspect: Best for; Parameter server: Sparse models, embeddings; Ring all-reduce: Dense neural networks

Exam trap: Using parameter servers for dense CNN/Transformer models - all parameters update every step; ring all-reduce is more efficient Ignoring the bottleneck at scale - parameter servers work well with few workers but degrade with dozens pushing simultaneously

Centralised Learning: The Parameter Server Model



Vector reconstruction of the source Mermaid visual

Concept map: Centralised Learning: The Parameter Server Model

BD-W12-T03

Decentralised Learning: The Ring All-Reduce Strategy

Tier A

Definition and why it matters: The parameter server model suffers a central bottleneck as worker count grows - all gradients flow through one server. Ring all-reduce solves this by eliminating the central point entirely

Core explanation: In ring all-reduce, every worker is an equal participant in a logical ring. No dedicated server stores weights
Concept flow: Worker 1 - Worker 2 - Worker 3 - Worker 4
Pass: Pass 1 (reduce-scatter); What happens: Gradients are partially aggregated and scattered
Pass: Pass 2 (all-gather); What happens: Fully aggregated chunks are gathered at every node
Instead of every worker sending all data to one central point, each node communicates only with its immediate neighbours (predecessor and successor in the ring)
Property: Data sent per node; Parameter server: $O(N)$ - all-to-one; Ring all-reduce: $O(1)$ - neighbour only
Per-step logic at each node: Receive a chunk of gradients from predecessor
Property: Scales with workers; Parameter server: Degrades; Ring all-reduce: Maintains efficiency
Add locally computed gradients to that chunk
Pass the accumulated result to successor
After two complete passes around the ring, every node holds the exact same fully aggregated gradient vector
Use case: CNN image classification; Ring all-reduce fit: Excellent
Use case: Transformer language models; Ring all-reduce fit: Excellent
The amount of data each node sends and receives is independent of the total number of nodes in the cluster
Use case: Multi-GPU clusters with fast interconnects; Ring all-reduce fit: Excellent
Use case: Sparse recommendation embeddings; Ring all-reduce fit: Poor - use parameter server
Use case: Heterogeneous slow networks; Ring all-reduce fit: Challenging - sync barriers hurt
Concept flow: GPU 1: full gradient - Ring all-reduce - GPU 2: full gradient - GPU 3: full gradient - Identical aggregated gradient on all GPUs
This is why ring all-reduce is called bandwidth optimal - it fully utilises high-speed interconnects (NVLink, InfiniBand) without overloading any single node
Real-world example: NVIDIA's NCCL library implements ring all-reduce for multi-GPU training, achieving near-linear scaling on 8-GPU DGX systems
Ring all-reduce is highly efficient for dense models - deep neural networks in computer vision and NLP where all parameters are updated every step

Must remember:

- Ring all-reduce eliminates the central parameter server bottleneck
- Each worker communicates only with immediate ring neighbours
- Two passes around the ring produce identical aggregated gradients on all nodes
- Bandwidth optimal - per-node communication independent of cluster size
- Best for dense models (CV, NLP) with full parameter updates every step
- Exploits high-speed GPU interconnects (NVLink, InfiniBand) at maximum efficiency
- Parameter server for sparse models; ring all-reduce for dense models
- Implemented in NCCL, Horovod, PyTorch DDP, TensorFlow MultiWorkerMirroredStrategy

Formula / decision rule: $O(N)$; $O(1)$

Example or contrast: Dimension: Weight storage; Parameter server: Central server; Ring all-reduce: Distributed across workers
 Dimension: Communication pattern; Parameter server: Star (hub-and-spoke); Ring all-reduce: Ring (neighbour-to-neighbour)
 Dimension: Bottleneck; Parameter server: Central server at scale; Ring all-reduce: None (bandwidth optimal)

Exam trap: Claiming ring all-reduce has a central server - there is no dedicated weight store; every worker is equal
 Using ring all-reduce for sparse embedding models - parameter server is more efficient when only a fraction of parameters update per step

BD-W12-T04

Synchronous Training: Consistency and the Straggler Problem

Tier A

Definition and why it matters: Distributed training architecture involves two separate decisions:

Core explanation: Where weights live - centralised (parameter server) vs decentralised (ring all-reduce)
Concept flow: W1 - Agg - W2 - late - W3
Step: 1; Behaviour: All workers compute local gradients in parallel
Step: 2; Behaviour: Barrier - fastest workers wait for slowest
Step: 3; Behaviour: Gradients aggregated (averaged or summed)

Step: 4; Behaviour: Global weight update applied to all workers When weights update - synchronous vs asynchronous
 Step: 5; Behaviour: All workers start next batch with identical weights This note focuses on synchronous training: the consistency-first approach In synchronous training, the entire cluster operates like a well-drilled marching band. The key rule is a barrier (wait-all): Every worker must finish computing its batch and calculating gradients before model weights can be updated Property: Weight version per step; Synchronous: Identical across all workers; Single-machine equivalent: One global model Property: Gradient aggregation; Synchronous: Mean/sum of all local gradients; Single-machine equivalent: Same Property: Convergence behaviour; Synchronous: Stable, predictable; Single-machine equivalent: Baseline reference Property: Effective batch size; Synchronous: $W \times B$ (workers $\times$ local batch); Single-machine equivalent: B Concept flow: GPU 1 waiting - GPU 2 waiting - GPU 9 waiting - GPU 10 - slow batch - Fast Because every worker waits for the update before starting the next batch, every worker always uses the exact same model version for every training step This makes synchronous training mathematically identical to training on a single machine with a larger effective batch size - just faster due to parallelism The major downside of synchronous training: the cluster is only as fast as its slowest member Metric: Worker utilisation; Synchronous: $\sim 70\%$ (straggler overhead); Asynchronous: $\sim 95\%+$ What causes stragglers: Metric: Steps per second; Synchronous: Lower; Asynchronous: Higher Minor hardware differences between nodes Metric: Final accuracy; Synchronous: Higher (e.g. 98%); Asynchronous: Lower or slower to converge Network latency spikes on one link Metric: Convergence stability; Synchronous: High; Asynchronous: Lower (stale gradients) Background OS tasks on one machine Metric: Mathematical equivalence to single machine; Synchronous: Yes; Asynchronous: No Uneven data shard sizes (some batches take longer) Thermal throttling on one GPU Cost: 9 fast GPUs sit idle, wasting expensive compute while waiting for one slow worker Despite the straggler risk, synchronous training is the most common choice for deep learning because it provides more stable convergence Real-world example: Google Brain and most production deep learning pipelines default to synchronous training on homogeneous GPU clusters with InfiniBand interconnects, where stragglers are rare

Must remember:

- Synchronous training uses a barrier: all workers must finish before global update
- Guarantees identical model version on every worker at every step
- Mathematically equivalent to single-machine training with larger effective batch
- Straggler problem: cluster speed = slowest worker; fast GPUs idle waiting
- Stragglers caused by hardware differences, network spikes, uneven shards
- Most common choice for deep learning despite straggler risk - stable convergence
- Sync achieves higher final accuracy; async achieves higher steps/second
- Best on homogeneous clusters with stable, high-speed networks

Formula / decision rule: $W \times B$

Exam trap: Claiming synchronous training is always faster - stragglers can reduce utilisation to $\sim 70\%$; async reaches $\sim 95\%$ Ignoring that sync = mathematically equivalent to single-machine training - this is a key exam point

BD-W12-T05 Asynchronous Training: Speed vs Convergence Quality

Tier A

Definition and why it matters: Synchronous training wastes compute when stragglers appear - fast GPUs sit idle at the barrier. Asynchronous training removes the wait-all rule entirely

Core explanation: Every worker updates global weights independently as soon as it finishes its batch. No waiting for peers Concept flow: step 1 - W_1 - PS - W_2 Behaviour: Wait for peers; Synchronous: Yes (barrier); Asynchronous: No Behaviour: Update timing; Synchronous: All at once; Asynchronous: Each worker independently Behaviour: GPU utilisation; Synchronous: $\sim 70\%$ (straggler waste); Asynchronous: $\sim 95\%+$ Behaviour: Weight version consistency; Synchronous: Guaranteed identical; Asynchronous: May differ across workers If one worker is faster, it simply keeps going - pulling latest weights, pushing new gradients, never pausing Because workers are not synchronised, a worker may compute gradients using an older version of the weights while other workers have already updated the master model This is the stale gradient problem - and it is the fundamental trade-off of async training Why stale gradients hurt: Gradient was computed for weight version w_t , but global model is already at $w(t+k)$ Applying stale gradient moves model in a direction optimised for outdated parameters Introduces noise into the optimisation trajectory Can cause instability or require more careful learning rate tuning Staleness = number of global updates since worker pulled weights Cluster condition: Homogeneous GPUs, fast InfiniBand; Recommended mode: Synchronous Cluster condition: Stable, high-speed network; Recommended mode: Synchronous Cluster condition: Unreliable nodes or varying hardware; Recommended mode: Asynchronous Cluster condition: Mixed GPU generations in cluster; Recommended mode: Asynchronous Cluster condition: Sparse models with parameter server; Recommended mode: Asynchronous (common) Cluster condition: Accuracy-critical production model; Recommended mode: Synchronous Cluster condition: Time-constrained experimentation; Recommended mode: Asynchronous Real-world example: Large-scale ad recommendation systems at Meta and Google often use asynchronous parameter server training because embedding tables are sparse and worker speeds vary across data centre racks TensorFlow's `tf.distribute` strategy API abstracts these choices: `MirroredStrategy` / `Multi`

WorkerMirroredStrategy → synchronous Parameter server strategy with async updates → asynchronous The choice is often a single configuration flag, but the underlying trade-offs remain

Must remember:

- Asynchronous training removes the barrier - workers update independently
- Achieves ~95%+ worker utilisation vs ~70% for synchronous
- Stale gradient problem: workers compute on outdated weight versions
- Stale gradients cause instability and require careful learning rate tuning
- Async wins on steps/second; sync wins on final accuracy
- Choose async for unreliable/heterogeneous clusters; sync for stable/homogeneous clusters
- Common with sparse parameter server architectures (recommendation systems)
- TensorFlow strategy API implements both modes with minimal code changes

Formula / decision rule: w_t ; w_{t+k}

Example or contrast: Metric: Steps per second; Synchronous: ~70% efficiency; Asynchronous: ~95%+ efficiency
Metric: Worker utilisation; Synchronous: Reduced by stragglers; Asynchronous: Near 100%
Metric: Final accuracy; Synchronous: Higher (~98% in benchmarks); Asynchronous: Lower or slower convergence

Exam trap: Claiming async always produces better models - it is faster per step but often achieves lower final accuracy Ignoring stale gradients - this is the defining problem of async training, not an edge case

BD-W12-T06

MirroredStrategy: Single-Node Multi-GPU Training

Tier B

Definition and why it matters: Up to this point, distributed training has been discussed at the architectural level. TensorFlow's tf.distribute Strategy API implements these concepts in production-ready code

Core explanation: The most common entry point for scaling is MirroredStrategy - designed specifically for single-node, multi-GPU setups Concept flow: GPU 0: mirrored variables + shard 0 - GPU 1: mirrored variables + shard 1 - GPU 2: mirrored variables + shard 2 - GPU 3: mirrored variables + shard 3 - Global batch - NCCL all-reduce - All replicas updated identically Step: 1. Data split; What happens: Each GPU receives a different portion of the batch Step: 2. Forward pass; What happens: Each GPU computes forward pass on its shard Step: 3. Backward pass; What happens: Each GPU computes local gradients Step: 4. Aggregation; What happens: TensorFlow all-reduces gradients across all GPUs (NCCL) Step: 5. Update; What happens: Aggregated gradient applied to all mirrored variables simultaneously Step: 6. Sync; What happens: Model stays perfectly in sync across every device Imagine a single powerful workstation or cloud instance with 2, 4, or 8 NVIDIA GPUs. MirroredStrategy uses all of them synchronously for training The name "mirrored" comes from how TensorFlow handles model variables: every GPU (replica) gets its own identical copy of all model variables - literally mirrored across devices Advantage: NCCL communication; Detail: Uses NVIDIA's high-speed on-device protocol Advantage: No network overhead; Detail: GPUs communicate over PCIe/NVLink, not Ethernet Advantage: Minimal code changes; Detail: Wrap existing model in strategy scope Advantage: Near-linear speedup; Detail: 4 GPUs ≈ 4x throughput on homogeneous hardware Advantage: Synchronous by default; Detail: Stable convergence guaranteed

Must remember:

- MirroredStrategy = TensorFlow's single-node multi-GPU synchronous data parallelism
- Every GPU gets mirrored (identical) copies of all model variables
- Each GPU processes a different data shard per batch
- Gradients aggregated via NCCL (NVIDIA's high-speed on-device protocol)
- Gold standard for 2-8 GPU workstations - minimal code changes, significant speedup
- Synchronous by default - model stays perfectly in sync across all GPUs
- Scale beyond one machine with MultiWorkerMirroredStrategy + TFCONFIG
- Near-linear speedup on homogeneous single-node GPU setups

Example or contrast: Aspect: Scope; MirroredStrategy: Single machine; MultiWorkerMirroredStrategy: Multiple machines (cluster) Aspect: Communication; MirroredStrategy: NCCL (on-device); MultiWorkerMirroredStrategy: NCCL + network (gRPC/MPI)

Exam trap: Using MirroredStrategy for multi-node clusters - it is single-node only; use MultiWorkerMirroredStrategy for clusters Assuming MirroredStrategy splits the model - it mirrors (replicates) the model; only data is split

BD-W12-T07

MultiWorkerMirroredStrategy: Multi-Node Cluster Training

Tier B

Definition and why it matters: MirroredStrategy handles up to ~8 GPUs on a single machine. When models or datasets demand more, training must scale out to a cluster of multiple worker machines

Core explanation: MultiWorkerMirroredStrategy is the big-brother extension of MirroredStrategy - synchronous data parallelism across an entire network cluster Concept flow: GPU 0 - GPU 1 - GPU 2 - GPU 3 - GPU 0 - GPU 1 - GPU 2

- GPU 3 - GPU 0 Property: Scope; Detail: Multiple machines, each with multiple GPUs Property: Core logic; Detail: Same as MirroredStrategy - synchronous, collective ops Property: Communication; Detail: Multi-node all-reduce over network (Ethernet, InfiniBand) Property: Key difference from MirroredStrategy; Detail: Data travels over network cables, not motherboard bus Field: cluster.worker; Purpose: List of all worker network addresses Field: task.type; Purpose: Role of this machine (worker or chief) Field: task.index; Purpose: This machine's index in the cluster (0, 1, 2, ...) Task: Data partitioning; Automated by strategy: Splits dataset across all workers Unlike single-machine setups where TensorFlow auto-detects GPUs, each machine in a cluster must know its role. This is configured via the TFCONFIG environment variable Task: Gradient synchronisation; Automated by strategy: Multi-node all-reduce across network TFCONFIG is a JSON string defining the cluster topology: Task: Variable mirroring; Automated by strategy: Identical model copies on every GPU of every worker Task: Chief worker coordination; Automated by strategy: One worker coordinates checkpointing Concept flow: W0 - W1 - W2 Scale: Small cluster; Example: 3 workers x 4 GPUs = 12 GPUs Critical: TFCONFIG must be set identically on every machine except for task.index, which is unique per machine Scale: Medium cluster; Example: 16 workers x 8 GPUs = 128 GPUs

Must remember:

- MultiWorkerMirroredStrategy scales synchronous data parallelism across multiple machines
- Each worker can have multiple GPUs - strategy handles both intra- and inter-node sync
- Uses multi-node all-reduce over network (not just on-device NCCL)
- TFCONFIG JSON env var defines cluster topology and each machine's role/index
- Must be set on every machine with unique task.index
- Automates data partitioning and gradient synchronisation across the cluster
- Industry-scale solution for dozens to hundreds of GPUs
- Network quality (InfiniBand vs Ethernet) directly impacts training efficiency

Exam trap: Forgetting to set TFCONFIG on every machine - without it, workers don't know cluster topology Using the same task.index on multiple machines - each worker needs a unique index

BD-W12-T08

Understanding Distributed Computational Graphs and Device Placement

Tier A

Definition and why it matters: High-level strategies like MirroredStrategy and MultiWorkerMirroredStrategy abstract away enormous complexity. At the lowest level, TensorFlow manages computational graphs and device placement to make distributed training work

Core explanation: Understanding these mechanics is essential for debugging performance bottlenecks in production clusters Concept flow: Input data - Conv2D on GPU 0 - MaxPool on GPU 0 - Dense on GPU 1 - Loss on CPU Concept: Main graph; Definition: Full model computation as one DAG Concept: Subgraph; Definition: Portion of the graph assigned to one device Concept: Partition boundary; Definition: Point where data must cross device boundaries When a model executes in TensorFlow, operations are represented as a directed acyclic graph (DAG) - nodes are operations (matrix multiply, activation, loss), edges are data tensors flowing between them In a distributed setting, this single main graph must be managed across many hardware units - CPUs and GPUs on multiple machines TensorFlow's runtime is responsible for splitting the main graph into subgraphs, where each subgraph contains operations assigned to a specific device Placement mode: Automatic; Behaviour: TensorFlow runtime decides based on op type and memory Placement mode: Manual (tf.device); Behaviour: Developer forces specific ops to specific devices Different operations have different hardware affinities (GPU for matmul, CPU for data preprocessing) Model parallelism requires layers on different GPUs Memory constraints force splitting across devices Device placement is the process of assigning specific operations to specific hardware TensorFlow device naming convention: Concept flow: Conv2D - Send node - Receive node - Dense layer Communication path: Same machine (GPU GPU); Protocol: Shared memory, NCCL Communication path: Same machine (GPU CPU); Protocol: PCIe DMA Example use case for manual placement: Communication path: Different workers (cluster); Protocol: gRPC or MPI over network Force data preprocessing (I/O, decoding) to CPU Force matrix multiplication and convolutions to GPU Prevents GPU idle time waiting for CPU-bound preprocessing Symptom: GPU utilisation low; Likely cause: Receive node waiting for slow network transfer Symptom: CPU at 100%, GPU idle; Likely cause: Data preprocessing not offloaded or bottlenecked on CPU Because the graph is partitioned across devices, data must travel between subgraphs. TensorFlow handles this automatically by inserting send and receive nodes into the partitioned graphs Symptom: Uneven GPU usage; Likely cause: Poor automatic placement - manual placement needed Symptom: High latency between steps; Likely cause: Send/receive nodes blocked on slow interconnect This ensures that even though the model is physically split across many machines, it behaves as one logically connected system Understanding graph partitioning and device placement helps diagnose why GPUs are idle:

Must remember:

- TensorFlow represents models as a DAG of operations and tensors
- Graph partitioning splits the main graph into per-device subgraphs

- Device placement assigns operations to specific CPUs/GPUs
- TensorFlow auto-inserts send/receive nodes at partition boundaries
- Same-machine communication uses shared memory/NCCL; cross-worker uses gRPC/MPI
- Manual `tf.device` placement optimises CPU/GPU workload distribution
- Debugging tip: idle GPUs often mean receive nodes blocked on slow transfers
- Understanding low-level mechanics is key to diagnosing distributed training bottlenecks

Exam trap: Calling the graph a "directed cyclic graph" - it is a directed acyclic graph (DAG); cycles would prevent execution ordering Assuming TensorFlow never needs manual device placement - automatic placement can be suboptimal; manual override is common in production

BD-W12-T09 Module Summary: Distributed ML Architectures in TensorFlow

Tier C

Definition and why it matters: Strategy: MirroredStrategy; Scope: Single node, multi-GPU; Communication: NCCL (on-device); Configuration: Automatic

Core explanation: Strategy: MultiWorkerMirroredStrategy; Scope: Multi-node cluster; Communication: All-reduce over network; Configuration: TFCONFIG required

Must remember:

- Parameter server for sparse models; ring all-reduce for dense models
- Synchronous = consistency + straggler risk; asynchronous = speed + stale gradients
- MirroredStrategy: single-node multi-GPU via NCCL - minimal setup
- MultiWorkerMirroredStrategy: multi-node cluster - requires TFCONFIG
- Graph partitioning and device placement are the low-level mechanics under strategies
- Send/receive nodes handle inter-device communication automatically
- Scaling across nodes is a requirement, not an option, for modern AI
- Choose architecture based on model sparsity, cluster homogeneity, and network quality

Example or contrast: Key insight: Parameter servers excel when only a small fraction of billions of parameters update per step. Ring all-reduce eliminates the central bottleneck for dense models with full parameter updates Architecture: Parameter server (centralised); Weight storage: Dedicated server(s); Best for: Sparse models, recommendation embeddings; Bottleneck risk: Yes - at scale

Exam trap: Using parameter servers for dense CNN/Transformer training - ring all-reduce is bandwidth optimal for dense models Forgetting TFCONFIG for multi-node training - MultiWorkerMirroredStrategy requires cluster topology on every machine

Week 12 Exam Lens

Likely questions

- Define and explain Architecting for Distributed Intelligence; include the mechanism and one limitation.
- Compare Architecting for Distributed Intelligence with Centralised Learning: The Parameter Server Model and state when each is preferred.
- Apply Decentralised Learning: The Ring All-Reduce Strategy to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $O(N)$; $O(1)$; $W \times B$; w_t ; w_{t+k}

Mistakes to avoid: Assuming more nodes always means faster training - communication overhead can make it slower than single-machine training Using parameter servers for dense CNN/Transformer models - all parameters update every step; ring all-reduce is more efficient Claiming ring all-reduce has a central server - there is no dedicated weight store; every worker is equal

Week 13: Stream Processing Systems

Week roadmap: The Shift to Real-Time Intelligence → Processing for Velocity: What Is a Stream? → Technical Differences: Windowing vs Global State → Financial Services: Real-Time Fraud Detection → E-Commerce: Dynamic Pricing and Personalization → Kafka as the Streaming Backbone → 2 more topics.

Coverage: 8 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

BD-W13-T01 The Shift to Real-Time Intelligence

Tier B

Definition and why it matters: Data engineering was historically built around historical analysis: collect data all day, store it in a warehouse, run a batch job at night to see what happened

Core explanation: The modern digital landscape demands something more immediate - architecting real-time data intelligence at scale Concept flow: Collect all day - Store in warehouse - Nightly batch job - What happened yesterday? - Continuous events - Process in real time - What is happening now? Question era: Business insight; Batch answer: What happened yesterday?; Stream answer: What is happening right now? Question era: Response time; Batch answer: Hours to days; Stream answer: Milliseconds to seconds Question era: Action type; Batch answer: Retrospective analysis; Stream answer: Immediate intervention The focus has moved from how much data we have to how fast we can act on it. Acting on data in milliseconds or microseconds is now the competitive benchmark Objective: Contrast batch vs stream; Detail: Latency and data velocity differences The questions we ask have shifted: Objective: Identify streaming use cases; Detail: Financial services, e-commerce Old: "What happened yesterday?" Objective: Evaluate processing guarantees; Detail: Exactly-once vs at-least-once reliability New: "What is happening right now?" Objective: Architect streaming solutions; Detail: Kafka, Storm, and related tools This enables immediate intervention and monitoring - catching problems before they escalate In a world of instant gratification, responding immediately to market fluctuations or user triggers provides a massive edge: Role: Data engineer; Batch-era skill: ETL pipelines, nightly jobs; Stream-era skill: Real-time ingestion, windowing Sudden stock price drops → automated trading response

Must remember:

- Data engineering is shifting from periodic batch to continuous real-time processing
- Velocity is the new volume - speed of action matters more than data size
- Questions changed from "what happened yesterday?" to "what is happening now?"
- Three drivers: velocity, operational efficiency, competitive advantage
- Streaming enables immediate intervention - fraud blocking, dynamic pricing, live monitoring
- Module covers: batch vs stream contrast, use cases, processing guarantees, Kafka/Storm architecture
- Batch processing remains important; streaming is a complementary paradigm, not a replacement
- Real-time intelligence is a critical requirement for modern data engineering roles

Exam trap: Treating streaming as just "faster batch" - streams are architecturally different (unbounded data, state management, messaging systems) Assuming batch processing is obsolete - batch remains essential for historical analysis; streaming complements it

BD-W13-T02 Processing for Velocity: What Is a Stream?

Tier A

Definition and why it matters: A stream is a continuous, unbounded sequence of events that must be processed as they arrive - not after collection is complete. Stream processing is architecturally distinct from batch processing in three fundamental ways

Core explanation: In batch systems, waiting hours for results is acceptable. In stream systems, results must be generated in milliseconds or seconds Property: Boundaries; Batch data: Has a start and end; Stream data: No end - continuous flow Property: Size; Batch data: Finite, known upfront; Stream data: Infinite, ever-growing Property: Sources; Batch data: Files, databases; Stream data: IoT sensors, web logs, clickstreams Property: Processing model; Batch data: Process entire dataset; Stream data: Process event-by-event This enables immediate automated decisions - authorising a credit card transaction before the user puts their phone away This is the most important conceptual shift: A stream is a continuous flow of events from sources like IoT sensors or web logs that, for all practical purposes, never ends Streams are not designed for storage alone - they are built to be acted upon the moment an event arrives: Real-time server health monitoring → automatic alert E-commerce clickstream → instant product recommendation Transaction event → fraud score and block/approve decision State type: Per-event; Example: Fraud score for one transaction; Challenge: Low - stateless State type: Per-window; Example: Average price in last 5 minutes; Challenge: Medium - windowed state State type: Global; Example: Cumulative sales since year start; Challenge: High - ever-growing state State type: Per-user; Example: Shopping cart contents; Challenge: High - keyed state per user In batch processing, complexity comes from the high volume of data. In streaming, the challenge shifts to high velocity and maintaining dynamic state Maintaining state - remembering what happened a few seconds ago while new data is flying in - is one of the biggest challenges in stream engineering If a stream is infinite, how do you compute aggregates like sums or

averages? You cannot wait for the "end" of the data Solution: windowing - temporal segmentation that groups events into finite chunks for computation. This is covered in the next topic

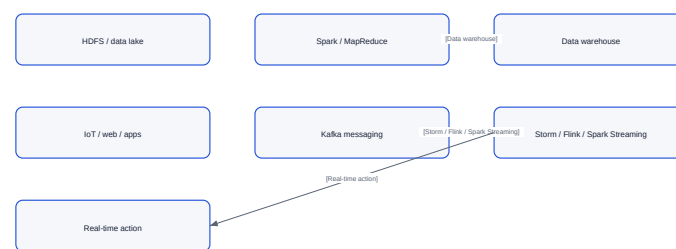
Must remember:

- A stream is an unbounded, continuous flow of events processed as they arrive
- Three pillars: low latency (ms-s), unbounded data (no end), actionability (triggers on arrival)
- Batch = bounded static data, hours/days latency, HDFS; Stream = unbounded events, ms/s latency, Kafka
- Stream complexity comes from velocity and dynamic state, not volume
- State management on infinite data is the core engineering challenge
- Streams require dedicated messaging systems (Kafka), not commodity file systems
- Windowing solves the problem of computing aggregates on infinite streams
- Streams enable immediate decisions: fraud blocking, live monitoring, instant personalisation

Example or contrast: Dimension: Data scope; Batch processing: Bounded, static datasets; Stream processing: Unbounded, continuous events Dimension: Latency; Batch processing: Hours to days; Stream processing: Milliseconds to seconds Dimension: Infrastructure; Batch processing: Commodity clusters, HDFS; Stream processing: Dedicated messaging systems (Kafka)

Exam trap: Calling batch data "unbounded" - batch data is bounded (has start and end); streams are unbounded Assuming streams don't need storage infrastructure - Kafka provides persistent, durable event storage; streams are not ephemeral

Processing for Velocity: What is a Stream?



Vector reconstruction of the source Mernaid visual

Concept map: Processing for Velocity: What Is a Stream?

BD-W13-T03

Technical Differences: Windowing vs Global State

Tier A

Definition and why it matters: Streams are unbounded - they never end. Yet applications need to compute metrics like sums, averages, and counts. You cannot wait for the "end" of the data to aggregate

Core explanation: Two complementary mechanisms solve this: windowing (temporal segmentation) and global state (persistent memory) Property: Size; Value: Fixed (e.g. 5 minutes) Property: Overlap; Value: None Property: Buffer behaviour; Value: Cleared after each window Property: Use case; Value: Hourly sales totals, periodic metrics Windowing places a frame over a specific section of the timeline to group events for processing. Think of it as slicing infinite time into finite, computable chunks Fixed size, non-overlapping Property: Slide interval; Value: Shorter than window size (e.g. 1 minute) Process data from 12:00-12:05, then clear buffers and start fresh for 12:05-12:10 Property: Overlap; Value: Yes - events appear in multiple windows Property: Use case; Value: Smoother, continuously updated metrics Property: Size; Value: Dynamic - depends on activity Property: Trigger; Value: Gap of inactivity (e.g. 30 minutes no events) Property: Use case; Value: User behaviour during a single website visit Example: average price over the last 10 minutes, updated every minute Window stays open as long as data is arriving; closes after a gap of inactivity (session timeout) Concept flow: Event 1 - Global State Store - Event 2 - Event 3 - Running total / user context / model params Use: Running totals; Description: Cumulative sum of all sales since year start; Why windows fail: Window clears data; need persistent accumulator Use: User context; Description: Logged-in status, shopping cart, last 5 clicks; Why windows fail: Must persist across many events per user Use: Online ML updates; Description: Continuously update model parameters as data arrives; Why windows fail: Model state must evolve, not reset per window While windowing looks at chunks of time, global state maintains a persistent in-memory store that grows across the entire lifecycle of the stream Unlike windowing, global state does not clear periodically - it accumulates Real-world example: A recommendation engine needs to know a user's shopping cart contents (global state) AND the average click rate in the last 5 minutes (tumbling window) simultaneously With windowing and global state defined, the next concern is reliability: how do you ensure data is processed correctly when systems fail? This leads to processing guarantees - exactly-once vs at-least-once

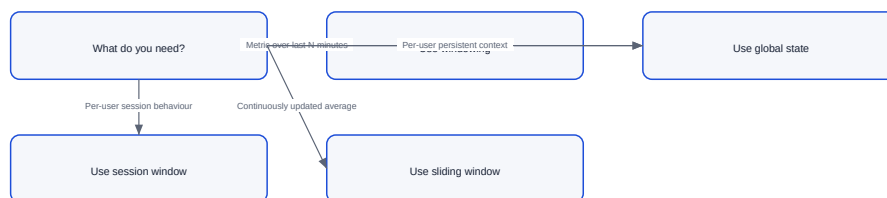
Must remember:

- Windowing solves aggregation on infinite streams by slicing time into finite chunks
- Tumbling: fixed, non-overlapping (e.g. 5-min windows)
- Sliding: fixed, overlapping (e.g. 10-min window updated every 1 min)
- Session: dynamic, closes after inactivity gap
- Global state: persistent in-memory store that never clears - for running totals, user context, online ML
- Use windows for time-bounded metrics; global state for lifetime accumulators and per-user context
- Production systems combine both: windowed metrics + persistent user state
- Next concern: processing guarantees (exactly-once vs at-least-once) for reliability

Example or contrast: Window type: Tumbling; Size: Fixed; Overlap: No; Trigger: Time boundary; Example use case: Hourly transaction count Window type: Sliding; Size: Fixed; Overlap: Yes; Trigger: Time + slide interval; Example use case: 10-min moving average, updated every minute Window type: Session; Size: Dynamic; Overlap: N/A; Trigger: Activity gap; Example use case: Single browsing session analysis

Exam trap: Using tumbling windows for per-user session analysis - session windows are activity-based, not time-fixed
Confusing sliding and tumbling windows - tumbling has no overlap; sliding overlaps by design

Technical Differences: Windowing vs Global State



Vector reconstruction of the source Mermaid visual

Concept map: Technical Differences: Windowing vs Global State

BD-W13-T04 Financial Services: Real-Time Fraud Detection

Tier B

Definition and why it matters: In the batch processing world, a bank might analyse card transactions at the end of the day. The problem: by the time the batch job identifies a fraudulent transaction, the money is gone and the attacker has moved on

Core explanation: Stream processing transforms fraud detection from retrospective analysis to preventive action
Concept flow: Transactions all day - End-of-day batch - Flag fraud - too late - Transaction at swipe - Instant scoring - Block before authorisation
Signal: Unusual location; Risk indicator: Purchase in country never visited
Signal: Amount anomaly; Risk indicator: 10x typical transaction size
Signal: Velocity spike; Risk indicator: 5 transactions in 2 minutes
Signal: Merchant mismatch; Risk indicator: Luxury goods from known grocery-only user
Requirement: Uptime; Target: 99.999%; Why: System failure = transactions cannot be processed
Requirement: Latency; Target: Sub-second (milliseconds); Why: Must not slow checkout for legitimate users
As soon as a card is swiped, the transaction data is streamed into a processing engine. It is immediately scored against the cardholder's historical patterns - maintained in global state (spending locations, typical amounts, merchant categories)
Requirement: Accuracy; Target: High precision + recall; Why: False positives annoy customers; false negatives lose money
If the score indicates high fraud probability - e.g., a large purchase in a country the cardholder has never visited - the system triggers an immediate rejection before the transaction is authorised

Must remember:

- Batch fraud detection is too late - money is gone before end-of-day analysis
- Stream fraud detection scores transactions at the moment of swipe
- Uses global state to maintain per-cardholder spending patterns
- High-risk signals: unusual location, amount anomaly, velocity spike
- Architecture requires 99.999% uptime and sub-second latency
- Must not slow checkout experience for legitimate users
- Fraud prevention is about speed, not retrospective analysis
- Kafka + Storm + in-memory state store is the typical architecture

Example or contrast: Aspect: Detection timing; Batch (end-of-day): Hours after fraud; Stream (real-time): Milliseconds at swipe
Aspect: Money at risk; Batch (end-of-day): Full day's losses; Stream (real-time): Single transaction at most
Aspect: Customer experience; Batch (end-of-day): No interruption; Stream (real-time): Must be invisible to legitimate users

Exam trap: Claiming batch fraud detection is sufficient - money is gone before batch identifies fraud Ignoring latency requirements - fraud check must complete in milliseconds, not seconds, to avoid slowing checkout

BD-W13-T05

E-Commerce: Dynamic Pricing and Personalization

Tier B

Definition and why it matters: While financial services use streaming to prevent loss, e-commerce uses it to actively drive revenue. E-commerce optimisation through real-time streaming delivers an average 15% revenue lift

Core explanation: In modern marketplaces, prices are no longer static - they respond instantly to supply, demand, and competitor data Input stream: High demand in area; Pricing action: Price multiplier increases Input stream: Low demand / excess drivers; Pricing action: Price returns to baseline Input stream: Special events nearby; Pricing action: Predictive surge before demand spike Concept flow: User browsing events - Streaming engine - Historical preferences - global state - Merge current + historical context - Personalised recommendations - Updated every second on site Logistics platforms (Uber, Lyft) use a continuous stream of location data and ride requests to adjust prices in real time based on current demand Source: Current events; Data: Products viewed right now, clicks, cart additions; Storage mechanism: Stream (real-time) Source: Historical context; Data: Past purchases, preferences, browsing history; Storage mechanism: Global state (persistent) Retailers trigger instant price drops when sensor or sales data shows stock isn't moving fast enough in a specific region Real-time coupons issued to a customer while they are still on the site based on immediate browsing behaviour - not yesterday's visit The goal: match user intent while they are still browsing. A recommendation engine that influences purchase decisions cannot rely on what a user liked last week - it must be near-instant

Must remember:

- E-commerce streaming drives revenue (15% average lift), not just loss prevention
- Dynamic pricing: surge pricing, inventory discounts, user intent coupons - all real-time
- Personalisation must be near-instant - based on current browsing, not last week's data
- Combines current events (stream) with historical context (global state)
- Move from generic top-seller lists to individualised per-user responses
- Surge pricing uses continuous location + demand streams (Uber/Lyft model)
- Updated every second as user browses - the "Amazon effect"
- Kafka + streaming engine architecture supports both pricing and personalisation pipelines

Example or contrast: Aspect: Goal; Dynamic pricing: Optimise price point; Real-time personalisation: Optimise product/content shown Aspect: Input; Dynamic pricing: Supply, demand, inventory; Real-time personalisation: User behaviour + history Aspect: Output; Dynamic pricing: Price change; Real-time personalisation: Recommendation, coupon, layout

Exam trap: Using batch recommendations for in-session personalisation - last week's preferences are irrelevant to current browsing intent Ignoring global state for historical context - current events alone are insufficient; past preferences must persist

BD-W13-T06

Kafka as the Streaming Backbone

Tier A

Definition and why it matters: Partitioning for parallelism: Multiple partitions allow multiple consumers to read in parallel - one consumer per partition for maximum throughput Concept: Topic; Definition: Named category/feed of messages (e.g. transactions, clickstream)

Core explanation: In a modern data ecosystem, dozens of data sources (web logs, IoT sensors, databases) need to feed dozens of destinations (processing engines, databases, dashboards). Without a backbone, you end up with a spaghetti of point-to-point connections - unmaintainable at scale Apache Kafka solves this by acting as a persistent, fault-tolerant log of events between producers and consumers Concept flow: Mobile app - IoT sensor - Database CDC - Kafka Cluster - brokers - Spark Streaming - Storm topology - Data warehouse Property: Disk persistence; Benefit: Messages survive broker restarts Property: Replication; Benefit: Copies across brokers for fault tolerance Property: Partitioning; Benefit: Parallel reads and writes at scale Property: Retention; Benefit: Consumers can replay historical events Applications that push data into the system: Mobile app sending clickstream events Thermometer in a shipping container sending temperature readings Database change-data-capture (CDC) streams The heart of the system. Kafka does not just pass messages - it stores them on disk in a scalable cluster of brokers Without Kafka: Producer must know all consumers; With Kafka: Producer writes to topic; unaware of consumers Without Kafka: Consumer must handle producer failures; With Kafka: Consumer reads at its own pace from durable log Without Kafka: Adding a new consumer requires producer changes; With Kafka: New consumer subscribes to existing topic Without Kafka: No replay capability; With Kafka: Consumers can re-read from any offset Because messages are written to disk and replicated, Kafka provides the durability required for production streaming. If a processing engine fails, data remains safe in the Kafka broker Concept flow: Producer - Kafka Topic - Consumer A - real-time - Consumer B - batch replay - Consumer C - new service Systems that read data to do actual work: Spark Streaming job Apache Storm topology The key architectural benefit: Kafka decouples sources from targets This decoupling enables scaling real-time intelligence to

millions of events per second Layer: Ingestion; Component: Producers; Role: Push events into Kafka Layer: Storage; Component: Kafka brokers; Role: Persist and replicate events Layer: Processing; Component: Storm / Flink / Spark Streaming; Role: Transform events into intelligence Layer: Action; Component: Downstream systems; Role: Fraud block, price update, recommendation

Must remember:

- Kafka is the industry-standard streaming backbone - persistent, fault-tolerant event log
- Producers push events; brokers store on disk with replication; consumers read at their own pace
- Decouples sources from targets - no spaghetti connections
- Messages persist (not deleted after consumption) - consumers can replay from any offset
- Scales to millions of events per second via topic partitioning
- Kafka stores data in motion; Storm/Flink processes it
- Production requires replication and careful partitioning strategy
- LinkedIn-scale: trillions of messages/day through Kafka topics

Example or contrast: Aspect: Message lifecycle; Traditional MQ (RabbitMQ): Deleted after consumption; Kafka: Retained for configurable period Aspect: Replay; Traditional MQ (RabbitMQ): Not possible; Kafka: Consumers re-read from any offset Aspect: Throughput; Traditional MQ (RabbitMQ): Thousands/sec; Kafka: Millions/sec

Exam trap: Calling Kafka a message queue - it is an event log (messages persist, consumers can replay); traditional queues delete after consumption Assuming Kafka processes data - Kafka stores and delivers; processing is done by Storm, Flink, or Spark Streaming

BD-W13-T07

Real-Time Topology with Apache Storm

Tier A

Definition and why it matters: Kafka stores and delivers events reliably. But to drive business value, raw events must be transformed into intelligence - fraud scores, price updates, recommendations. This requires a stream processing engine

Core explanation: Apache Storm is a distributed real-time computation system designed for ultra-low-latency processing of unbounded data streams Concept flow: Kafka topic - Spout - read events - Bolt 1 - filter/enrich - Bolt 2 - aggregate - Bolt 3 - score/output - Fraud block / price update Responsibility: Data ingestion; Detail: Read from Kafka, message queue, or custom source Responsibility: Tuple emission; Detail: Convert raw events into Storm tuples Responsibility: Acknowledgement; Detail: Track which tuples have been fully processed The sources of the stream within a Storm topology. A spout typically reads directly from Kafka topics and emits tuples into the topology Bolt type: Filter bolt; Example: Drop events with null fields Bolt type: Aggregation bolt; Example: Compute 5-minute tumbling window average Bolt type: Enrichment bolt; Example: Join transaction with user profile from DB Bolt type: Scoring bolt; Example: Apply ML model for fraud probability Where the actual processing happens. A bolt can: Filter data (drop irrelevant events) Aggregate data (compute windowed sums) DAG property: Direction; Detail: Data flows one way - spout to bolts Join with external databases (enrich with user profile) DAG property: Acyclic; Detail: No loops - each tuple processed once per path Invoke ML models (fraud scoring, recommendation) DAG property: Distributed; Detail: Components run in parallel across cluster nodes DAG property: Scalable; Detail: Add more bolt instances to handle higher throughput Metric: Throughput; Storm capability: Millions of tuples per second across a cluster Metric: Latency; Storm capability: Sub-second, often milliseconds Storm organises spouts and bolts into a directed acyclic graph (DAG) that defines exactly how data flows: Metric: Fault tolerance; Storm capability: Automatic replay of failed tuples Metric: Scalability; Storm capability: Horizontal - add workers to scale Concept flow: Mobile app - IoT sensors - Database CDC - Topic: events - Spout reads from Kafka - Bolt: enrich - Bolt: aggregate/score - Bolt: action - Fraud block Storm is designed for high throughput and ultra-low latency: Layer: Ingestion; Tool: Kafka producers; Role: Push events to topics Layer: Storage; Tool: Kafka brokers; Role: Durable, replicated event log Layer: Processing; Tool: Storm topology; Role: Transform events into intelligence Layer: Action; Tool: Downstream systems; Role: Business decisions in real time High-frequency trading (microsecond decisions) Real-time network monitoring (instant anomaly detection)

Must remember:

- Apache Storm transforms raw Kafka events into real-time intelligence
- Runs topologies (permanent processing graphs), not batch jobs
- Spouts = stream sources (read from Kafka); Bolts = processing (filter, aggregate, score)
- Topology is a distributed DAG - data flows spout → bolts → output
- Ultra-low latency: millions of tuples/sec, sub-second (often millisecond) response
- Ideal for: high-frequency trading, fraud detection, network monitoring
- Kafka + Storm = complete pipeline: durable storage + real-time processing
- Storm processes; Kafka stores - complementary, not competing tools

Example or contrast: Property: Job type; Hadoop (MapReduce): Batch jobs that finish; Apache Storm: Topologies that run forever Property: Data; Hadoop (MapReduce): Bounded datasets; Apache Storm: Unbounded streams Property: Latency; Hadoop (MapReduce): Minutes to hours; Apache Storm: Milliseconds

Exam trap: Calling Storm topologies "jobs" - topologies run permanently until killed; Hadoop jobs finish Confusing spouts with bolts - spouts ingest/read data; bolts process/transform data

BD-W13-T08 Module Summary: Stream Processing Systems

Tier C

Definition and why it matters: Three drivers: velocity is the new volume, operational efficiency through immediate intervention, and competitive advantage from instant response

Core explanation: Era: Historical; Paradigm: Batch processing; Question: What happened yesterday?; Latency: Hours to days Era: Modern; Paradigm: Stream processing; Question: What is happening now?; Latency: Milliseconds to seconds

Must remember:

- Shift from batch ("what happened?") to stream ("what is happening now?")
- Streams: low latency, unbounded data, actionability
- Windowing for time-bounded metrics; global state for persistent context
- Kafka = durable backbone; Storm = real-time processing (spouts + bolts)
- Partitioning = parallelism; Replication = fault tolerance - different purposes
- One partition = one task for peak cluster performance
- Hash partitioning for point lookups; range partitioning for analytical queries
- Custom partitioners fix data skew and straggler problems
- Real-time intelligence defines modern industry leaders in finance, e-commerce, and infrastructure

Example or contrast: Domain: Financial services; Application: Real-time fraud detection; Key mechanism: Instant scoring + global state

Exam trap: Confusing partitioning with replication - partitioning = parallelism; replication = fault tolerance Assuming one partitioning strategy fits all workloads - hash for point lookups, range for analytical queries

Week 13 Exam Lens

Likely questions

- Define and explain Processing for Velocity: What Is a Stream?; include the mechanism and one limitation.
- Compare Processing for Velocity: What Is a Stream? with Technical Differences: Windowing vs Global State and state when each is preferred.
- Apply Kafka as the Streaming Backbone to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Treating streaming as just "faster batch" - streams are architecturally different (unbounded data, state management, messaging systems) Calling batch data "unbounded" - batch data is bounded (has start and end); streams are unbounded Using tumbling windows for per-user session analysis - session windows are activity-based, not time-fixed

Rapid Revision Master Sheet

Week 1: Big Data Constraints and Scaling. The Three Vs of Big Data: Volume, Velocity, and Variety; Why Bigger Boxes Eventually Fail; Vertical Scaling (Scale Up): Deep Dive; Horizontal Scaling (Scale Out): Deep Dive; Economics of Commodity Hardware.

Week 2: Distributed Systems Challenges. The Fallacies of Distributed Computing; Network Partitions and Partition Tolerance; The CAP Theorem: Consistency, Availability, and Partition Tolerance; CP vs AP Systems: Choosing the Right Trade-Off; ACID Properties in Distributed Contexts.

Week 3: The MapReduce Programming Model. MapReduce Programming Model: Introduction; The Map Operation: Transforming Raw Data; The Reduce Operation: Aggregating State; MapReduce Data Flow: Input Splits and Map Phase; Shuffle and Sort: The Bridge from Map to Reduce.

Week 4: Batch Processing Architectures: Hadoop and Spark. Hadoop vs Spark: The Shift from Disk-Bound to In-Memory Batch Processing; HDFS Deep Dive: Blocks, Replication, and Rack-Aware Fault Tolerance; MapReduce Data Flow on HDFS: From Blocks to Final Output; High I/O Latency and Materialisation Cost: Hadoop's Performance Bottleneck; Introduction to In-Memory Computing: RAM as Spark's Primary Workspace.

Week 5: Spark Core I: RDD Fundamentals. Resilient Distributed Datasets (RDDs): Definition and Anatomy; RDD Resilience and Distribution: Lineage vs Replication; RDD Immutability: Why Spark Data Never Changes; RDD Partitioning: The Atomic Unit of Parallelism in Spark; Data Locality and the Spark Cluster: Move Code, Not Data.

Week 6: Spark Core II: Execution Model. Spark Execution Engine: Architecture Overview; Lazy Evaluation: Why Spark Waits; Benefits of Lazy Evaluation: Optimisation and Pipelining; Building the Directed Acyclic Graph (DAG); Breaking the DAG into Stages and Tasks.

Week 7: Resilience I: Lineage Mechanisms. Challenges of Distributed Memory Recovery; Anatomy of an RDD Lineage Graph; RDD Lineage Graph Walkthrough: Sales Analysis by Category; Recovery in Narrow Dependencies: Fast, Local, Isolated; The Complexity of Wide Dependency Failures.

Week 8: Resilience II: Checkpointing. When Lineage Becomes a Liability; Stack Overflow and Performance Degradation from Deep Lineage; Breaking the Family Tree: Strategic Truncation via Checkpointing; Caching vs Checkpointing: Two Critical Resilience Mechanisms; Writing Checkpoints to Reliable Storage: HDFS and S3.

Week 9: Data Partitioning I: Strategies and Structures. Why Partitioning Is the Secret to Horizontal Scale; Partitioning vs Replication: Complementary but Distinct; How Partitions Enable Parallel Execution; Hash Partitioning Mechanics: The Modulo Operator and Uniformity; Use Cases: When to Choose Hash Partitioning.

Week 10: Data Partitioning II: Optimization and Skew. Advanced Data Partitioning: Optimisation and Skew; Uniform Distribution versus Data Skew in Distributed Clusters; Identifying Data Skew and Hotspots in Production Pipelines; Salting: Breaking Up Skewed Partitions for Balanced Execution; The Shuffle Problem in Large-Scale Joins.

Week 11: Distributed ML I: Concepts and Algorithms. Distributed Machine Learning: Concepts and Algorithms Overview; Communication Overhead and Data Movement in Distributed ML; Synchronisation, Stragglers, and Convergence in Distributed Training; Data Parallelism: Sharding the Dataset Across Workers; Model Parallelism: Splitting the Network Across Devices.

Week 12: Distributed ML II: Architectures. Architecting for Distributed Intelligence; Centralised Learning: The Parameter Server Model; Decentralised Learning: The Ring All-Reduce Strategy; Synchronous Training: Consistency and the Straggler Problem; Asynchronous Training: Speed vs Convergence Quality.

Week 13: Stream Processing Systems. Processing for Velocity: What Is a Stream?; Technical Differences: Windowing vs Global State; Kafka as the Streaming Backbone; Real-Time Topology with Apache Storm.

Formula and Framework Recall

- 2,000 standard servers =
- Total cluster cost $\sim n \times \text{cost per commodity node}$
- Big data cluster $\Rightarrow$ Partitions are inevitable $\Rightarrow P$ is required
- During partition: C uparrow $\Rightarrow$ A downarrow or A uparrow $\Rightarrow$ C downarrow
- 1000 in New York | Balance updated to
- 100 | Transaction 1 locks, reads
- 100 | Transaction 1 deducts $\rightarrow$ balance =
- No new writes $\Rightarrow$ All replicas converge to the same value
- LWW = $\text{argmax}_w (w \text{ in conflicts}) t_{\text{wall-clock}}(w)$
- During partition: choose CP or AP - not both
- Input $x \rightarrow (\text{map})$ Intermediate pairs $x \rightarrow (\text{shuffle/sort})$ Grouped values $x \rightarrow (\text{reduce})$ Output
- map: $(\text{key}, \text{value}) \rightarrow [(\text{key}_1, \text{value}_1), (\text{key}_2, \text{value}_2), \dots]$
- reduce: $(\text{key}, [\text{value}_1, \text{value}_2, \dots]) \rightarrow (\text{key}, \text{result})$
- Output: $(\text{texttt{/home.html}}, 10000)$
- lceil file size / split size rceil

- $\text{reducer_id} = \text{hash}(\text{key}) \bmod R$
- $\text{map}(\log \text{ line}) \rightarrow (\text{URL}, 1)$
- $\text{reduce}(\text{URL}, [1, 1, \dots, 1]) \rightarrow (\text{URL}, \text{total count})$
- Cluster of 1000 nodes $\Rightarrow$ Multiple failures per day are expected
- Physical disk writes = 3 x logical intermediate data size
- Resilience $\uparrow$ $\rightarrow$ Speed $\downarrow$ (for disk-bound workloads)
- Efficiency proportional to $(\text{Transfer Time}) / (\text{Seek Time} + \text{Transfer Time})$
- HDFS read $\rightarrow$ local disk write $\rightarrow$ network shuffle $\rightarrow$ HDFS write
- Iteration k: Read data $\rightarrow$ Compute $\rightarrow$ Write results $\rightarrow$ Repeat
- Total I/O Cost $\sim N_{\text{iterations}} \times (\text{Read Cost} + \text{Write Cost})$
- Map $\rightarrow$ Shuffle $\rightarrow$ Reduce
- $\text{centroid}_j^{(t+1)} = (1/|C_j|) \sum_{x \in C_j} x$
- Read HDFS $\rightarrow$ Process $\rightarrow$ Write HDFS $\rightarrow$ Repeat
- RDD = $\underbrace{\text{Collection of objects}}_{\text{Dataset}} \times \underbrace{\text{Spread across nodes}}_{\text{Distributed}} \times \underbrace{\text{Self-healing via lineage}}_{\text{Resilient}}$
- Fault tolerance cost = $N_{\text{copies}} \times \text{Storage size} \times \text{Network bandwidth for sync}$
- Recovery cost = Recompute lost partition only, not entire dataset
- premium partition = $\text{filter}_{(>100)}(\text{map}_{(-10)}(\text{base_prices partition}))$

Must-Draw and Must-Compare Sheet

Use these seven structures when a question asks you to explain, compare, design, or evaluate. Draw the skeleton first, label every arrow or decision axis, then use the prose answer to explain why each element exists and what can fail.

- CAP decision triangle: consistency, availability, and partition tolerance with CP/AP choices.
- MapReduce flow: split → map → shuffle/sort → reduce → output, including where disk and network cost occur.
- Spark execution map: transformations → DAG → stages → tasks, with narrow and wide dependencies.
- RDD recovery decision: lineage recomputation versus cache, checkpoint, replication, and reliable source storage.
- Partition-skew remedies: hash/range/custom partitioning, salting, broadcast join, and co-location.
- Distributed training comparison: parameter server versus ring all-reduce, including communication bottlenecks.
- Streaming topology: producers → Kafka → stream processor → window/state → sink/dashboard.

Scoring method: A diagram earns marks only when the labels are explained. After drawing, state the governing assumption, one metric or formula, one realistic example, and one limitation or trade-off. For a comparison, keep the dimensions parallel: purpose, data/input, mechanism, speed/cost, failure mode, and best-fit situation.

Final 30-Minute Answer Checklist

- Define the exact term before describing tools or examples.
- State the causal mechanism in a clear sequence; label each stage.
- Write the relevant formula, define its variables, and interpret the result.
- Compare alternatives on assumptions, cost, latency, risk, or customer value as appropriate.
- Use one realistic example and explicitly connect it to the concept.
- Close with a limitation, failure mode, ethical concern, or decision rule.
- Check that the answer addresses the command word: define, explain, compare, calculate, design, or evaluate.